Table of Contents (Jupyter)

LEGEND

These are insights got from analysing the data, in Pink color

''' ''' code sections are notes to self, some quick code snippets, or some notes while doing analysis

In [ ]:

GITHUB LINK FOR EASY VIEWING OF .ipynb NOTEBOOK

**Please note, when converting to PDF, the links in TOC stop working. You may like to see the original report's .ipynb file. Please see it on my Github Repository here **

- HTML Page
  - https://github.com/amitguptaforwork/dataanalytics_portfolio_projects/blob/main/CaseStudyAdEase_TimeSeriesSARIMAXProphet/Ad
- JupyterNotebook
  - https://github.com/amitguptaforwork/dataanalytics_portfolio_projects/blob/main/CaseStudyAdEase_TimeSeriesSARIMAXProphet/Ad

1.0 Understanding the Problem and the Data

About AdEase

Ad Ease is an ads and marketing based company helping businesses elicit maximum clicks @ minimum cost. They provide ad

infrastructure to help businesses promote themselves easily, effectively, and economically. The interplay of 3 AI modules - Design, Dispense, and Decipher, come together to make it this an end-to-end 3 step process digital advertising solution for all.

Objective of this Exercise

The **objective of this problem** is to use historical Wikipedia page-view data to **forecast future traffic**

- We have page view data for each page
- User is interested in placing an ad based on a language or region. They want to know how their ads will perform on pages in different languages.
- Thus the goal of the case study is to predict the page view per segment (segment could be language or language + some more criteria, which will be determined during the course of the study)
- Final aim of user is to answer this question
  - **If I place my ad on French pages, how many page views can I expect**,
  - **If I place my ad on English pages, how many page views can I expect**

2.0 Import and Inspect the Data

In [109…
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# plt.rcParams['figure.figsize'] = (20, 8) sets a global default figure size for all Matplotlib plots in the cu
# rcParams = runtime configuration parameters for Matplotlib
plt.rcParams['figure.figsize'] = (20,8)
```

In [109…
```
'''
Notes to Self
During this step, looking into the statistics is critical to gain initial know-how of its structure, variable k

'''
```

Out[109…
```
' \nNotes to Self\nDuring this step, looking into the statistics is critical to gain initial know-how of its st
ructure, variable kinds, and capability issues.\n\n'
```

In [109…
```
df =pd.read_csv("train_1.csv", encoding="utf-8") # we need to use utf-8 as we have data not just in English but
df.columns
```

Out[109…
```
Index(['Page', '2015-07-01', '2015-07-02', '2015-07-03', '2015-07-04',
       '2015-07-05', '2015-07-06', '2015-07-07', '2015-07-08', '2015-07-09',
       ...
       '2016-12-22', '2016-12-23', '2016-12-24', '2016-12-25', '2016-12-26',
       '2016-12-27', '2016-12-28', '2016-12-29', '2016-12-30', '2016-12-31'],
      dtype='object', length=551)
```

Split the Page column to create more columns

In [109…
```
df[['page_title', 'lang_domain', 'access_type', 'access_origin']] = \
    df['Page'].str.rsplit('_', n=3, expand=True)

df[['language', 'domain']] = \
    df['lang_domain'].str.split('.', n=1, expand=True)

df.drop(columns='lang_domain', inplace=True)

df.head()
```

Out[109…

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2016-12-27	2016-12-28	2016-12-29	201 12-
0	2NE1_zh.wikipedia.org_all-access_spider	18.0	11.0	5.0	13.0	14.0	9.0	9.0	22.0	26.0	...	20.0	22.0	19.0	18
1	2PM_zh.wikipedia.org_all-access_spider	11.0	14.0	15.0	18.0	11.0	13.0	22.0	11.0	10.0	...	30.0	52.0	45.0	26
2	3C_zh.wikipedia.org_all-access_spider	1.0	0.0	1.0	1.0	0.0	4.0	0.0	3.0	4.0	...	4.0	6.0	3.0	4
3	4minute_zh.wikipedia.org_all-access_spider	35.0	13.0	10.0	94.0	4.0	26.0	14.0	9.0	11.0	...	11.0	17.0	19.0	10
4	52_Hz_I_Love_You_zh.wikipedia.org_all-access_s...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	11.0	27.0	13.0	36

5 rows × 556 columns

2.2 Statistical summary for numerical columns and Unique values for categorical data

- Instead of manually creating tables and collecting values lets automate the table creation
- We will generate a table of all columns in Markdown format
- We will paste this table into a markdown cell and then we may add additional analysis

In [ ]:
```
#!pip install tabulate
from tabulate import tabulate
def generate_markdown_table_with_details(df):
    column_details = []
    for col in df.columns:
        dtype = str(df[col].dtype)

        if dtype == 'object' or dtype=='category':
            cat_or_num = 'Categorical'
            details = df[col].unique().tolist()
            if(len(details)>10):
                details= f'{df[col].nunique()} unique values.Ex.{",".join([str(x) for x in details[:3]])}'

        else:
            cat_or_num = 'Numerical'
            details = f"Min: {df[col].min()}, Max: {df[col].max()}"
        column_details.append([col, dtype, cat_or_num, details, ''])

    # Create a DataFrame for the table
    table_df = pd.DataFrame(column_details, columns=['Column Name', 'Data Type', 'Categorical or Numerical', 'D

    # Convert DataFrame to Markdown
    markdown_table = tabulate(table_df, headers='keys', tablefmt='pipe', showindex=False)
    return markdown_table

markdown_table = generate_markdown_table_with_details(df)
print(f'There are a total of {df.shape[0]:,} rows and {df.shape[1]} columns')
print(markdown_table)
```

2.3 Adding insights about columns and data size.

- We will copy the markdown table generated above and add our inputs by analysing the columns.
- We identify which columns are numerical, which are categorical
- Within categorical, we identify if it is nominal or ordinal data. This helps us to decide best graph to use
- Also ordinal data can be converted into numeric using codes thereby making it available for some kind of graphs.

2.4 Data Dictionary

There are a total of 145,063 rows and 556 columns

Column Name	Data Type	Categorical or Numerical	Details	Business Description(filled manually)
Page	object	Categorical	145063 unique values.Ex.2NE1_zh.wikipedia.org_all-access_spider,	Name of page along with other metadata like domain, language, access_type , access_origin
Date (550 Columns)	float64	Numerical	Min: 2015-07-01, Max: 2016-12-31	Click count from 550 consecutive days for each page
page_title	object	Categorical	49174 unique values.Ex.2NE1,2PM,3C	Page name
access_type	object	Categorical	['all-access', 'desktop', 'mobile-web']	How the content is accessed - all-access → aggregated access (desktop + mobile + others), desktop → desktop browsers only, mobile-web → mobile browsers (not apps)
access_origin	object	Categorical	['spider', 'all-agents']	Who is making the request - all-agents → normal users and bots combined , spider → automated crawlers (search engine bots like Googlebot, Bingbot)
language	object	Categorical	['zh', 'fr', 'en', 'commons', 'ru', 'www', 'de', 'ja', 'es']	Language
domain	object	Categorical	['wikipedia.org', 'wikimedia.org', 'mediawiki.org']	wikipedia.org → article text, wikimedia/commons → images, audio, video & media catalogues, mediawiki.org → software resources such as JavaScript, CSS, system messages, and technical documentation.

- 2,05,843 records

2.5 Data Cleaning

We can check for duplicate data

2.5.1 Check duplicate data

In [109…
```
#Lets check if we have duplicates.
df_duplicate  = df[df.duplicated(keep=False)].sort_values(by='Page')
print(f'We have {len(df_duplicate)*100/len(df)}% duplicate rows')
df_duplicate
```

```
We have 0.0% duplicate rows
```

Out[109…

Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2016-12-27	2016-12-28	2016-12-29	2016-12-30	2016-12-31	page_title	access_typ

0 rows × 556 columns

2.6 Handling Missing Values

Now let's check if there are any missing values in our dataset or not.

In [110…
```
'''
Notes to Self
Missing Data can also refer to as NA(Not Available) values in pandas. There are several useful functions for de

isnull()
notnull()
dropna()
fillna()
replace()
interpolate()
'''
```

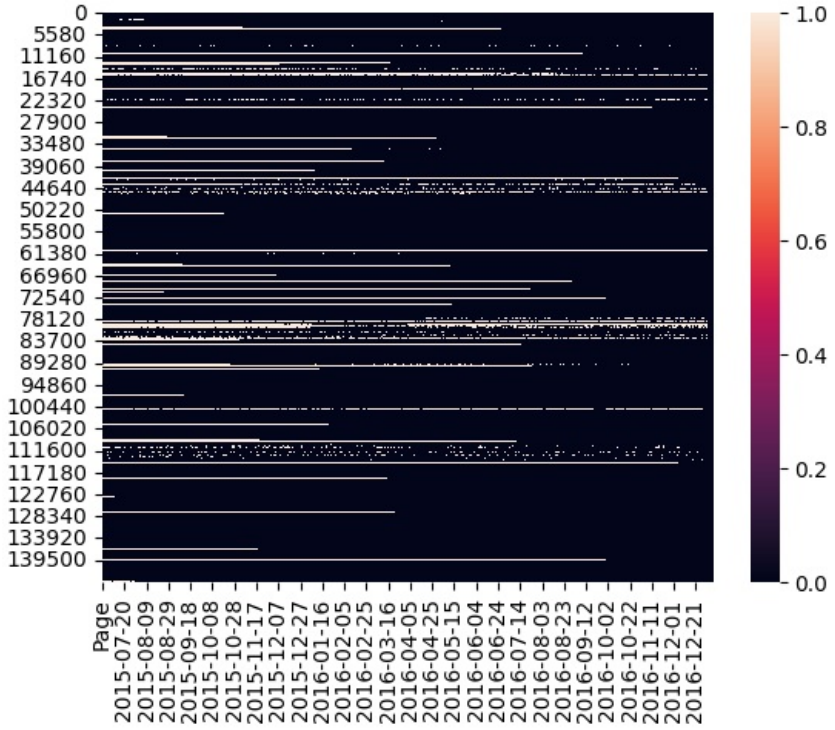
Out[110…
```
' \nNotes to Self\nMissing Data can also refer to as NA(Not Available) values in pandas. There are several usef
ul functions for detecting, removing, and replacing null values in Pandas DataFrame :\n\nisnull()\nnotnull()\nd
ropna()\nfillna()\nreplace()\ninterpolate()\n'
```

- Lets check if we have ALL dates, no dates are missing
- We will use column[1] and column[-6], the first and last date columns and
  - count how many days are there between them.
  - It should be same as the number of date columns.

In [110…
```
print(f'We expected {(pd.to_datetime(df.columns[-6])- pd.to_datetime(df.columns[1])).days+1} date columns which
```

```
We expected 550 date columns which is same as the given number of date columns.
```

In [110…
```
# sns.heatmap(df.isnull())
```



- The absense of vertical lines shows that there is no date where data from all pages is missing
- Continuous horizontal line means data is continuously missing. This may indicate that the page was created after that date.

This missing data is actually MNAR - missing not at random. Because we know the reason why its missing.

- Broken lines means data is missing in-between. This is genuine case of missing data and its MCAR- Missing completely at Random or MAR - missing at random.

Lets investigate the missing data

In [110…
```
#Create a new column in the dataframe that counts how many missing values are there in that row
df['NumberOfMissingValues'] = pd.DataFrame(df.isnull().sum(axis=1))
```

In [110…
```
totalRows = len(df)
print("Total rows = ",totalRows)
lenNoMissing = len(df.loc[df.NumberOfMissingValues==0])
print(f"Rows with no missing data for 550 days={lenNoMissing} ({round(lenNoMissing*100/totalRows,1)}%)")
lenSomeMissing = len(df.loc[df.NumberOfMissingValues>0])
print(f"Rows with atlease one missing data={lenSomeMissing} ({round(lenSomeMissing*100/totalRows,1)}%)")

lenAllMissing = len(df.loc[df.NumberOfMissingValues==550])
print(f"Rows with ALL 550 days missing data={lenAllMissing} ({round(lenAllMissing*100/totalRows,1)}%)")
```

```
Total rows =  145063
Rows with no missing data for 550 days=117277 (80.8%)
Rows with atlease one missing data=27786 (19.2%)
Rows with ALL 550 days missing data=652 (0.4%)
```

- Rows with no missing data for 550 days=117277 (80.8%)
- Rows with atlease one missing data=27786 (19.2%) - if we have data missing in between, we can fill using extrapolation.
- Rows with ALL 550 days missing data=652 (0.4%)- These rows can be dropped

Drop rows that have all 550 dates' data missing

In [110…
```
df.drop(df.loc[df.NumberOfMissingValues==550].index, inplace=True)
len(df)
```

Out[110… 144411

- From 1,45,063, 652 rows were dropped, and now we have 1,44,411 rows

2.6.2 Imputation of Dates

We have missing data on a row basis. So we will do this

Rule

- For each row:
  - Traverse left → right
  - Leading NaNs (before first valid value) → fill with 0

- After first valid value → interpolate missing values

In [110…
```
df.head()
```

Out[110…

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2016-12-28	2016-12-29	2016-12-30	201 12-
0	2NE1_zh.wikipedia.org_all-access_spider	18.0	11.0	5.0	13.0	14.0	9.0	9.0	22.0	26.0	...	22.0	19.0	18.0	2(
1	2PM_zh.wikipedia.org_all-access_spider	11.0	14.0	15.0	18.0	11.0	13.0	22.0	11.0	10.0	...	52.0	45.0	26.0	2(
2	3C_zh.wikipedia.org_all-access_spider	1.0	0.0	1.0	1.0	0.0	4.0	0.0	3.0	4.0	...	6.0	3.0	4.0	17
3	4minute_zh.wikipedia.org_all-access_spider	35.0	13.0	10.0	94.0	4.0	26.0	14.0	9.0	11.0	...	17.0	19.0	10.0	1(
4	52_Hz_I_Love_You_zh.wikipedia.org_all-access_s...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	27.0	13.0	36.0	1(

5 rows × 557 columns

In [110…
```
df.drop(columns=['NumberOfMissingValues'],inplace=True)
```

In [110…
```
def processRowNans(row):
    row = row.copy()
    first_valid_idx = row.first_valid_index()

    if first_valid_idx is None:
        return row.fillna(0)

    row.loc[:first_valid_idx] = row.loc[:first_valid_idx].fillna(0)
    row.loc[first_valid_idx:] = row.loc[first_valid_idx:].interpolate(
        method='linear',
        limit_direction='forward'
    )

    return row

cat_cols = df.select_dtypes(include='object').columns
num_cols = df.select_dtypes(exclude='object').columns

df[num_cols] = df[num_cols].apply(processRowNans, axis=1)
```

Lets check if any nulls now.

In [110…
```
# PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling frame.insert many t
# which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead.
# To get a de-fragmented frame, use newframe = frame.copy()
df = df.copy()


df['NumberOfMissingValues'] = pd.DataFrame(df.isnull().sum(axis=1))
totalRows = len(df)
print("Total rows = ",totalRows)
lenNoMissing = len(df.loc[df.NumberOfMissingValues==0])
print(f"Rows with no missing data for 550 days={lenNoMissing} ({round(lenNoMissing*100/totalRows,1)}%)")
```

```
Total rows =  144411
Rows with no missing data for 550 days=144411 (100.0%)
```

- Total rows = 144411
- Rows with no missing data for 550 days=144411 (100.0%)

In [111…
```
df.drop(columns=['NumberOfMissingValues'],inplace=True)
```

2.7 Creation of Helper functions

- The analysis we did above will be put in function form so that we can process data in a pipeline.

`ingesterCleanerImputerEngineer()`: Helper function for ingestion

1. Function will read a CSV
2. Split Page into columns
3. Drop empty rows
4. Impute null values in rows that have some data
5. Return dataframe

In [111…

```
def ingesterCleanerImputerEngineer(filepath):
    l_df =pd.read_csv(filepath)

    #Split the Page column to create more columns
    l_df[['page_title', 'lang_domain', 'access_type', 'access_origin']] = \
    l_df['Page'].str.rsplit('_', n=3, expand=True)

    l_df[['language', 'domain']] = \
        l_df['lang_domain'].str.split('.', n=1, expand=True)

    l_df.drop(columns='lang_domain', inplace=True)

    # Lets investigate the missing data
    #Create a new column in the dataframe that counts how many missing values are there in that row
    l_df['NumberOfMissingValues'] = pd.DataFrame(df.isnull().sum(axis=1))
    totalRows = len(l_df)
    print("Total rows = ",totalRows)
    lenNoMissing = len(l_df.loc[l_df.NumberOfMissingValues==0])
    print(f"Rows with no missing data for 550 days={lenNoMissing} ({round(lenNoMissing*100/totalRows,1)}%)")
    lenSomeMissing = len(l_df.loc[l_df.NumberOfMissingValues>0])
    print(f"Rows with atlease one missing data={lenSomeMissing} ({round(lenSomeMissing*100/totalRows,1)}%)")

    lenAllMissing = len(l_df.loc[l_df.NumberOfMissingValues==550])
    print(f"Rows with ALL 550 days missing data={lenAllMissing} ({round(lenAllMissing*100/totalRows,1)}%)")

    # Drop rows that have all 550 dates' data missing
    l_df.drop(l_df.loc[l_df.NumberOfMissingValues==550].index, inplace=True)
    print('After dropping empty rows',len(l_df))
    l_df.drop(columns=['NumberOfMissingValues'],inplace=True)


    # Imputation of Dates
        #   We have missing data on a row basis.  So we will do this

        # **Rule**
        # - For each row:
        # - Traverse left → right
        # - Leading NaNs (before first valid value) → fill with 0
        # - After first valid value → interpolate missing values
    print("Now imputing missing data...pl wait for sometime")
    cat_cols = l_df.select_dtypes(include='object').columns
    num_cols = l_df.select_dtypes(exclude='object').columns

    l_df[num_cols] = l_df[num_cols].apply(processRowNans, axis=1)
    l_df = l_df.copy()
    print("Imputation completed.")
    return l_df

df_with_extra_columns= ingesterCleanerImputerEngineer("train_1.csv")
df_with_extra_columns.head()
```

```
Total rows =  145063
Rows with no missing data for 550 days=144411 (99.6%)
Rows with atlease one missing data=0 (0.0%)
Rows with ALL 550 days missing data=0 (0.0%)
After dropping empty rows 145063
Now imputing missing data...pl wait for sometime
Imputation completed.
```

Out[111…

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2016-12-27	2016-12-28	2016-12-29	201 12-
0	2NE1_zh.wikipedia.org_all-access_spider	18.0	11.0	5.0	13.0	14.0	9.0	9.0	22.0	26.0	...	20.0	22.0	19.0	18
1	2PM_zh.wikipedia.org_all-access_spider	11.0	14.0	15.0	18.0	11.0	13.0	22.0	11.0	10.0	...	30.0	52.0	45.0	26
2	3C_zh.wikipedia.org_all-access_spider	1.0	0.0	1.0	1.0	0.0	4.0	0.0	3.0	4.0	...	4.0	6.0	3.0	4
3	4minute_zh.wikipedia.org_all-access_spider	35.0	13.0	10.0	94.0	4.0	26.0	14.0	9.0	11.0	...	11.0	17.0	19.0	10
4	52_Hz_I_Love_You_zh.wikipedia.org_all-access_s...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	11.0	27.0	13.0	36

5 rows × 556 columns

`search_data()`: Extract rows from a DataFrame based on specified column-value conditions.

- This will help us to select rows, say we want to select all rows with pages in English, or all English rows not accessed via spider
- This function supports passing any number of search criteria by way of **kwargs
- It also provides flexibility to select some random rows or all rows.

- This is useful because we use this function sometimes to just sample some rows matching a criteria and plotting them (used during EDA)
- And sometimes to select ALL rows matching a criteria (we use this to select data for our various segments later in the study)

In [111…

```python
#Only one search criteria
def search_dataV1(sampleSize=10, colName=None, colValue=None ):
    if colName is not None:
        msg = f"{sampleSize} random rows where {colName} == {colValue}"
        selectedRows = df_with_extra_columns.loc[df_with_extra_columns[colName]==colValue].sample(sampleSize)
    else:
        msg =f"{sampleSize} random rows from entire data"
        selectedRows = df_with_extra_columns.sample(sampleSize)
    print(f"Extracting ",msg)
    return selectedRows,msg

# More flexible version.  This support infinite search conditions (all equalities though, no negation)
def search_data(sampleSize=10, **kwargs):
    """
    Extract random rows from a DataFrame based on specified column-value conditions.

    Parameters:
    -----------
    sampleSize : int, optional
        The number of random rows to return. Default is 10.
        -1 = All rows

    **kwargs : keyword arguments
        Multiple pairs of column names and their corresponding values to filter the DataFrame.
        Each column name should be prefixed with 'colName' followed by an index (e.g., 'colName1').
        The corresponding values should be provided as 'val' followed by the same index (e.g., 'val1').

        For example:
        - colName1='column_a' and val1=5 will filter rows where column_a equals 5.
        - You can specify multiple conditions:
          - colName1='column_a', val1=5, colName2='column_b', val2=10

    Returns:
    --------
    selectedRows : DataFrame
        A random sample of rows that meet the specified conditions.

    msg : str
        A message indicating the filtering criteria used for extraction.

    Example:
    --------
    # Suppose df_with_extra_columns is your DataFrame.
    selected_data, message = search_data(10, colName1='age', val1=30, colName2='city', val2='New York')

    This would return 10 random rows from `df_with_extra_columns` where 'age' is 30 and 'city' is 'New York'.

    Notes:
    ------
    - If no column-value pairs are provided in `kwargs`, the function will return a random sample from the enti
    - Ensure that the DataFrame is named `df_with_extra_columns` before calling this function.

    """
    if(sampleSize==-1):
        sampleSizeStr="All"
    else:
        sampleSizeStr=f'random'
    # Initialize the query string and parameters dictionary
    query_conditions = []
    titleDict={}
    # Loop through the kwargs to create conditions for the query
    for key, value in kwargs.items():
        # print(key,value)
        if key.startswith('colName') and value is not None:
            # Get the corresponding val parameter
            col_index = key[-1]  # Extract the index (1, 2, ...) from 'colName1', 'colName2', ...
            val_key = f'val{col_index}'
            col_value = kwargs.get(val_key)
            # print(value,col_value)
            if col_value is not None:
                # Add to the query conditions
                query_conditions.append(f"({value} == '{col_value}')")  # Using @ to reference variable in `que
                titleDict[value]=col_value
                # print("added======",titleDict)

    # Construct the final query string

    if query_conditions:
```

```
        query_string = ' & '.join(query_conditions)
        # print(query_string)
        if(sampleSize!=-1):
            selectedRows = df_with_extra_columns.query(query_string).sample(sampleSize)
        else:
            selectedRows = df_with_extra_columns.query(query_string)
        msg = f"{sampleSizeStr} {len(selectedRows)} rows where " + ' and '.join(f"{key} == {val}" for key, val
    else:
        if(sampleSize!=-1):
            selectedRows = df_with_extra_columns.sample(sampleSize)
        else:
            selectedRows = df_with_extra_columns
        msg = f"{sampleSizeStr} {len(selectedRows)} rows from entire data"

    print(f"search_data():Searched", msg)
    return selectedRows, msg

#Trial run to see how the data looks like.
rows,title=search_data(5,
                       colName1='language',val1='www',
                       colName2='access_origin',va12='spider',
                       colName3='access_type',val3='desktop')
rows
```

```
search_data():Searched random 5 rows where language == www and access_type == desktop
```

Out[111…

	Page	2015-07-01	2015-07-02	2015-07-03	2015-07-04	2015-07-05	2015-07-06	2015-07-07	2015-07-08	2015-07-09	...	2016-12-27
43712	Special:WhatLinksHere/Help:Categories/ja_www.m...	1.0	1.0	1.0	1.0	1.0	1.0	1.125	1.25	1.375	...	1.0
42464	How_to_contribute/an_www.mediawiki.org_desktop...	33.0	40.0	63.0	87.0	70.0	72.0	58.000	76.00	42.000	...	48.0
43494	MediaWiki:Gadget-UTCLiveClock.js_www.mediawiki...	1.0	6.0	4.0	2.0	7.0	4.0	15.000	3.00	2.000	...	6.0
43010	API:Protect_www.mediawiki.org_desktop_all-agents	15.0	28.0	15.0	15.0	15.0	27.0	24.000	22.00	18.000	...	18.0
43739	Topic:Szhapu3bcbotvq3l_www.mediawiki.org_deskt...	0.0	0.0	0.0	0.0	0.0	0.0	0.000	0.00	0.000	...	2.0

5 rows × 556 columns

`readExoData()` : Helper function to read exogeneous data

In [111…

```
def readExoData(filepath):
    print(f"readExoData():Reading {filepath}")
    return pd.Series(pd.read_csv(filepath, header='infer').iloc[:, 0].values)
exog_campaign_series = readExoData("Exog_Campaign_eng")
```

```
readExoData():Reading Exog_Campaign_eng
```

In [111…

```
len(exog_campaign_series.values)
```

Out[111… 550

`addExogeneousColumnToADataFrame()`

We pass a dataframe and a series containing exogeneous column's data, it returns a new dataframe with exogeneous column added to the dataframe

In [111…

```
def addExogeneousColumnToADataFrame(data, exogSeries,exogSeriesName):
        print("addExogeneousColumnToADataFrame():Adding the exogeneous variable (campaign data) as a column")
        new_df = data.copy()
        # This is the fastest method
        #     No index alignment => .values avoids expensive index matching =>
        #     Pure NumPy assignment => Pandas internally performs a vectorized copy
        #     Memory layout is contiguous → cache-friendly
        #     O(n) operation
        new_df[exogSeriesName] = exogSeries.values
        return new_df
```

`extractRowTransposeColumn()` : Select row, transpose

Helper function to convert one seleted row into a transposed dataframe

In [111…

```
# rowIndexToExtract=-1 means extract all rows from given dataframe
def extractRowTransposeColumn(wide_data_with_extra_columns = df_with_extra_columns, rowIndexToExtract=-1,**kwar
    # rowIndexToExtract=-1 means extract all rows from given dataframe
    datasetName = kwargs.get("modelName","given dataframe")
    if rowIndexToExtract!=-1:
```

```
        #0th column is Page
        #Last 5 columns are strings, that were created as part of analysis.
        #We want to remove these columns and get only the date columns
        #We also transpose the data so that dates come in rows.
        print(f"extractRowTransposeColumn():Transposing row {rowIndexToExtract} from {datasetName}")
        row_df = wide_data_with_extra_columns.iloc[rowIndexToExtract, 1:-5].T.to_frame()
        row_df.columns=['clickCount'] #we rename the column
    else:
        print(f"extractRowTransposeColumn():Transposing ALL rows from {datasetName} that has {len(wide_data_wit
        row_df = wide_data_with_extra_columns.iloc[:, 1:-5].T

    #To suppress SARIMAX error- ValueError: Pandas data cast to numpy dtype of object
    #The clickCount data is read as an object.  SARIMAX needs it as a int
    #Using apply makes it possible to work on multiple columns if available.
    row_df = row_df.apply(pd.to_numeric, errors="coerce")

    #To suppress SARIMAX warning - ValueWarning: No frequency information was provided, so inferred frequency D
    #This avoids NaNs but only works if there are no missing days.  As we have already ensured that in this pro
    #so we can use this approach.
    #Setting .index.freq does nothing unless the index is first converted to a DatetimeIndex.
    #The transpose got us the dates as index, but pandas still does not consider it DatetimeIndex
    row_df.index = pd.to_datetime(row_df.index)
    row_df.index.freq = "D" # sets metadata ONLY, no reindexing


    return row_df
```

`plot_data_staggered()` :Creates a staggered plot for click counts of multiple pages with optional campaign indicators.

In [111…

```
def plot_transposed_data_staggered(transposed_data, title, staggerByY=2000, campaign=False, **kwargs):
    """
    Creates a staggered plot for click counts of multiple pages with optional campaign indicators.

    Parameters:
    -----------
    data : DataFrame
        A pandas DataFrame containing the click counts for multiple pages.

    title : str
        The title for the plot.

    staggerByY : int, optional
        The value by which to stagger each subsequent plot for visibility. Default is 2000.

    campaign : bool, optional
        A flag indicating whether to show campaign days as vertical red lines on the plot. Default is False.

    **kwargs : keyword arguments
        Additional parameters that can modify the plot behavior:
        - ylim : float, optional
            The upper limit for the y-axis. If not provided, it is set to twice the maximum staggering value.

    Returns:
    --------
    None

    Notes:
    ------
    - The function modifies the input DataFrame by adding an extra column for campaign data.
    - Y-ticks are suppressed for better readability since the data is staggered to avoid overlap.
    - The average click count for each page is displayed as text above its respective plot line.

    Example:
    --------
    import pandas as pd

    # Sample data creation
    df = pd.DataFrame({
        'page1': [100, 200, 300],
        'page2': [150, 250, 350],
        'campaign': [1, 0, 1]
    })

    plot_data_staggered(data=df, title='Click Counts Comparison', campaign=True)

    This example will plot the click counts for 'page1' and 'page2', staggered by 2000,
    and indicate campaign days with red vertical lines.

    """
    cols = transposed_data.columns.drop("campaign",errors="ignore")
```

```
    increments = staggerByY * (np.arange(len(cols)))
    transposed_incremented_data = transposed_data.copy()
    transposed_incremented_data[cols] = transposed_data[cols] + increments

    if 'campaign' in transposed_data.columns:
        # Plot except the campaign column
        transposed_incremented_data.iloc[:, :-1].plot()
    else:
        transposed_incremented_data.plot()


    # If campaign is True, add vertical lines
    if campaign and 'campaign' in transposed_data.columns:
        rowIndicesWithcampaignTrue = transposed_incremented_data.loc[transposed_incremented_data.campaign == 1]
        for day in rowIndicesWithcampaignTrue:
            plt.axvline(x=day, color='#FA8072')

    ylim = kwargs.get("ylim", 2 * increments[-1])
    plt.ylim(0, ylim)

    # Suppress y-ticks as they dont really make sense, as we have staggered the
    plt.yticks([])
    plt.ylabel(f"Plot of {len(cols)} pages' clickcount shown one over the other for comparison")
    plt.title(f'{title}{" . Campaign days shown as vertical red lines" if campaign else ""}')
    # Specify the desired date for x-axis annotation
    # Calculate and display the averages
    for i, col in enumerate(cols):
        avg_value = transposed_data[col].mean()
        median_value = transposed_data[col].median()

        # Positioning the text above the positioning of the corresponding plot
        plt.text(x= 16620.0,  # Adjust this value if necessary
                y= 300+increments[i],  # Add increment to avoid overlap
                s=f'ClickCount Avg: {avg_value:.2f} Median: {median_value:.2f}',
                ha='left', va='center', fontsize=8, color='black') # Adjust font size and color as needed

    plt.show()

def plot_data_staggered(data, title, staggerByY=2000, campaign=False, **kwargs):
    transposed_data = extractRowTransposeColumn(data)
    plot_transposed_data_staggered(transposed_data, title, staggerByY=staggerByY, campaign=campaign, **kwargs)
```

`prepare_time_series()`

Convert wide-format data to time series. This is THE time series we are trying to predict in this case study

In [111…

```
def prepare_time_series(segment_data,modelName):
    """Convert wide-format data to time series"""
    ts_df = extractRowTransposeColumn(segment_data,modelName=modelName).sum(axis=1).to_frame()
    ts_df.columns=['clickCount']
    return ts_df

# daily_totals = prepare_time_series(modelDict[modelNames[3]]['data'], modelName=modelNames[3])
# daily_totals
```

`dataSegmenter()`

Purpose: Splits the original dataframe into separate segments based on access origin and language.

What it does:

- Queries the dataset using `search_data()` function
- Creates 10 different data segments stored in `modelDict`
- Each segment will later have its own trained model and MAPE score

Segments Created: (Note, this was revealed AFTER EDA. That knowledge has bubbled up and we are giving the segment names below)

Segment	Description
`ModelSpider`	Web crawler/bot traffic
`ModelNonSpiderLangCommons`	Human traffic - Wikimedia Commons
`ModelNonSpiderLangDe`	Human traffic - German Wikipedia
`ModelNonSpiderLangEn`	Human traffic - English Wikipedia
`ModelNonSpiderLangEs`	Human traffic - Spanish Wikipedia
`ModelNonSpiderLangFr`	Human traffic - French Wikipedia

`ModelNonSpiderLangJa`	Human traffic - Japanese Wikipedia
`ModelNonSpiderLangRu`	Human traffic - Russian Wikipedia
`ModelNonSpiderLangWww`	Human traffic - WWW subdomain
`ModelNonSpiderLangZh`	Human traffic - Chinese Wikipedia

Data Structure:

```
modelDict = {
    "ModelName": {
        # ===== DATA =====
        "data": DataFrame,              # Segmented data
        "aggregatedData": DataFrame,    # Total daily page views for this segment.
                                        #THIS IS THE TIME SERIES we are trying to model in this case
study

        # ===== MODELS TRIED =====
        "modelsTried": [
            {
                "modelName": "ARIMA",           # Model type
                "modelParams": (1, 0, 1),       # Order / hyperparameters
                "mape": 12.35,                  # MAPE score
                "comments": "Baseline from ACF/PACF analysis"
            },
            {
                "modelName": "ARIMA",
                "modelParams": (2, 1, 0),
                "mape": 10.21,
                "comments": "After differencing"
            },
            # ... more attempts
        ],

        # ===== BEST MODEL =====
        "bestModel": {
            "modelName": "ARIMA",
            "modelParams": (2, 1, 0),
            "mape": 10.21,
            "comments": "After differencing"
            "fittedModel": <model_object>,     # Optional: store fitted model
        },
    },
    ...
}
    ...
}
```

In [111…

```
modelNames = ['ModelSpider',
'ModelNonSpiderLangCommons',
'ModelNonSpiderLangDe',
'ModelNonSpiderLangEn',
'ModelNonSpiderLangEs',
'ModelNonSpiderLangFr',
'ModelNonSpiderLangJa',
'ModelNonSpiderLangRu',
'ModelNonSpiderLangWww',
'ModelNonSpiderLangZh']

#The main store of all our analysis
#This will be like our MLOps database.
modelDict={}
```

In [112…

```
def dataSegmenter():
    """
    Segments data and initializes modelDict with structure:
    - data: DataFrame
    - aggregatedData: DataFrame
    - modelsTried: []
    - bestModel: None
    """

    # Define segment configurations
    segments = [
        # (index, search_params)
        (0, {"colName1": "access_origin", "val1": "spider"}),
        (1, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "commons"}),
        (2, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "de"}),
```

```
        (3, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "en"}),
        (4, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "es"}),
        (5, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "fr"}),
        (6, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "ja"}),
        (7, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "ru"}),
        (8, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "www"}),
        (9, {"colName1": "access_origin", "val1": "all-agents", "colName2": "language", "val2": "zh"}),
    ]
    #This will store time series of all segments.  Will be useful during modelling and EDA
    timeSeriesFull_df = pd.DataFrame()
    for i, params in segments:
        rows, title = search_data(-1, **params)
        timeSeries_df = prepare_time_series(rows, modelName=modelNames[i])

        modelDict[modelNames[i]] = {
            "data": rows,
            "aggregatedData":timeSeries_df,
            "modelsTried": [],
            "bestModel": None
        }
        # Concatenate column-wise
        timeSeriesFull_df = pd.concat([timeSeriesFull_df, timeSeries_df.rename(columns={'clickCount': modelNames

    print(f"✅ Initialized {len(modelDict)} segments")
    return timeSeriesFull_df

#This will store time series of all segments.  Will be useful during modelling and EDA
timeSeriesFull_df = dataSegmenter()
```

```
search_data():Searched All 34913 rows where access_origin == spider
extractRowTransposeColumn():Transposing ALL rows from ModelSpider that has 34913 rows
search_data():Searched All 8005 rows where access_origin == all-agents and language == commons
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangCommons that has 8005 rows
search_data():Searched All 13960 rows where access_origin == all-agents and language == de
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangDe that has 13960 rows
search_data():Searched All 19177 rows where access_origin == all-agents and language == en
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangEn that has 19177 rows
search_data():Searched All 10532 rows where access_origin == all-agents and language == es
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangEs that has 10532 rows
search_data():Searched All 13302 rows where access_origin == all-agents and language == fr
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangFr that has 13302 rows
search_data():Searched All 15424 rows where access_origin == all-agents and language == ja
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangJa that has 15424 rows
search_data():Searched All 11280 rows where access_origin == all-agents and language == ru
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangRu that has 11280 rows
search_data():Searched All 5551 rows where access_origin == all-agents and language == www
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangWww that has 5551 rows
search_data():Searched All 12919 rows where access_origin == all-agents and language == zh
extractRowTransposeColumn():Transposing ALL rows from ModelNonSpiderLangZh that has 12919 rows
✅ Initialized 10 segments
```

In [112…
```
timeSeriesFull_df.head()
```

Out[112…

	ModelSpider	ModelNonSpiderLangCommons	ModelNonSpiderLangDe	ModelNonSpiderLangEn	ModelNonSpiderLangEs	Mo
2015-07-01	664648.000000	1.115089e+06	1.322512e+07	8.443447e+07	1.516379e+07	
2015-07-02	619718.174491	1.134739e+06	1.304097e+07	8.418241e+07	1.449318e+07	
2015-07-03	595253.378576	1.104986e+06	1.251964e+07	7.992619e+07	1.333192e+07	
2015-07-04	614089.965060	9.228855e+05	1.147857e+07	8.321397e+07	1.252267e+07	
2015-07-05	622593.359069	1.012272e+06	1.335621e+07	8.596247e+07	1.361801e+07	

`addModelAttempt()` -Adds results from trying a model on a data

This is MLOps implemented from scratch !

```
modelDict = {
    "ModelName": {
        # ===== DATA =====
        "data": DataFrame,              # Segmented data
        "aggregatedData": DataFrame,    # Total daily page views for this segment.
                                        #THIS IS THE TIME SERIES we are trying to model in this case study

        # ===== MODELS TRIED =====
        "modelsTried": [
            {
                "modelName": "ARIMA",           # Model type
                "modelParams": (1, 0, 1),       # Order / hyperparameters
                "mape": 12.35,                  # MAPE score
                "comments": "Baseline from ACF/PACF analysis"
            },
            {
                "modelName": "ARIMA",
                "modelParams": (2, 1, 0),
                "mape": 10.21,
                "comments": "After differencing"
            },
            # ... more attempts
        ],

        # ===== BEST MODEL =====
        "bestModel": {
            "modelName": "ARIMA",
            "modelParams": (2, 1, 0),
            "mape": 10.21,
            "comments": "After differencing"
            "fittedModel": <model_object>,      # Optional: store fitted model
        },
    },
```

This function adds this dictionary to the modelsTried array

Also updates the "bestModel", if MAPE of the new trial is lower

In [112…

```python
def addModelAttempt(modelDict, segmentName, modelName, modelParams, mape, comments="", verbose=False):
    """
    Add a model attempt to the modelsTried list for a segment.

    Parameters:
    -----------
    verbose : bool
        If True, prints status message. Default False.
    """
    attempt = {
        "modelName": modelName,
        "modelParams": modelParams,
        "mape": round(mape, 4) if mape is not None else None,
        "comments":comments
    }

    modelDict[segmentName]["modelsTried"].append(attempt)

    # Auto-update best model if this is the best so far
    current_best = modelDict[segmentName]["bestModel"]
    is_new_best = False

    if mape is not None:
        if current_best is None or mape < current_best["mape"]:
            modelDict[segmentName]["bestModel"] = {
                "modelName": modelName,
                "modelParams": modelParams,
                "mape": round(mape, 4),
            }
            is_new_best = True

    # Only print if verbose
    if verbose:
        if mape is not None:
            if is_new_best:
                print(f"✅ [{segmentName}] New best: {modelName}{modelParams} | MAPE: {mape:.4f}%")
            else:
                print(f" [{segmentName}] Added: {modelName}{modelParams} | MAPE: {mape:.4f}%")
        else:
            print(f"⚠  [{segmentName}] Added: {modelName}{modelParams} | MAPE: None | {modelName}")

    return is_new_best
```

getModelHistory()

In [112…
```python
def getModelHistory(modelDict, segmentName):
    """
    Get all model attempts for a segment as a DataFrame.
    """
    import pandas as pd
    attempts = modelDict[segmentName]["modelsTried"]
    if not attempts:
        print(f"No models tried yet for {segmentName}")
        return None
    return pd.DataFrame(attempts)
```

getAllBestModels()

In [112…
```python
def getAllBestModels(modelDict):
    """
    Get best models for all segments as a DataFrame.
    """
    import pandas as pd
    results = []
    for name, config in modelDict.items():
        best = config["bestModel"]
        results.append({
            "Segment": name,
            "BestModel": best["modelName"] if best else None,
            "Params": best["modelParams"] if best else None,
            "MAPE": best["mape"] if best else None,
            "Comments": best["comments"] if best else None,
        })
    return pd.DataFrame(results)
```

getAttemptsSummaryTable()

Returns a pandas datafame containing data of all experiments

In [112…
```python
def getAttemptsSummaryTable(modelDict):
    """
    Get summary of all attempts across all segments.
    """
    import pandas as pd
    rows = []
    for segmentName, config in modelDict.items():
        for attempt in config["modelsTried"]:
            rows.append({
                "Segment": segmentName,
                "Model": attempt["modelName"],
                "Params": attempt["modelParams"],
                "MAPE": attempt["mape"],
                "Comments": attempt["comments"],
            })
    return pd.DataFrame(rows)

summary = getAttemptsSummaryTable(modelDict)
summary
```

Out[112…

printModelDictSummary()

Purpose: Displays a summary of all data segments and validates data integrity.

What it does:

- Prints a markdown-formatted table showing all segments
- Displays row counts for each segment
- Shows model and MAPE values (if trained)
- Calculates total rows across all segments
- Compares against original dataframe to ensure no data loss

Output Includes:

- Summary table of all models
- Total row count
- Validation check against `df_with_extra_columns`

- Match/mismatch indicator

Why it's useful:

- Ensures data segmentation is complete
- Verifies no rows were lost or duplicated
- Provides quick overview of data distribution across segments

In [112…

```
def printModelDictSummary():
    """
    Prints summary of modelDict with new structure:
    - data, modelsTried, bestModel
    """
    totalRows = 0

    # Header
    print("## Model Dictionary Summary\n")
    print("| Model Name | Data Rows | # Attempts | Best Model | Best Params | Best MAPE |")
    print("|------------|-----------|------------|------------|-------------|-----------|")

    # Data rows
    for modelName, modelInfo in modelDict.items():
        rowCount = len(modelInfo['data']) if modelInfo['data'] is not None else 0
        totalRows += rowCount

        numAttempts = len(modelInfo['modelsTried'])

        if modelInfo['bestModel'] is not None:
            bestModelName = modelInfo['bestModel']['modelName']
            bestParams = modelInfo['bestModel']['modelParams']
            bestMape = f"{modelInfo['bestModel']['mape']:.4f}%" if modelInfo['bestModel']['mape'] is not None e
        else:
            bestModelName = 'None'
            bestParams = '-'
            bestMape = '-'

        print(f"| {modelName} | {rowCount} | {numAttempts} | {bestModelName} | {bestParams} | {bestMape} |")

    # Total row
    totalAttempts = sum(len(m['modelsTried']) for m in modelDict.values())
    print(f"| **TOTAL** | **{totalRows}** | **{totalAttempts}** | - | - | - |")

    # Validation
    originalRows = len(df_with_extra_columns)

    print("\n## Validation\n")
    print("| Metric | Count |")
    print("|--------|-------|")
    print(f"| Total rows across models | {totalRows} |")
    print(f"| Original dataframe rows | {originalRows} |")
    print(f"| Difference | {abs(totalRows - originalRows)} |")

    if totalRows == originalRows:
        print("\n✅ **MATCH:** Total rows equals original dataframe!")
    else:
        print(f"\n❌ **MISMATCH:** Difference of {abs(totalRows - originalRows)} rows")
```

`printModelDictSummaryDETAILED()`

Generates a beautiful markdown summary of results in the modelDict

In [112…

```
def printModelDictSummaryDETAILED(showAttempts=False):
    """
    Prints summary of modelDict.

    Parameters:
    -----------
    showAttempts : bool
        If True, shows all attempts for each model
    """
    totalRows = 0

    # Header
    print("**Model Dictionary Summary** \n")
    print("| Model Name | Data Rows | # Attempts | Best Model | Best Params | Best MAPE |")
    print("|------------|-----------|------------|------------|-------------|-----------|")

    # Data rows
    for modelName, modelInfo in modelDict.items():
```

```
        rowCount = len(modelInfo['data']) if modelInfo['data'] is not None else 0
        totalRows += rowCount

        numAttempts = len(modelInfo['modelsTried'])

        if modelInfo['bestModel'] is not None:
            bestModelName = modelInfo['bestModel']['modelName']
            bestParams = modelInfo['bestModel']['modelParams']
            bestMape = f"{modelInfo['bestModel']['mape']:.4f}%" if modelInfo['bestModel']['mape'] is not None e

        else:
            bestModelName = 'None'
            bestParams = '-'
            bestMape = '-'


        print(f"| {modelName} | {rowCount} | {numAttempts} | {bestModelName} | {bestParams} | {bestMape}")

        # Show all attempts if requested
        if showAttempts and numAttempts > 0:
            print(f"|   ↳ Attempts: | | | | | |")
            for j, attempt in enumerate(modelInfo['modelsTried'], 1):
                mape_str = f"{100*attempt['mape']:.1f}%" if attempt['mape'] is not None else 'Error'
                comment = attempt['comments'][:30] + '...' if len(attempt['comments']) > 30 else attempt['comme
                print(f"|   {j}. {attempt['modelName']}{attempt['modelParams']} | | | | {mape_str} | {comment}

    # Total row
    totalAttempts = sum(len(m['modelsTried']) for m in modelDict.values())
    print(f"| **TOTAL** | **{totalRows}** | **{totalAttempts}** | - | - | - |")

    # Validation
    originalRows = len(df_with_extra_columns)

    print("\n**Validation**\n")
    print("| Metric | Count |")
    print("|--------|-------|")
    print(f"| Total rows across models | {totalRows} |")
    print(f"| Original dataframe rows | {originalRows} |")
    print(f"| Difference | {abs(totalRows - originalRows)} |")

    if totalRows == originalRows:
        print("\n✅ **MATCH:** Total rows equals original dataframe!")
    else:
        print(f"\n❌ **MISMATCH:** Difference of {abs(totalRows - originalRows)} rows")

    # Best MAPE Summary
    print("\n**Best MAPE Ranking**\n")
    print("| Rank | Segment Name | Best MAPE |Source")
    print("|------|------------|-----------|------|")

    ranked = sorted(
        [(name, info['bestModel']['mape'], info['bestModel']['modelName'],info['bestModel']['modelParams'] ) fo
        key=lambda x: x[1]
    )

    for rank, (name, mape,modelName, modelParams) in enumerate(ranked, 1):
        print(f"| {rank} | {name} | {100*mape:.1f}% |{modelName}{modelParams}")

    if not ranked:
        print("| - | No models fitted yet | - |")

# Usage
# printModelDictSummary()              # Basic summary
# printModelDictSummary(showAttempts=True)  # With all attempts
```

3.0 Explore Data Characteristics

3.1 Univariate Analysis (Analysis of one attribute at a time.)

univariate_ordinal_analysis

Creating a helper function to plot multiple graphs in one figure for ordinal data

In [112…

```
import pandas as pd
import matplotlib.pyplot as plt

def univariate_ordinal_analysis(series, column_name, order=None, plotCount=None ):
    """
    Print summary statistics and plot distribution for an ordinal categorical Series.
```

```
    Parameters:
        series : pd.Series
            The ordinal categorical column.
        order : list (optional)
            Ordered list of categories (if not the Series' CategoricalDtype).
        plotCount:Pass none if both graphs wanted, 1 if only first one wanted
            in some cases, the second graph is not that useful
            Maybe in future if i find a better graph I will put it.
    """

    # Ensure categorical with order
    if not pd.api.types.is_categorical_dtype(series):
        series = series.astype("category")
    if order is not None:
        series = series.cat.set_categories(order, ordered=True)

    # Value counts in order
    counts = series.value_counts().reindex(series.cat.categories)
    perc = counts / counts.sum() * 100



    # Plot
    ax=[]
    if plotCount==1:
        fig, ax1 = plt.subplots(figsize=(10, 4))
        ax.append(ax1)
    else:
        fig, ax = plt.subplots(1, 2, figsize=(10, 4))


    # Bar plot (counts)
    bars = counts.plot(kind="bar", ax=ax[0], color="skyblue")
    ax[0].set_title(f"{column_name} Counts by Category")
    ax[0].set_ylabel("Count")

    # --- Annotate each bar with count and percentage ---
    total = sum(p.get_height() for p in ax[0].patches)
    for patch in ax[0].patches:
        height = patch.get_height()
        if height > 0:
            pct = height / total * 100
            ax[0].text(
                patch.get_x() + patch.get_width() / 2,  # center of bar
                height + (total * 0.005),               # position slightly above bar
                f"{int(height)}\n({pct:.1f}%)",         # count + % in next line
                ha='center', va='bottom', fontsize=8, color='black'
            )
    if(len(ax)>1): #draw only if plotCount is not 1
        # Cumulative percentage
        perc.cumsum().plot(marker="o", ax=ax[1], color="red")
        ax[1].set_title("Cumulative % by Category")
        ax[1].set_ylabel("Cumulative %")
        ax[1].set_ylim(0, 100)
        ax[1].grid(True, linestyle="--", alpha=0.5)

    plt.tight_layout()
    plt.show()
        # Print summary
    print(f"\n Summary Statistics for {column_name}")
    print("="*50)
    print(f"Mode   : {series.mode().iloc[0]}")
    print(f"Median : {series.sort_values().iloc[len(series)//2]}")
    # print("\nValue Counts (%):")
    # df_analysis=pd.DataFrame({"Count": counts, "Percent": perc.round(2)})
    # print(df_analysis)
    # df_analysis.sort_values(by="Percent", ascending=False, inplace=True)
    # print("Breakup of grades-"+", ".join([f"{idx} ({row['Percent']}%)" for idx, row in df_analysis.iterrows()
```

page_title

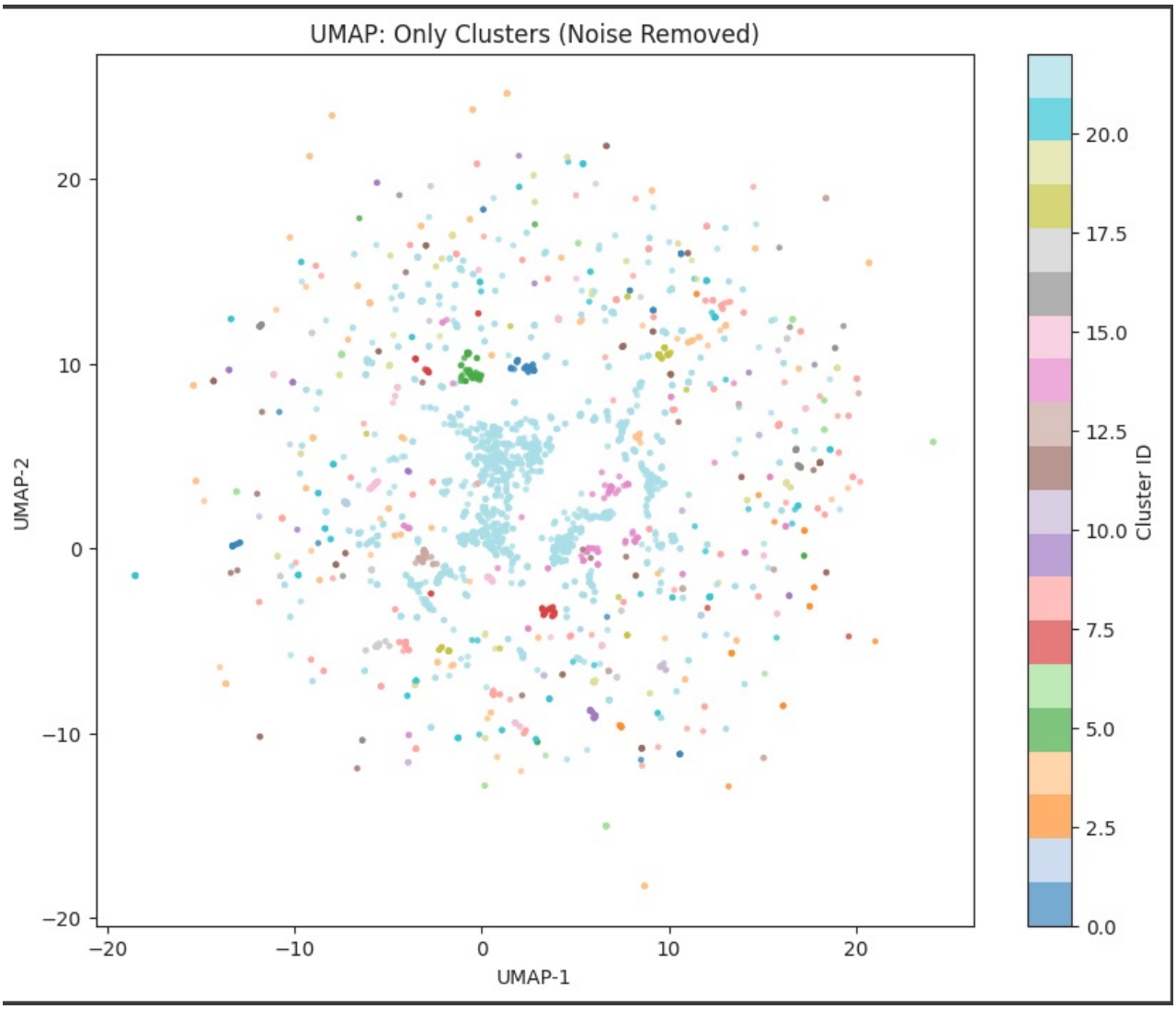
In [112…
```
df_analysis = df.groupby("page_title").language.count()
print(f"We have {len(df_analysis)} unique pages across {len(df)} rows, as one page can be in different language
```

```
We have 48874 unique pages across 144411 rows, as one page can be in different languages
```

We have 48874 unique pages across 144411 rows, as one page can be in different languages Page names are not only in English alphabet. For ex, Russian pages are in their alphabet.

	.wiki
0	!vote_en.wikipedia.org_desktop_all-agents
1	"Awaken,_My_Love!"_en.wikipedia.org_desktop_all-agents
2	"European_Society_for_Clinical_Investigation"_en.wikipedia.org_desktop_all-ager
3	"Weird_Al"_Yankovic_en.wikipedia.org_desktop_all-agents
4	100_metres_en.wikipedia.org_desktop_all-agents
5	10_Cloverfield_Lane_en.wikipedia.org_desktop_all-agents
6	10_Gigabit_Ethernet_en.wikipedia.org_desktop_all-agents
7	13_Hours:_The_Secret_Soldiers_of_Benghazi_en.wikipedia.org_desktop_all-agents
8	1551_en.wikipedia.org_desktop_all-agents
9	1896_Summer_Olympics_en.wikipedia.org_desktop_all-agents
0	1918_flu_pandemic_en.wikipedia.org_desktop_all-agents
1	1923_San_Pedro_Maritime_Strike_en.wikipedia.org_desktop_all-agents
2	1936_Summer_Olympics_en.wikipedia.org_desktop_all-agents
3	1976_Summer_Olympics_en.wikipedia.org_desktop_all-agents
4	1980_Summer_Olympics_en.wikipedia.org_desktop_all-agents
5	1984_Summer_Olympics_en.wikipedia.org_desktop_all-agents
6	1989_(Taylor_Swift_album)_en.wikipedia.org_desktop_all-agents
7	1999_(Prince_album)_en.wikipedia.org_desktop_all-agents

- The page name has useful info.
- We use a small embedding model to help understand the category. Maybe we can cluster pages.
- We want to identify some major clusters so that when user gives a page, we can find its category and use that sarimax model (so basically looking for another dimension for segmenting data)
- This study was done separately and **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2** was used to embed (as the data contains multiple languages)
- After that we ran HDBScan on it to auto identify clusters.
- Following clusters were identified

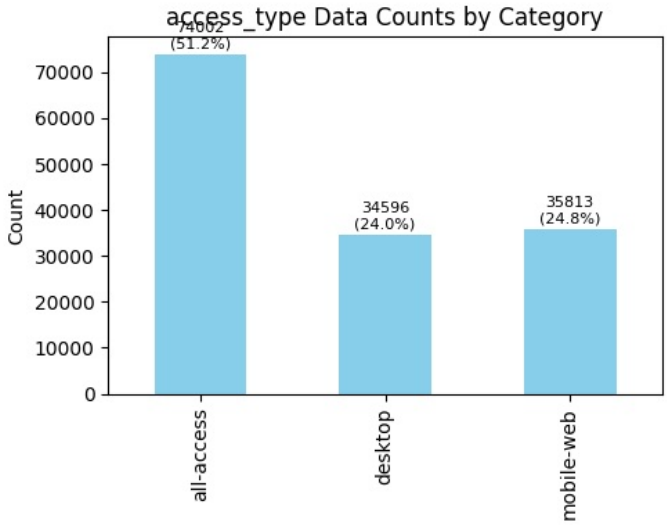


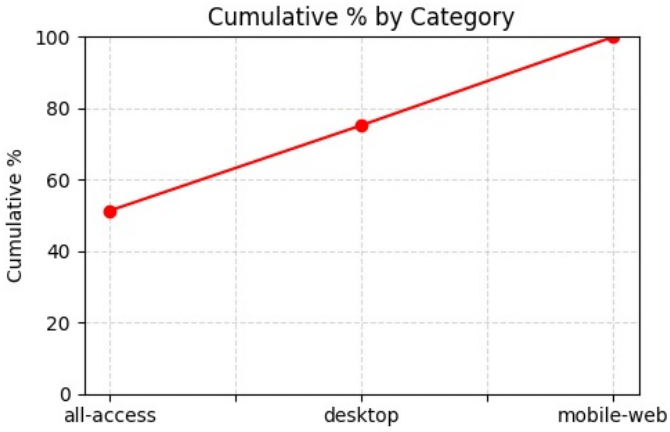
- Not incorporated that study in this submission due to time constraints.
- Next step is to use a LLM to identify titles for these clusters by passing it some of the pages in these clusters and asking it to give a nice representative cluster name..
- This can be done in future.

- We have 48874 unique pages across 144411 rows, as one page can be in different languages

access_type

In [113…
```
col = 'access_type'

univariate_ordinal_analysis(df[col], f"{col} Data")
```



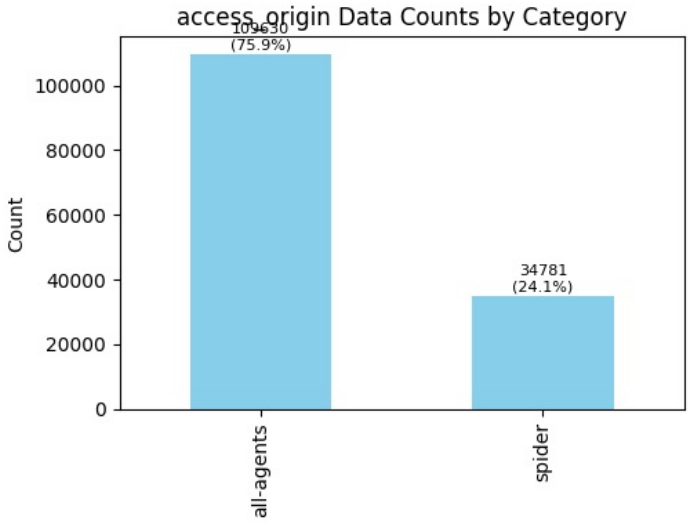


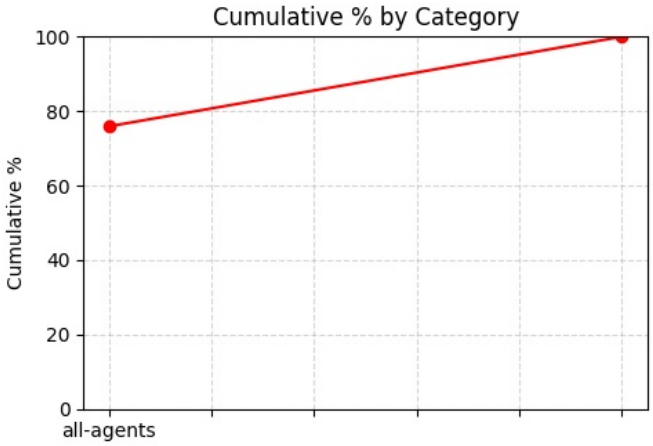

```
 Summary Statistics for access_type Data
==================================================
Mode   : all-access
Median : all-access
```

- We have three access_types - all-access(51%), desktop(24%) and mobile(25%)

access_origin

In [113…
```
col = 'access_origin'

univariate_ordinal_analysis(df[col], f"{col} Data")
```



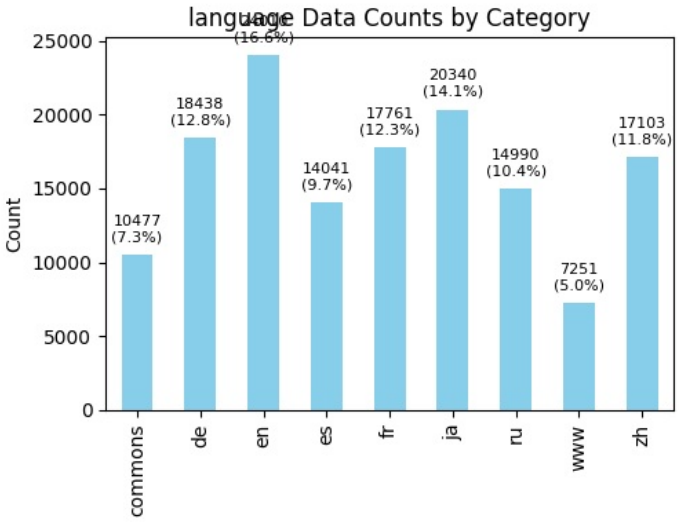


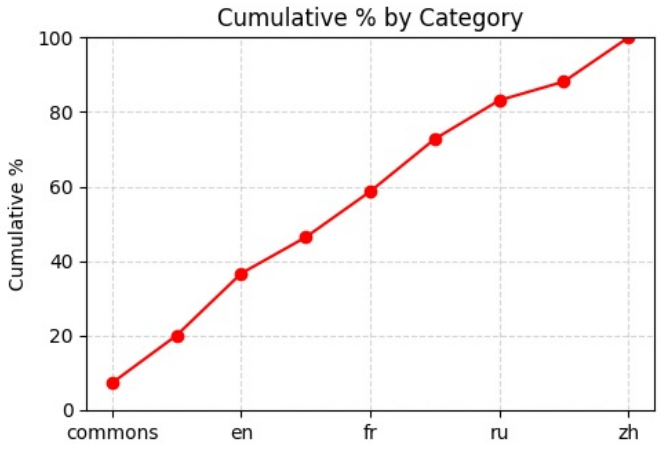

```
 Summary Statistics for access_origin Data
==================================================
Mode   : all-agents
Median : all-agents
```

- In terms of origin of request, 24% originate as spider queries, while rest are all-agents

language

In [113…
```
col = 'language'

univariate_ordinal_analysis(df[col], f"{col} Data")
```






```
 Summary Statistics for language Data
==================================================
Mode   : en
Median : fr
```

In [113…
```
df.groupby(col).size()
```

Out[113…
```
language
commons    10477
de         18438
en         24010
es         14041
fr         17761
ja         20340
ru         14990
www         7251
zh         17103
dtype: int64
```

- Here is the count of pages wrt language. See explanation of commons further in the note.
- commons 10477
- de 18438
- en 24010
- es 14041
- fr 17761
- ja 20340
- ru 14990
- www 7251
- zh 17103

On Wikipedia (and Wikimedia projects in general), **“commons” refers to *Wikimedia Commons***.

What is Wikimedia Commons?

Wikimedia Commons is a **central media repository** for all Wikimedia projects. It stores **images, audio, video, and other media files** that are **free to use**.

Website: commons.wikimedia.org

What do those labels mean?

When you see file links like:

- **en** → English Wikipedia
- **fr** → French Wikipedia
- **de** → German Wikipedia
- **commons** → Wikimedia Commons

they indicate **where the file is hosted**.

Why is “commons” special?

Files on **Wikimedia Commons**:

- Can be used on **all language Wikipedias** (en, fr, de, etc.)
- Don't need to be uploaded separately for each language
- Must have **free licenses** (e.g., CC BY, CC BY-SA, public domain)

Files uploaded to **en/fr/de Wikipedia**:

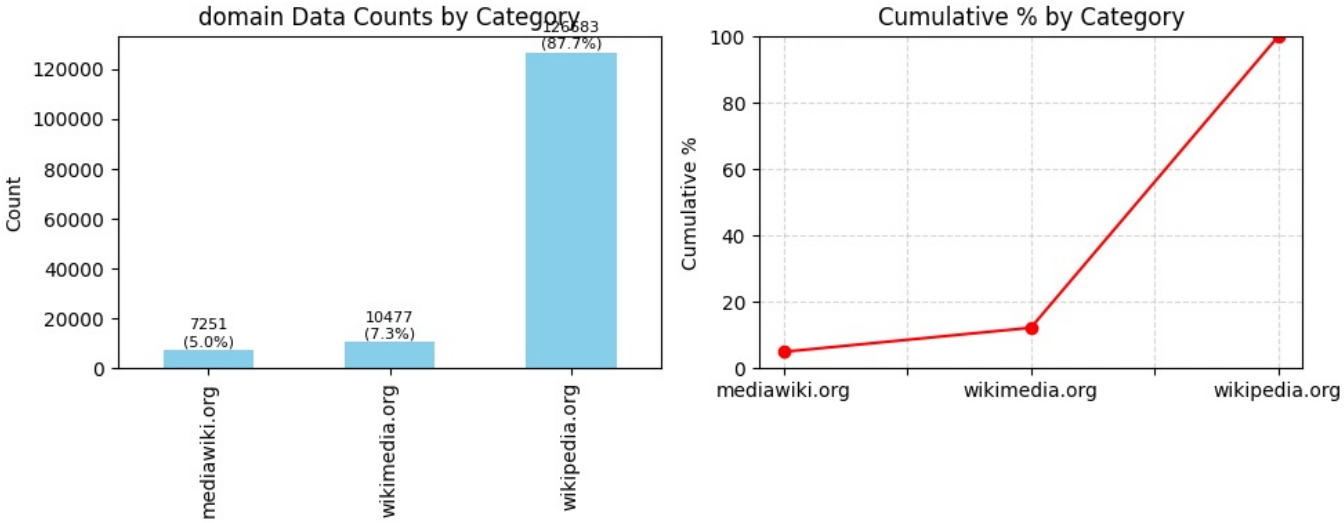
- Are usually **local uploads**
- Often used when licensing doesn't allow global reuse (e.g., fair use)
- Can only be used on that specific Wikipedia

Summary

- **commons = shared by everyone**
- **en/fr/de = local to that language**

domain

In [113…
```
col = 'domain'

univariate_ordinal_analysis(df[col], f"{col} Data")
```




```
 Summary Statistics for domain Data
==================================================
Mode   : wikipedia.org
Median : wikipedia.org
```

- We have pages hosted across three domains - wikipedia.org(88%), wikimedia.org (7%) and mediawiki.org(5%)

What is wikipedia, wikimedia and mediawiki

These three are related, but they are **not the same thing**. Think of them as **content → organization → software**.

---

1 Wikipedia

What it is: A free online encyclopedia

- Website where people read and write articles (e.g., *Photosynthesis*, *India*, *Albert Einstein*).
- Content is written and edited by volunteers around the world.
- Anyone can read it; most pages can be edited by anyone.
- Articles aim to be **neutral**, **verifiable**, and **well-sourced**.

Example:

- You search "Electrolysis" → you are using **Wikipedia**

Website: wikipedia.org

---

2 Wikimedia

What it is: □ A non-profit organization

- Full name: **Wikimedia Foundation**
- Owns and operates Wikipedia **and many other projects**.
- Provides servers, legal support, and funding.
- Does **not** write articles itself.

Projects run by Wikimedia:

- **Wikipedia** – encyclopedia
- **Wikimedia Commons** – images, videos, audio
- **Wiktionary** – dictionary
- **Wikibooks** – textbooks
- **Wikinews** – news
- **Wikidata** – structured data

Example:

- The reason Wikipedia is free and has no ads → **Wikimedia Foundation**

Website: wikimediafoundation.org

3□ MediaWiki

What it is: Software

- The **wiki software** that runs Wikipedia.
- Written mainly in PHP.
- Open-source (anyone can download and use it).
- Handles:
  - Editing pages
  - Version history
  - User accounts
  - Links between pages

Example:

- Wikipedia runs on **MediaWiki**
- You can install MediaWiki and create your **own wiki**

For this case study perspective, files that are mentioned under mediawiki look like this

From the screenshot, we can infer this

- They are wiki system pages stored in the MediaWiki database.
- **Understanding MediaWiki: namespace**
  - It stores site-wide system messages and resources, such as:
    - CSS `MediaWiki:Filepage.css` which Controls styling of file pages
    - JavaScript `MediaWiki:Gadget-HotCat.js` Used for: UI behavior, Upload dialogs, Tooltips, Gadgets (optional

user tools)
    - Text / i18n `MediaWiki:UploadDialogtext` Used for: Messages,Translations, Notices. These are stored as wiki pages in the database
-

Website: mediawiki.org

Simple analogy

Think of a **school**:

Real World	Wiki World
Textbook	**Wikipedia**
School management	**Wikimedia Foundation**
Blackboard & classroom system	**MediaWiki**

- **Case study-centric definition (based on what the URL/page name represents)**

wikipedia.org

Textual encyclopedic content

- Articles written for reading
- Mostly text + references (images are embedded but not hosted here)
- Examples:
    - `Photosynthesis`
    - `World_War_II`
    - `Electrolysis`

Rule of thumb:

> If it's **wikipedia**, it's **article text and structured prose**

---

wikimedia.org / commons.wikimedia.org

Media assets (files) + media description pages

- Images, diagrams, maps
- Audio files
- Video files
- File description & category pages

Examples

	Page
35	Accueil_commons.wikimedia.org_all-access_spider
36	Atlas_of_Asia_commons.wikimedia.org_all-access_spider
37	Atlas_of_Europe_commons.wikimedia.org_all-access_spider
38	Atlas_of_World_War_II_commons.wikimedia.org_all-access_spider
39	Atlas_of_colonialism_commons.wikimedia.org_all-access_spider
40	Atlas_of_the_United_Kingdom_commons.wikimedia.org_all-access_spider
41	Atlas_of_the_United_States_commons.wikimedia.org_all-access_spider
42	Bikini_commons.wikimedia.org_all-access_spider
43	Campaign:OFBA2016_commons.wikimedia.org_all-access_spider
44	Catalogue_of_Wilhelm_von_Gloeden's_pictures_commons.wikimedia.org_all-acces

These are:

- Image collections
- Media catalogues
- Visual resources

Rule of thumb:

> If it's **wikimedia / commons**, it's **media files or pages describing media**

---

mediawiki.org

Software, UI, and site-behavior resources

- JavaScript (gadgets, UI logic)
- CSS (styling)
- System messages
- API & extension documentation

Examples

MediaWiki:Filepage.css_commons.wikimedia.org_all-access_spider
MediaWiki:Gadget-HotCat.js_commons.wikimedia.org_all-access_spider
MediaWiki:ImageAnnotatorConfig.js_commons.wikimedia.org_all-access_spider
MediaWiki:Tooltips.js_commons.wikimedia.org_all-access_spider
MediaWiki:UIElements.js_commons.wikimedia.org_all-access_spider
MediaWiki:Uploadnologintext_commons.wikimedia.org_all-access_spider
Special:WhatLinksHere/File:MediaWiki_logo_without_tagline.png_commons.wikimedia
Special:WhatLinksHere/File:Mediawiki.png_commons.wikimedia.org_all-access_spide
MediaWiki:LAPI.js_commons.wikimedia.org_all-access_spider
MediaWiki:Sitenotice-translation_commons.wikimedia.org_all-access_spider
MediaWiki:TextCleaner.js_commons.wikimedia.org_all-access_spider
MediaWiki:Uploadnologintext/ja_commons.wikimedia.org_all-access_spider

These are:

- Front-end behavior
- Interface logic
- Configuration artifacts

Rule of thumb:

> If it's **mediawiki**, it's **code, configuration, or software documentation**

3.2 Bivariate Analysis

Helper function to plot all categories

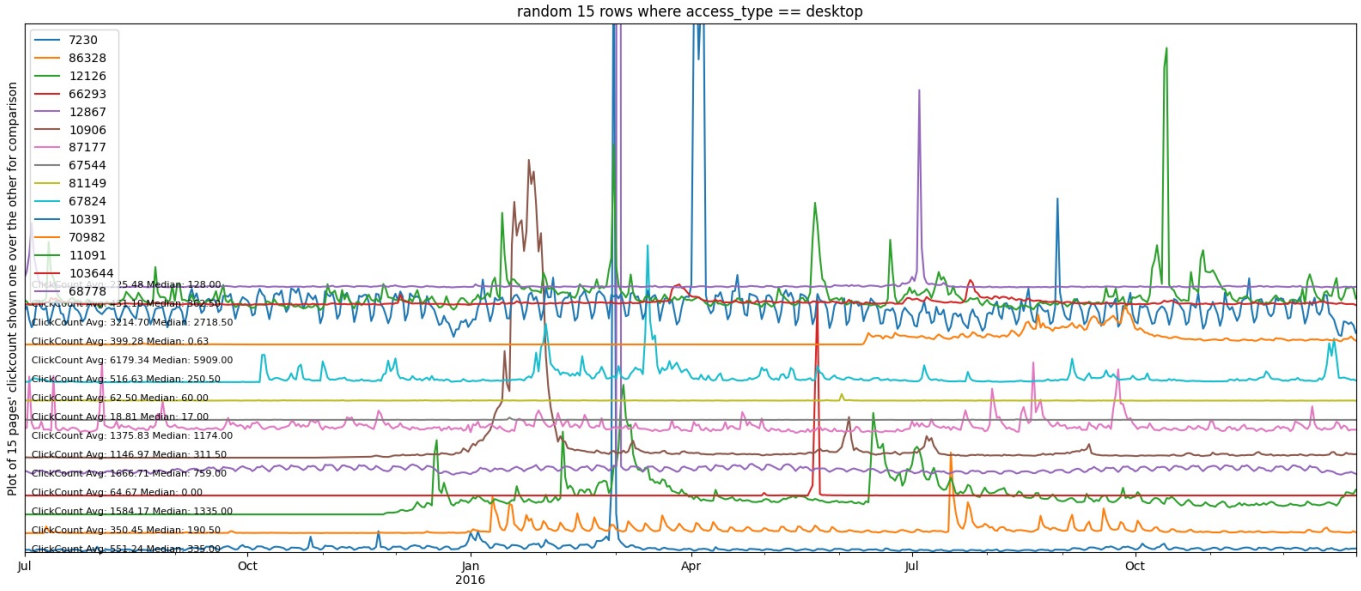
In [113…
```
#Quickly plot sample data based on colName
def plotSamplesFrom(colName, sampleSize=15, campaign=False, **kwargs):
    # Iterate over all unique values in colName
    for val in df[colName].unique():
        # Select sampleSize random rows based on colName and each unique value
        rows, title = search_data(sampleSize, colName1=colName, val1=val)
        plot_data_staggered(rows, title, campaign=campaign, **kwargs)
```

ClickCount and access_type

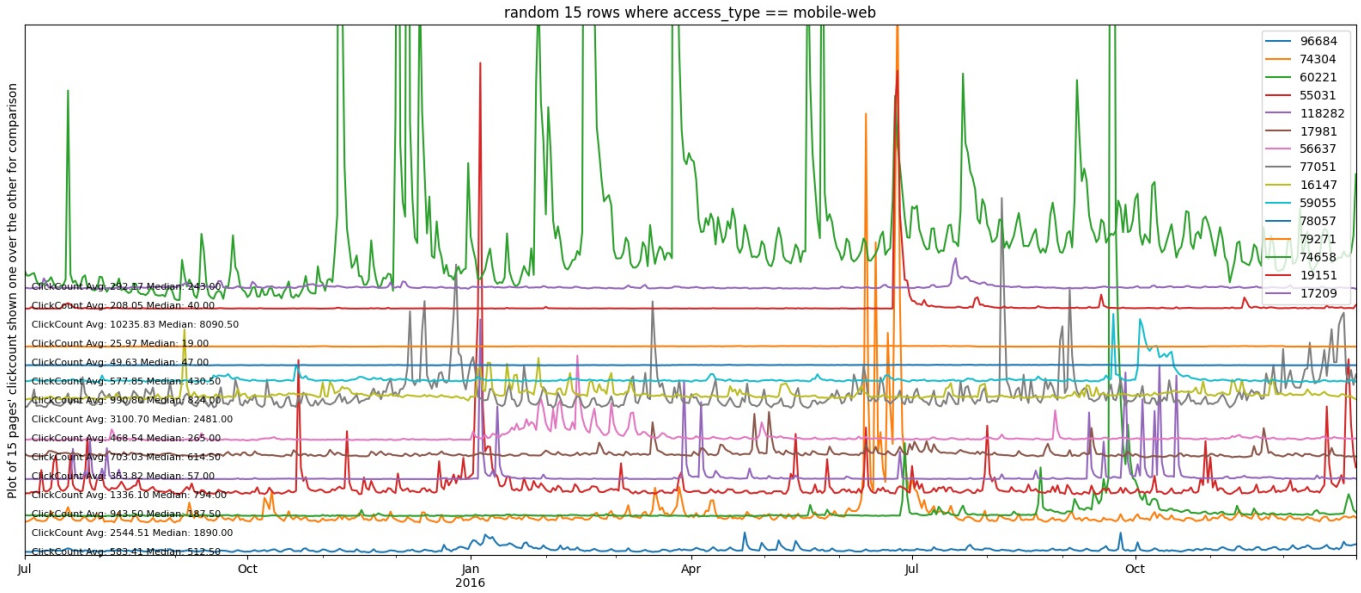
In [113…
```
plotSamplesFrom('access_type', campaign=False)
```

```
search_data():Searched random 15 rows where access_type == all-access
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```

search_data():Searched random 15 rows where access_type == desktop
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



search_data():Searched random 15 rows where access_type == mobile-web
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



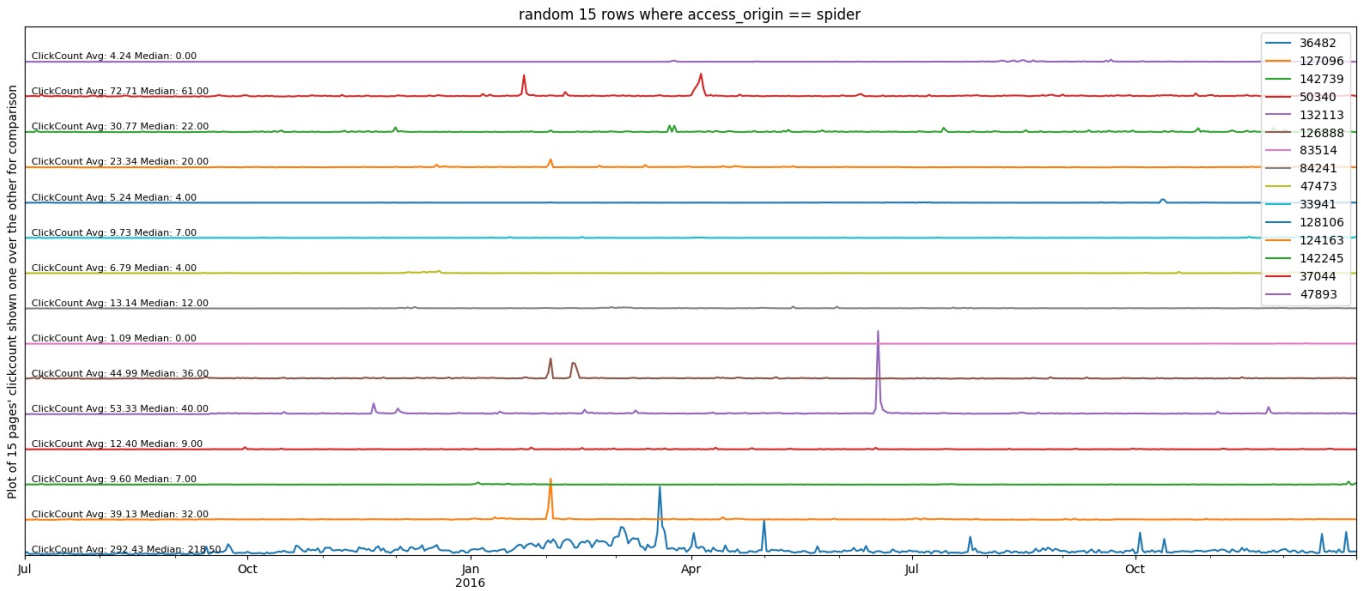
- Data across pages is NOT dependent on access_type as we can see that for same access type, randomly selected rows dont have similar pattern

ClickCount and access_origin

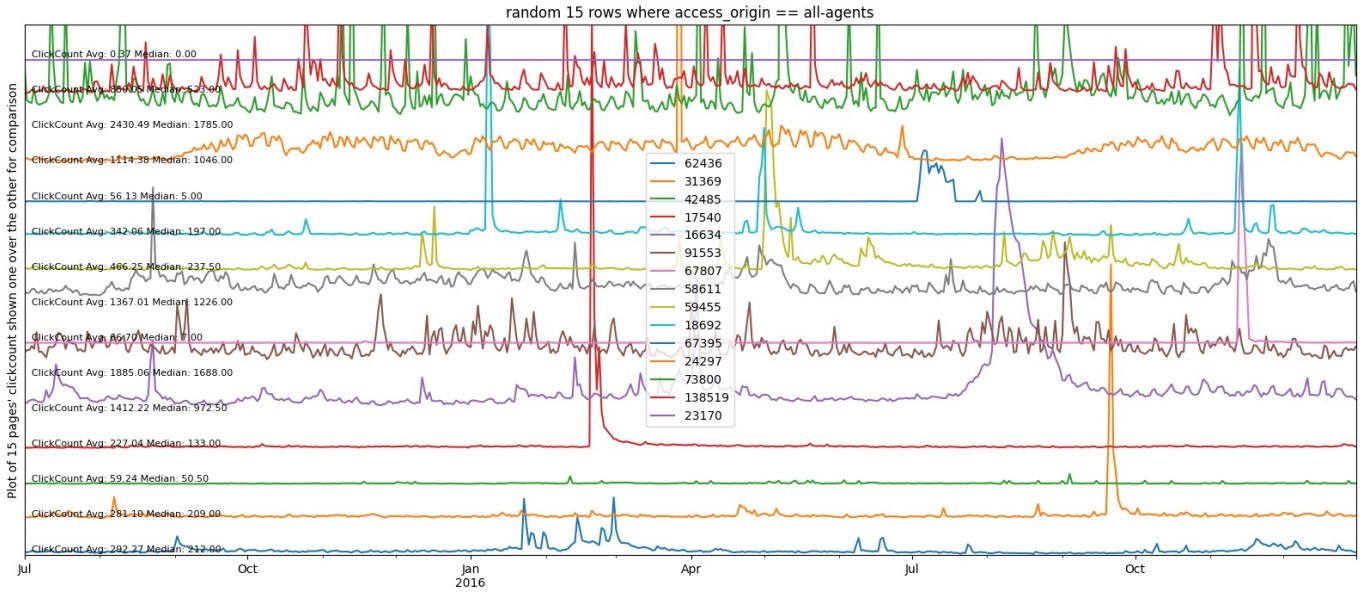
In [113…

```
plotSamplesFrom('access_origin',ylim=30000)
```

```
search_data():Searched random 15 rows where access_origin == spider
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```




```
search_data():Searched random 15 rows where access_origin == all-agents
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```

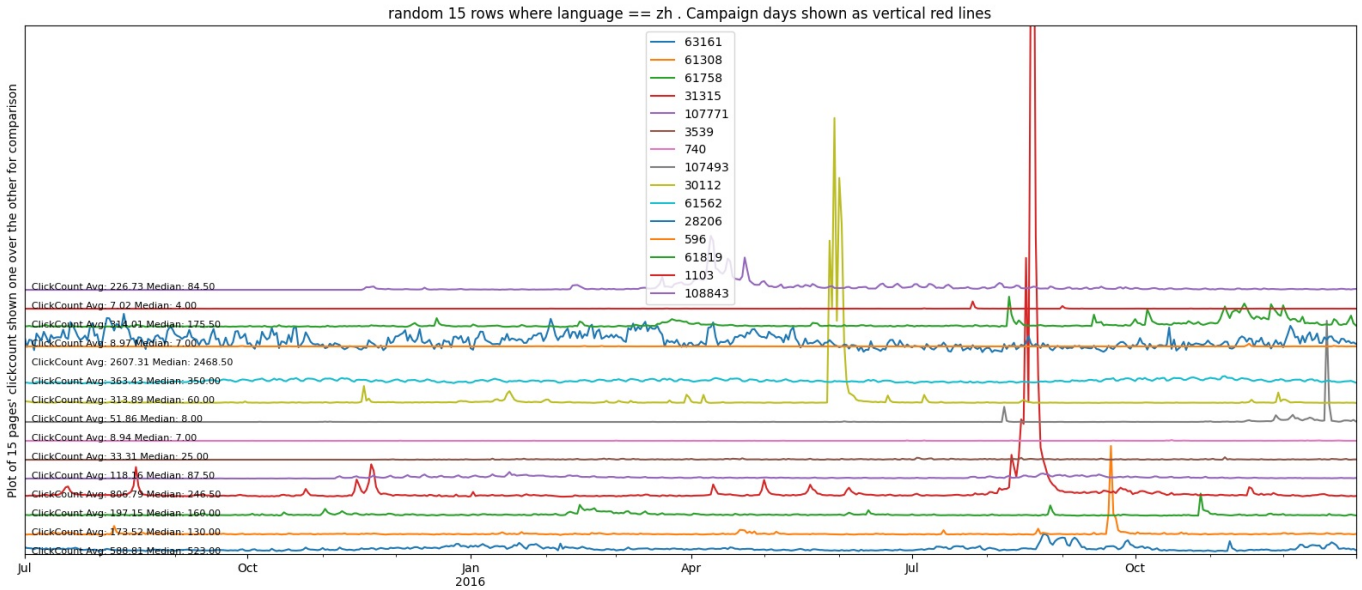


- Click Count across pages **IS** dependent on access_origin as we can clearly see a difference.
  - In spider, the graph is having almost no seasonality or variation except the occasional spikes
  - In all_agents, data is much more varying across time

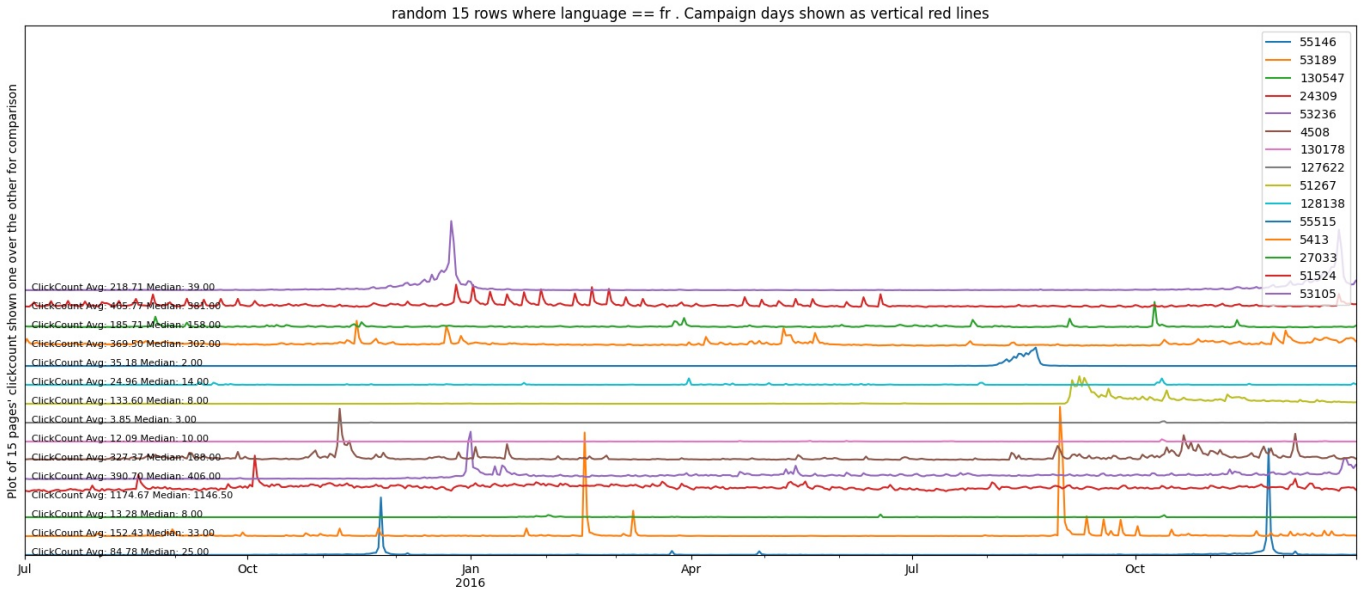
ClickCount and language

In [113…
```
plotSamplesFrom('language', campaign=True)
```

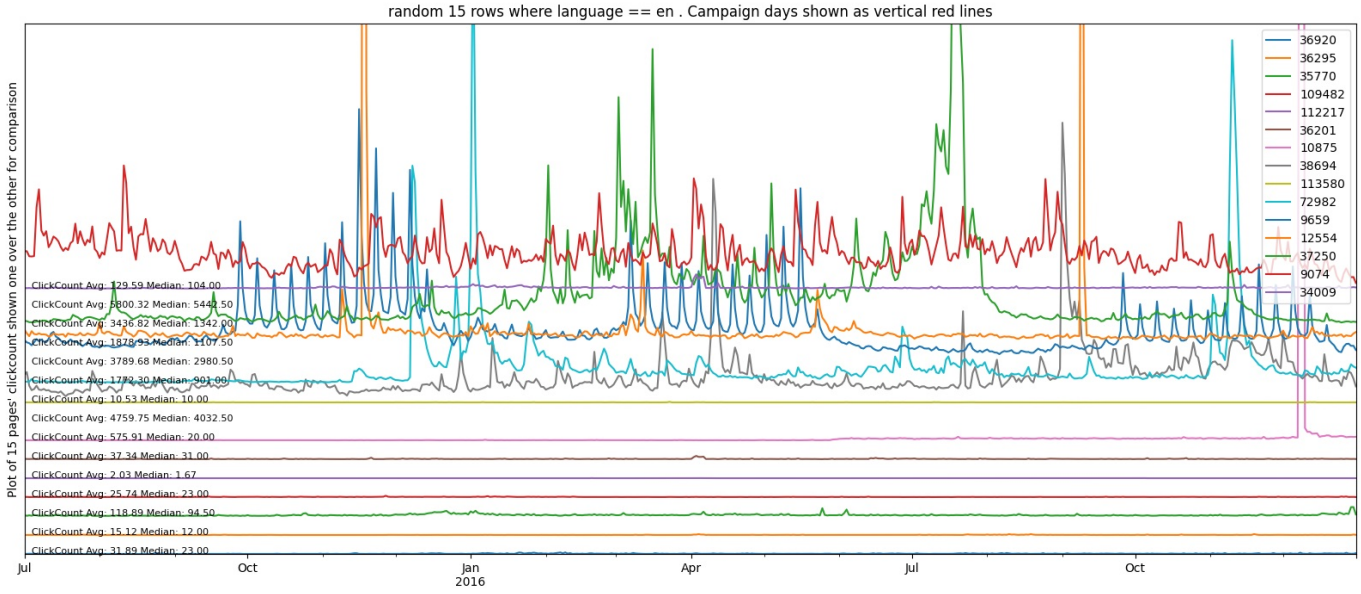
```
search_data():Searched random 15 rows where language == zh
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```



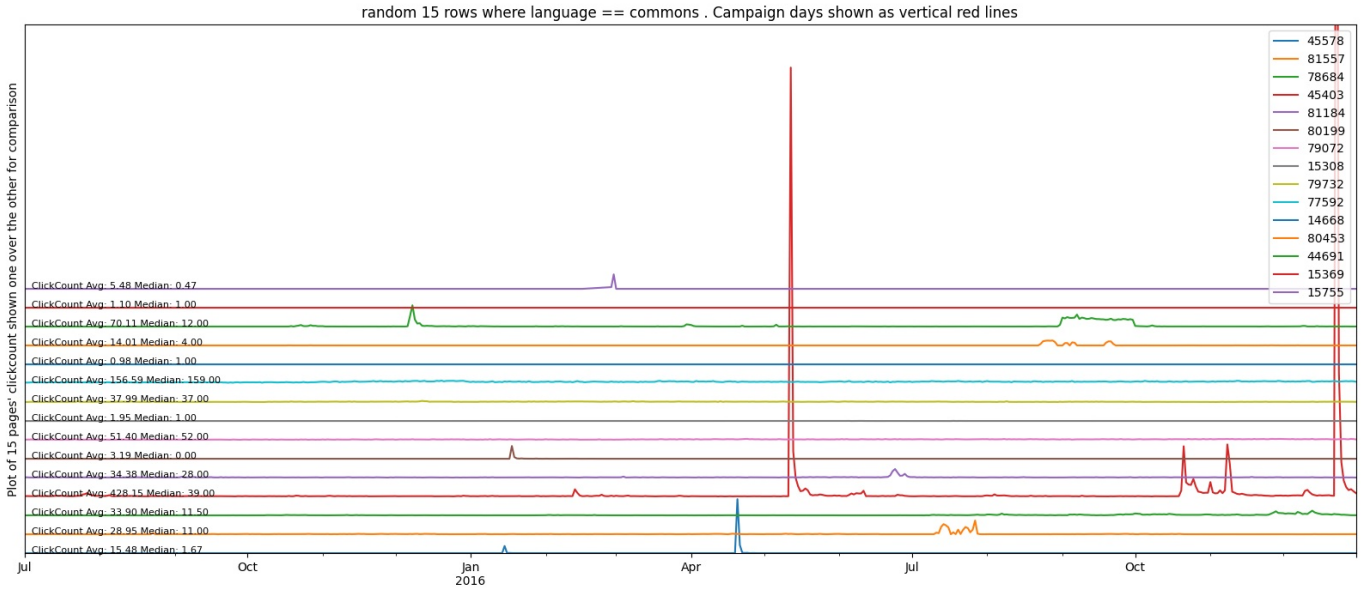

```
search_data():Searched random 15 rows where language == fr
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```



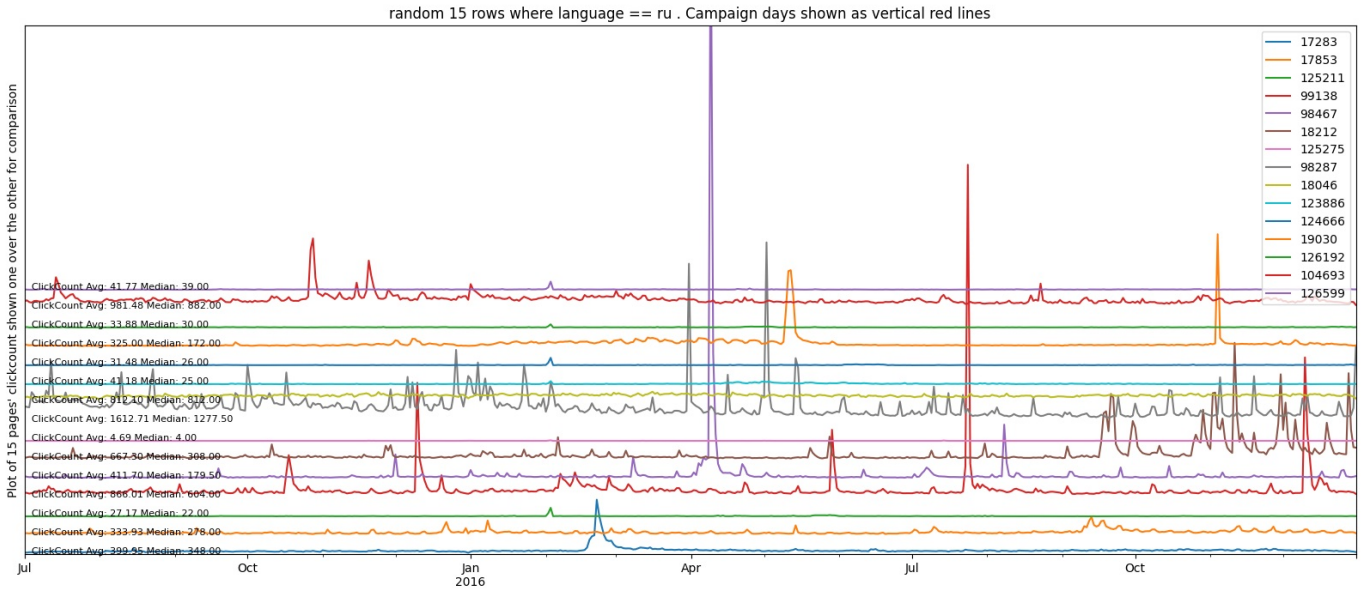
search_data():Searched random 15 rows where language == en
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



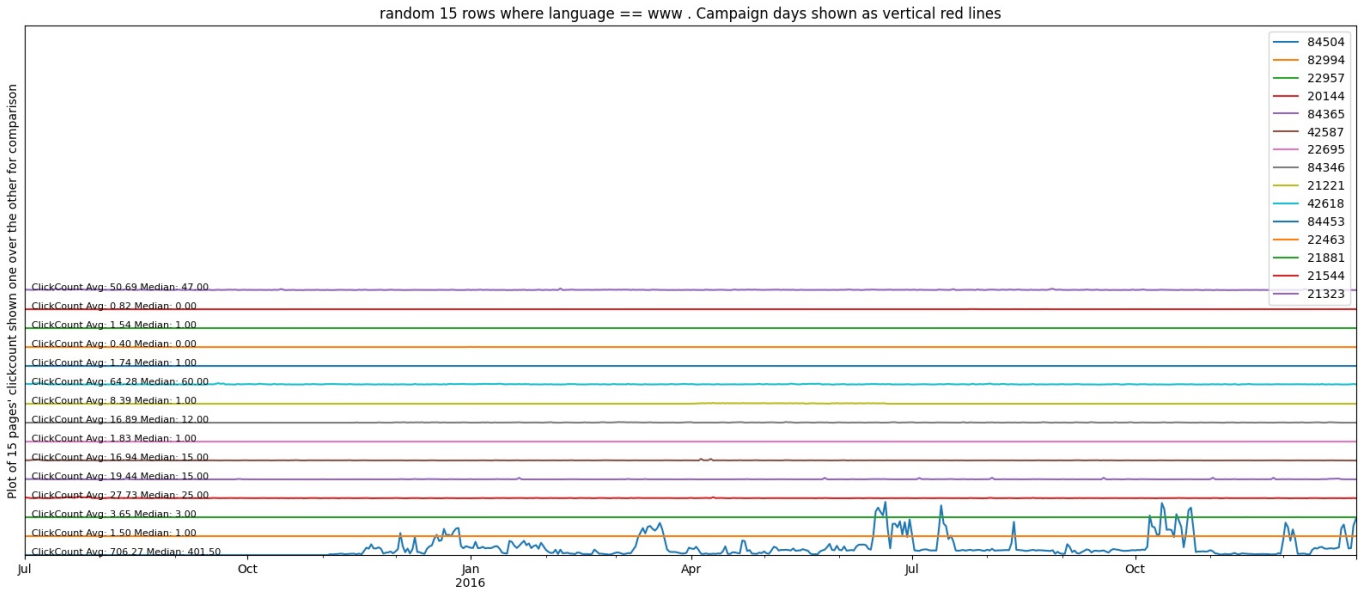
search_data():Searched random 15 rows where language == commons
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



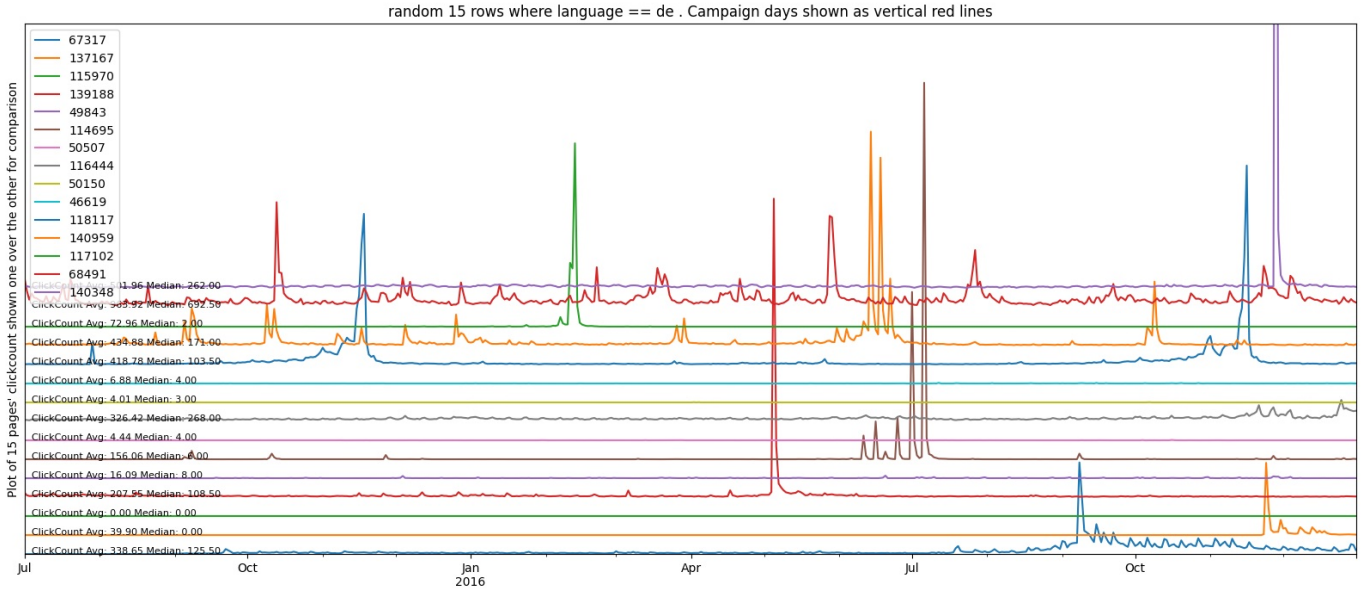
search_data():Searched random 15 rows where language == ru
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



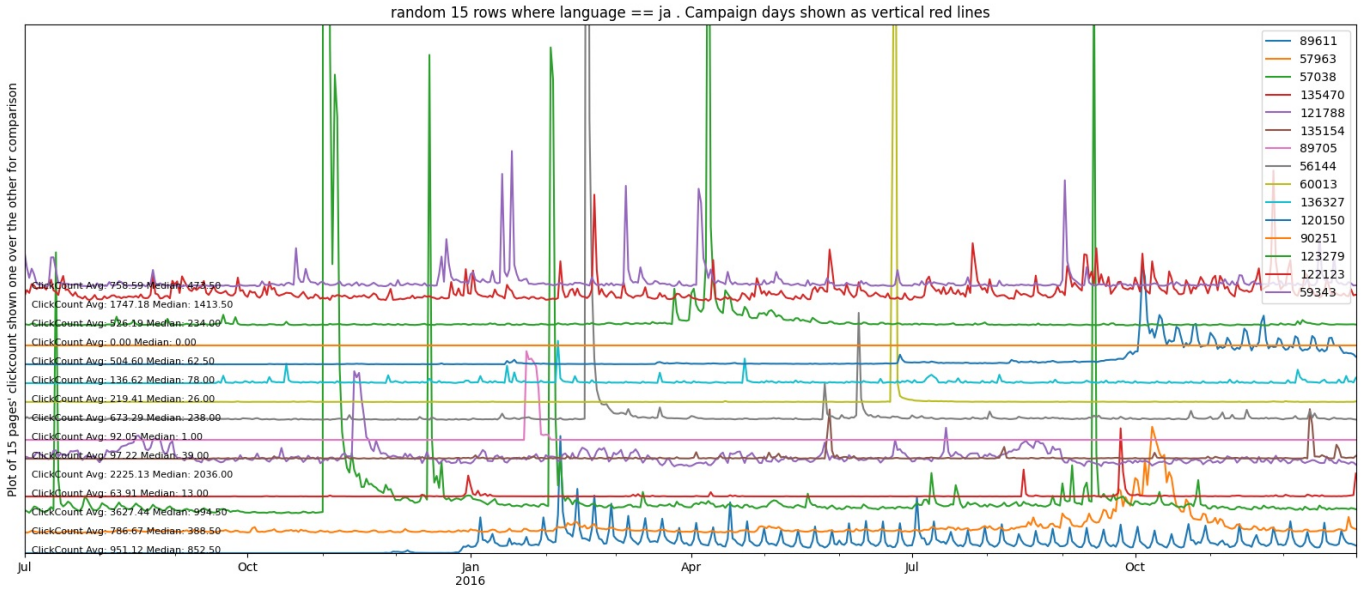
search_data():Searched random 15 rows where language == www
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



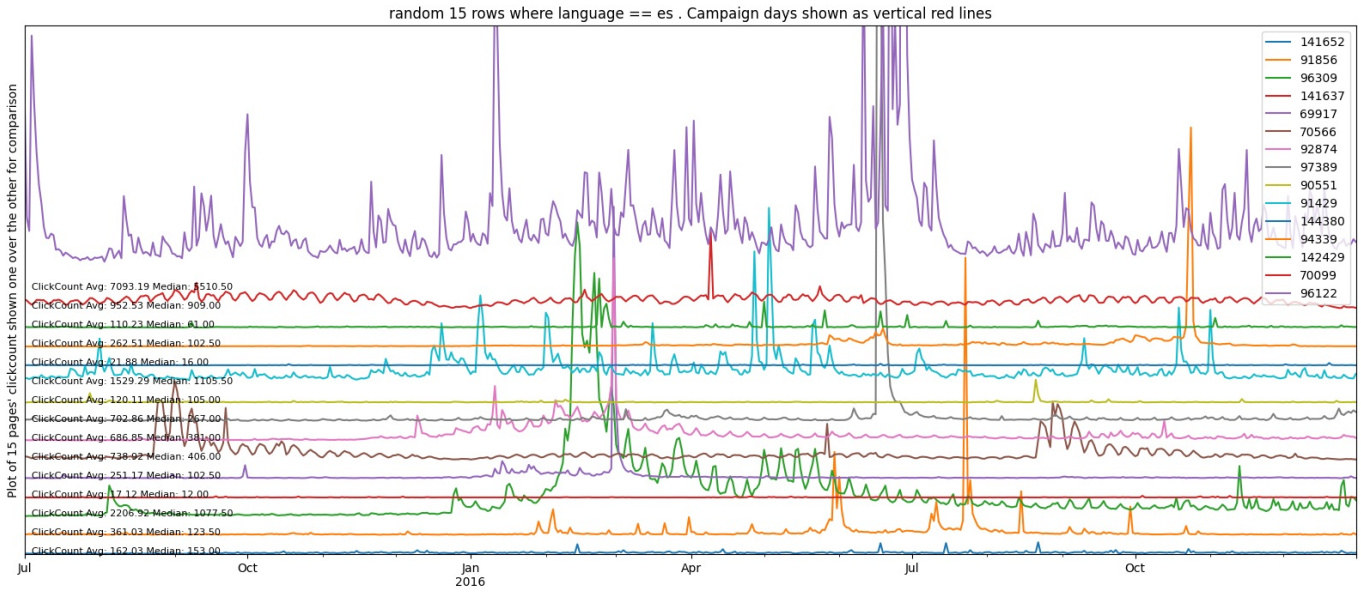
search_data():Searched random 15 rows where language == de
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows



search_data():Searched random 15 rows where language == ja
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows




```
search_data():Searched random 15 rows where language == es
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```



- Visual inspection reveals that
  - EN pages are quite different compared to other languages.
  - DE pages are having much less spikes
  - www has almost NO spikes !
  - commons is also flattish
  - FR is flattish
  - Overall feel is that we may need to build different models for different languages.
  - To confirm this feel, we can use some statistical tests. We do this next to establish one thing- does language be used for segmentation

Do EN pages cluster differently from FR pages in level/variance/seasonality/autocorrelation (things SARIMAX cares about)?

The feature + label-permutation test is aimed at exactly that.

- Treat each page time series as an intact block (we never shuffle days within a page).
- Compute a set of time-series features per page (level, variability, tail behavior, autocorrelation, weekly seasonality, etc.).
- Do a permutation test that shuffles the language labels across pages and checks whether EN vs FR differ in those features.
- This directly tests: “Is language a meaningful segmentation variable?” (without requiring the unrealistic claim that all EN pages have the same distribution).

In [113…

```python
import re
import numpy as np
import pandas as pd

def get_date_columns(df):

    # Find all columns like "2015-07-01"
    date_cols = [c for c in df.columns if re.fullmatch(r"\d{4}-\d{2}-\d{2}", str(c))]

    #ensure they are in chronological order (important so lags like 7 days make sense).
    date_cols = sorted(date_cols)

    if not date_cols:
        raise ValueError("No date columns found. Are your date columns named like YYYY-MM-DD?")

    # converts those column-name strings into actual datetime64 values
    dates = pd.to_datetime(date_cols)
    return date_cols, dates

# Purpose: compute autocorrelation at a specific lag.
# Lag 1: correlation between today and yesterday
# Lag 7: correlation between today and same weekday last week (weekly pattern)
def autocorr(x, lag):
    x = np.asarray(x, float)
    if len(x) <= lag + 2:
        return np.nan
    #align the series with itself shifted by lag.
    x0 = x[:-lag]
    x1 = x[lag:]

    #Check if  data are constant (std=0).
    if np.std(x0) == 0 or np.std(x1) == 0:
        return np.nan
    # Pearson correlation between the aligned vectors.
    return float(np.corrcoef(x0, x1)[0, 1])

# Our dataframe is wide: one row = one page; many columns are dates (counts), plus some metadata columns like la
# The goal of this code block is to transform our wide table into a new table where:
    # each row = one page
    # each column = a summary feature of that page’s time series (mean, std, quantiles, autocorrelation, weekly

# Purpose: take one page’s time series (one row of date columns) and convert it into numeric features.
def page_features_from_row(counts, dates):
    """
    counts: array-like length T (daily counts for a page)
    dates: DatetimeIndex length T
    """

    # a) Create a time-indexed series and drop missing
    # pairs each count with its date
    s = pd.Series(counts, index=dates).dropna()
    if len(s) < 60:   # require enough history
        return None

    # stabilize count scale
        # log1p(z) = log(1+z)
        # helps because page hits can be heavy-tailed (big spikes) and variance increases with level
        # makes features like mean/std more comparable across pages
    x = np.log1p(s.to_numpy())

    # Compute “distribution” features
        # mean / std / quantiles of x
        # "p_zero": fraction of days with exactly zero hits (important for sparse pages)
    feats = {
        # marginal-ish distribution features
        "mean": float(np.mean(x)),
        "std":  float(np.std(x, ddof=1)),
        "q10":  float(np.quantile(x, 0.10)),
        "q50":  float(np.quantile(x, 0.50)),
        "q90":  float(np.quantile(x, 0.90)),
        "p_zero": float(np.mean(s.to_numpy() == 0)),
```

```
        # dynamics (time dependence)
        "acf1": autocorr(x, 1),
        "acf7": autocorr(x, 7),
    }

    # weekday seasonality strength
    #If different weekdays have very different average traffic,
    # dow_means varies a lot → larger numerator → bigger weekday_strength.

    # If weekdays don't matter, weekday means are similar → low numerator → small strength.
    ser = pd.Series(x, index=s.index)
    dow_means = ser.groupby(ser.index.dayofweek).mean()
    feats["weekday_strength"] = float(np.var(dow_means) / (np.var(ser) + 1e-12))

    return feats

# ---- build feature matrix from the wide df ----
date_cols, dates = get_date_columns(df)

langs = df["language"].to_numpy()
# extract just the date columns as a matrix:
Y = df[date_cols].to_numpy()  # shape (n_pages, T)

rows = []
lang_keep = []
for i in range(len(df)):
    feats = page_features_from_row(Y[i, :], dates)
    if feats is None:
        continue
    rows.append(feats)
    lang_keep.append(langs[i])

F = pd.DataFrame(rows).replace([np.inf, -np.inf], np.nan).dropna()
lang_keep = np.array(lang_keep)[F.index]  # align

# standardize features (so distance isn't dominated by one feature) for distance computations
# This makes each feature roughly mean 0 and std 1 across pages.
# Without this, a feature on a larger numeric scale (say mean) could dominate the energy distance,
#  drowning out seasonality/autocorrelation features.
X = F.to_numpy()
X = (X - X.mean(axis=0)) / (X.std(axis=0) + 1e-12)

print(F.head())
print("Kept pages:", len(F), "Languages:", pd.Series(lang_keep).value_counts().to_dict())
```

```
       mean       std       q10       q50       q90    p_zero      acf1  \
0  2.872338  0.613641  2.197225  2.833213  3.666093  0.000000  0.238057
1  3.019816  0.621835  2.397895  2.890372  3.830792  0.000000  0.320428
2  1.499150  0.646674  0.693147  1.609438  2.079442  0.038182  0.304911
3  2.677635  0.590726  2.079442  2.639057  3.367296  0.000000  0.152513
4  0.883309  1.108791  0.000000  0.000000  2.397895  0.541818  0.818911

       acf7  weekday_strength
0  0.238220          0.010182
1  0.148605          0.005000
2  0.116656          0.003280
3  0.085106          0.006536
4  0.753200          0.001832
Kept pages: 144191 Languages: {'en': 23980, 'ja': 20328, 'de': 18399, 'fr': 17745, 'zh': 17079, 'ru': 14974, 'es
': 14031, 'commons': 10464, 'www': 7191}
```

- energy_stat_fast computes the energy-distance separation between two language groups using a precomputed pairwise distance matrix and indicator vectors, enabling thousands of label permutations without recomputing distances or allocating enormous intermediate arrays.

Context: what statistic are we computing? For two groups of pages (e.g., EN vs FR), each page is represented by a feature vector (mean, std, acf1, …). The **energy distance** between the two groups is:

$$\mathcal{E}(A,B) = 2\,\mathbb{E}\|X-Y\| \;-\; \mathbb{E}\|X-X'\| \;-\; \mathbb{E}\|Y-Y'\|$$

- X, X' are two (independent) samples from group A
- Y, Y' are two samples from group B
- $\|\cdot\|$ is Euclidean distance in feature space

If A and B come from the same distribution (in feature space), this tends to be small; if they differ, it tends to be larger.

Why `energy_stat_fast` is needed A direct implementation often computes all pairwise distances between EN and FR by broadcasting, which can create a massive temporary array

Instead, we:

1. **Precompute one distance matrix** (D) of size (N\times N) for a subsample of pages, where:
   - (D_{ij} = |x_i - x_j|)
2. For each permutation, we only change group membership labels and recompute the statistic using **matrix-vector operations**—no expensive recomputation of distances, no 3D arrays.

That's exactly what `energy_stat_fast` does.

What `energy_stat_fast(D, mask_a)` computes

Inputs

- `D`: an (N\times N) matrix of pairwise distances between all sampled pages.
- `mask_a`: boolean vector length (N). `True` means "this page is in group A" (e.g., EN).

Inside the function:

```
g = mask_a.astype(np.float32)   # indicator vector for group A (1 if in A else 0)
gb = 1.0 - g                    # indicator vector for group B
n = g.sum()                     # size of A
m = N - n                       # size of B
```

Key trick: compute average distances via quadratic forms Using the indicator vectors, these quantities can be computed:

- **Average distance between groups** (A–B):

$$\bar{D}_{AB} = \frac{1}{nm} \sum_{i \in A} \sum_{j \in B} D_{ij}$$

Computed as:

```
d_ab = (g @ D @ gb) / (n * m)
```

- **Average distance within group A**:

$$\bar{D}_{AA} = \frac{1}{n^2} \sum_{i \in A} \sum_{j \in A} D_{ij}$$

Computed as:

```
d_aa = (g @ D @ g) / (n * n)
```

- **Average distance within group B**:

$$\bar{D}_{BB} = \frac{1}{m^2} \sum_{i \in B} \sum_{j \in B} D_{ij}$$

Computed as:

```
d_bb = (gb @ D @ gb) / (m * m)
```

Then the energy statistic is:

```
2*d_ab - d_aa - d_bb
```

Why this is fast

- `D` is computed **once**.
- Each permutation only changes `mask_a`, and the statistic is computed using a few dense linear algebra operations.
- No huge intermediate arrays; memory stays manageable.

Note about including the diagonal This implementation averages with denominators (n^2), (m^2) (so it includes (D_{ii}=0) terms). Since (D_{ii}=0), including them mainly changes the exact scaling slightly; importantly, it's **consistent across permutations**, so the permutation p-value remains valid.

In [114…

```
import numpy as np
from sklearn.metrics import pairwise_distances
```

```python
# Key idea:
# Keep each page's time series intact (already done when you computed features from the full series).
# Test whether EN vs FR differ by shuffling language labels across pages.
# To make it computationally feasible, randomly sample e.g. 2000–5000 pages per language.

def energy_stat_fast(D, mask_a):
    """
    Energy statistic from a precomputed distance matrix D (N x N),
    where mask_a is a boolean vector for group A membership.
    Uses matrix-vector products (no huge submatrix copies).
    """
    g = mask_a.astype(np.float32)
    n = float(g.sum())
    m = float(len(g) - n)
    if n < 2 or m < 2:
        return np.nan

    gb = 1.0 - g

    # Mean distances (including diagonal; consistent across permutations)
    d_ab = (g @ D @ gb) / (n * m)
    d_aa = (g @ D @ g)  / (n * n)
    d_bb = (gb @ D @ gb) / (m * m)

    return float(2.0 * d_ab - d_aa - d_bb)

def perm_test_energy_subsample(X, labels, lang_a="en", lang_b="fr",
                               k_per_group=3000, n_perm=1000, seed=0):
    """
    X: feature matrix (n_pages, p)
    labels: language labels (n_pages,)
    Subsamples k_per_group from each language, precomputes distances once,
    then runs label permutations.
    """
    rng = np.random.default_rng(seed)
    labels = np.asarray(labels)

    ia = np.where(labels == lang_a)[0]
    ib = np.where(labels == lang_b)[0]
    if len(ia) == 0 or len(ib) == 0:
        raise ValueError("One of the languages has 0 pages.")

    ka = min(k_per_group, len(ia))
    kb = min(k_per_group, len(ib))

    sa = rng.choice(ia, size=ka, replace=False)
    sb = rng.choice(ib, size=kb, replace=False)

    idx = np.concatenate([sa, sb])
    Xs = X[idx].astype(np.float32, copy=False)
    labs = labels[idx]

    # Precompute distances once (N x N). With N=6000 float32 ~ 144MB.
    D = pairwise_distances(Xs, metric="euclidean", n_jobs=-1).astype(np.float32)

    mask_a = (labs == lang_a)
    obs = energy_stat_fast(D, mask_a)

    perm_stats = np.empty(n_perm, dtype=np.float32)
    for i in range(n_perm):
        perm_mask = rng.permutation(mask_a)
        perm_stats[i] = energy_stat_fast(D, perm_mask)

    pval = (1 + np.sum(perm_stats >= obs)) / (n_perm + 1)
    return obs, pval, perm_stats

# Example usage:
obs, pval, _ = perm_test_energy_subsample(
    X, lang_keep,
    lang_a="en", lang_b="fr",
    k_per_group=3000,
    n_perm=1000,
    seed=0
)
print("Energy statistic:", obs)
print("Permutation p-value:", pval)
```

```
Energy statistic: 0.2497506666666669
Permutation p-value: 0.000999000999000999
```

- **How to interpret**
  - pval < 0.05: EN vs FR pages differ systematically (in the chosen features) → language is likely a useful segmentation.

- pval >= 0.05: no clear evidence language separates them (by these features).

Conclusion from this test

- **EN vs FR pages are not drawn from the same distribution of time-series behavior** (as summarized by our features: mean/std/quantiles/zero-rate/autocorr/weekday strength).
- We can **reject the null** “English and French pages are similar (in these features)” because the permutation p-value is **~0.001**.

So **language is a statistically strong separator** in our data

What the numbers mean

- **Energy statistic = 0.2498**: an effect-size measure of how far apart the two groups are in feature space (bigger = more separable).
- **Permutation p-value ≈ 0.001**: in 1000 permutations, only ~1 produced a separation at least as large as observed. So it’s very unlikely this separation is due to random assignment of language labels.

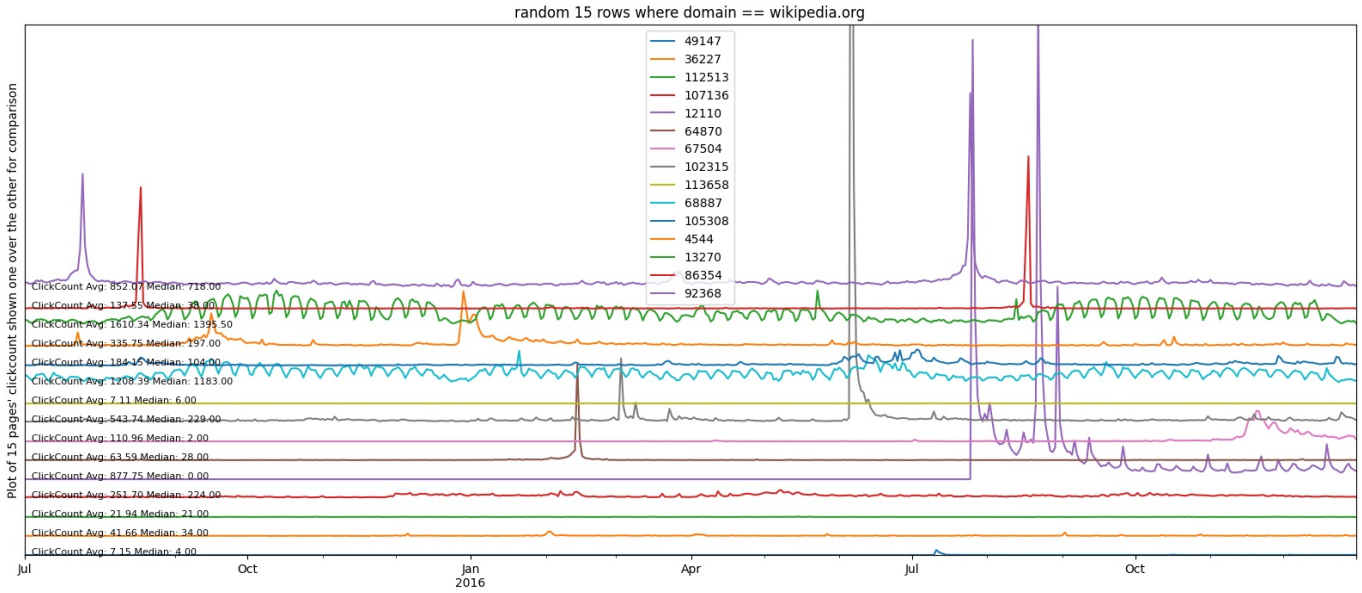
Implication for model creation

- This **supports** using language as a segmentation criterion (EN behaves differently from FR in distribution/dynamics).
- We may consider creating separate models based on languages.

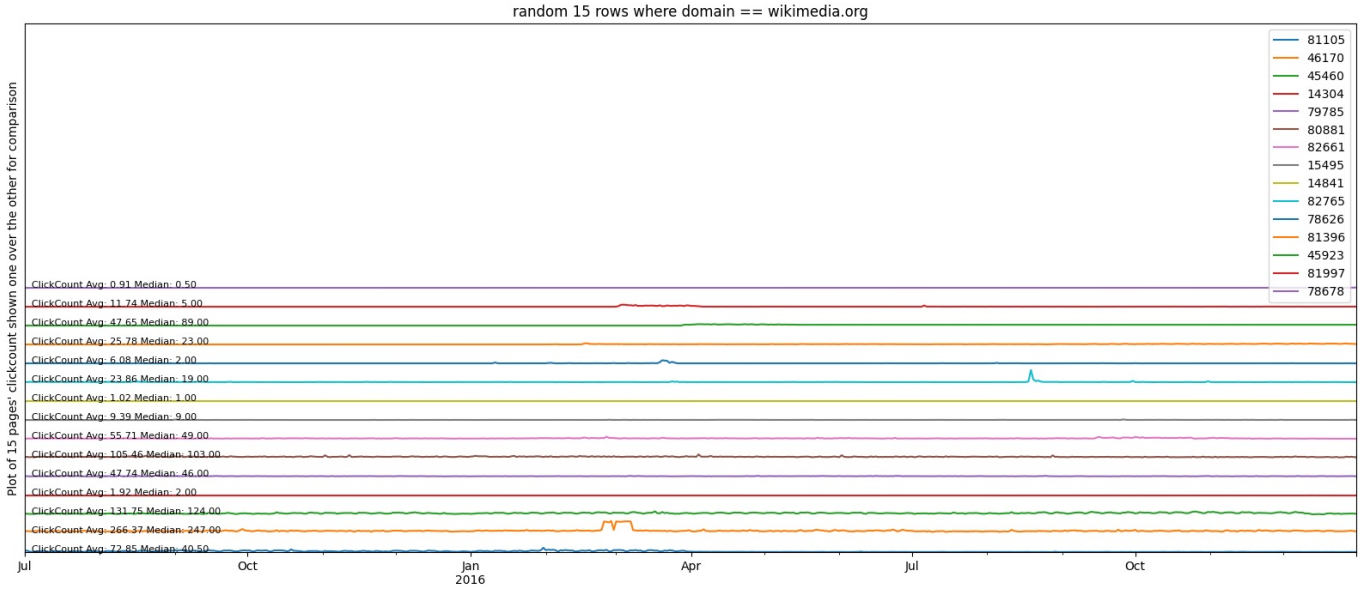
ClickCount and domain

In [114…
```
plotSamplesFrom('domain')
```

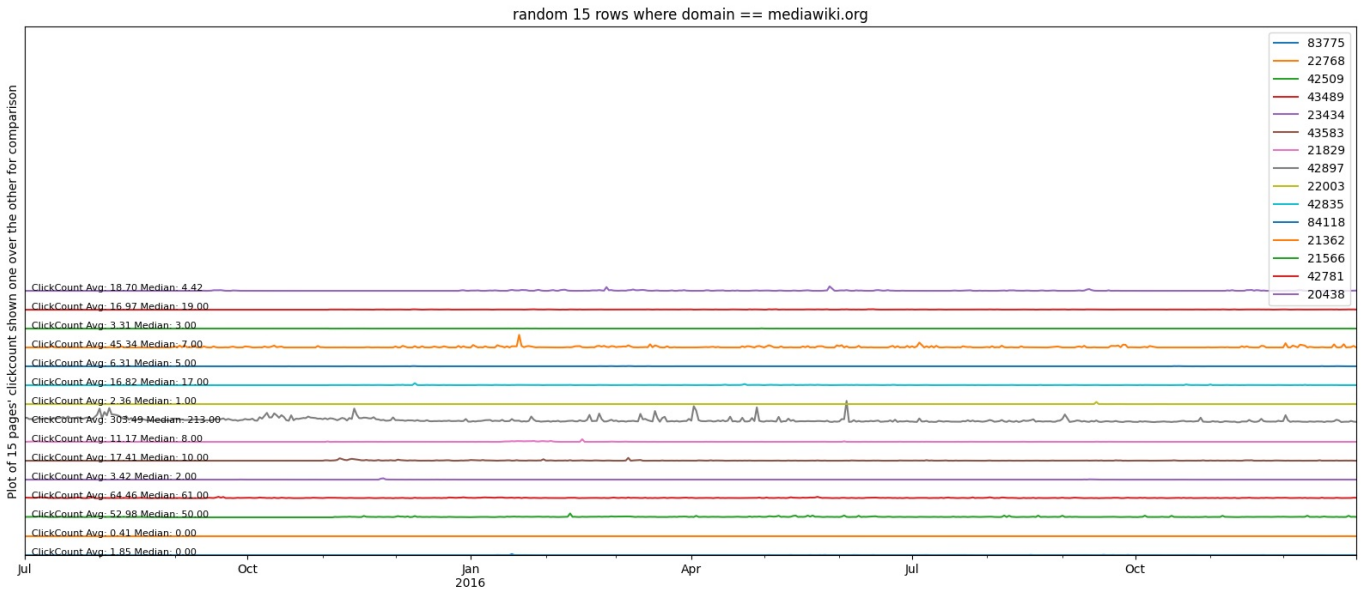
```
search_data():Searched random 15 rows where domain == wikipedia.org
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```




```
search_data():Searched random 15 rows where domain == wikimedia.org
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```




```
search_data():Searched random 15 rows where domain == mediawiki.org
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 15 rows
```



- Like language, domain too has a significant difference in distribution. It can be used as a criteria for segmenting and creating separate model based on domain.

annotate_bars

Helper function to add values on top of bars.

In [114…

```
#Helper function to add values on top of bars.
def annotate_bars(barplot,fmt='%.1f'):
    # --- Annotate the bars ---
```

```
    for container in barplot.containers:
        barplot.bar_label(
            container,
            fmt=fmt,       # Format as 1 decimal + %
            label_type='edge',  # Label outside bar; use 'center' to put inside
            fontsize=9,
            padding=2
        )
```

In [114…

```
from scipy.stats import chi2_contingency



def perform_chi_square_test(df, col1, col2, alpha=0.05,log="detailed", **kwargs):
    """
    Perform Chi-square test on two categorical columns from a DataFrame
    Also checks the Cramer's V value to establish if the association is of practical significance or only statis

    Parameters:
    df: pandas DataFrame
    col1: string, name of first categorical column
    col2: string, name of second categorical column
    alpha: float, significance level (default 0.05)

    Returns:
    dict with test results
    """
    li="<li>"
    # Create contingency table
    contingency_table = pd.crosstab(df[col1], df[col2])


    # Perform chi-square test
    Ho=f"{li}{col1} and {col2} are independent."
    Ha=f"{li}{col1} and {col2} are DEPENDENT"
    chi2_stat, p_value, dof, expected = chi2_contingency(contingency_table)

    # Compute effect size (Cramér's V)
    n = contingency_table.sum().sum()
    phi2 = chi2_stat / n
    r, k = contingency_table.shape
    cramers_v = (phi2 / min(k-1, r-1)) ** 0.5

    # p-value Interpretation
    if p_value < 0.05:
        relation = "Dependent (Significant)"
    else:
        relation = "Independent (Not significant)"


    # Cramer V Effect size Interpretation
    if cramers_v < 0.1:
        effectDesc = f"However, association is Negligible(CramersV={cramers_v:.2f})"
    elif cramers_v < 0.3:
        effectDesc = f"However, association is Weak(CramersV={cramers_v:.2f})"
    elif cramers_v < 0.5:
        effectDesc = f"And, association is Moderate(CramersV={cramers_v:.2f})"
    else:
        effectDesc = f"And, association is Strong(CramersV={cramers_v:.2f})"

    # Results
    results = {
        'chi2_statistic': chi2_stat,
        'p_value': p_value,
        'degrees_of_freedom': dof,
        'expected_frequencies': expected,
        'contingency_table': contingency_table,
        'significant': (p_value < alpha)
    }

    # Print results
    if("detailed" in log):
        print("Contingency Table:")
        print(contingency_table)
        print("\n")
        print(f"Chi-square Test Results:")
        print(f"Chi-square statistic: {chi2_stat:.4f}")
        print(f"p-value: {p_value:.4f}")
        print(f"Degrees of freedom: {dof}")
        print(f"Significant at α={alpha}: {'Yes' if results['significant'] else 'No'}")

    if results['significant']:
        if("detailed" in log):
```

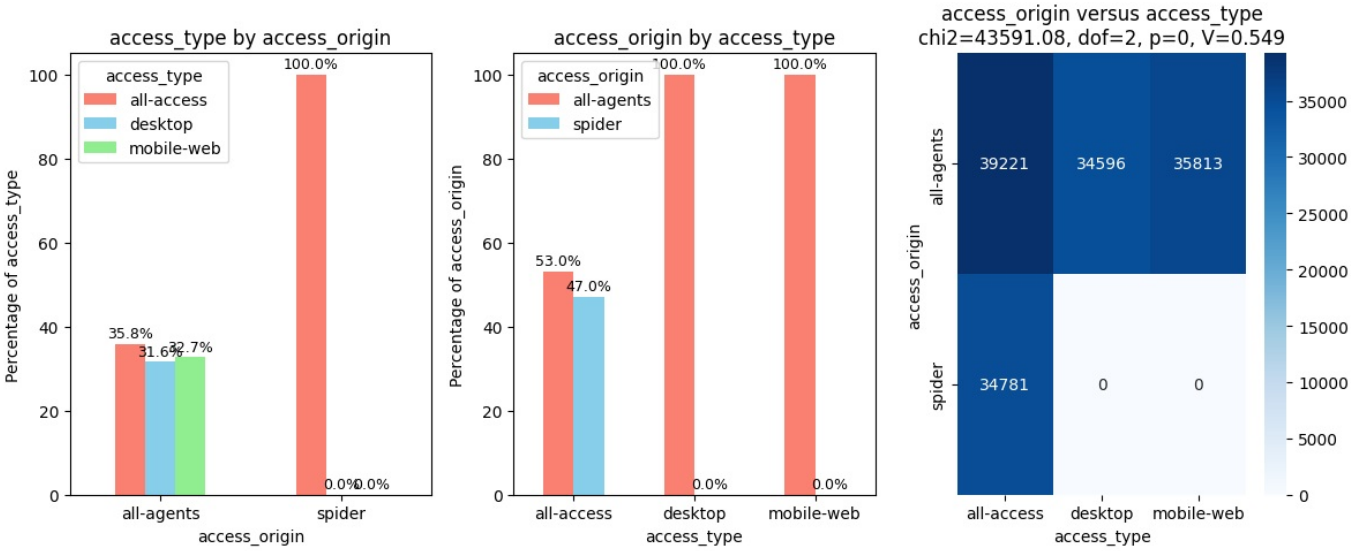
```
            print(f"Reject Null Hypothesis. Alternate Hypothesis '{Ha}' is True, p-value:{results['p_value']}. 
        else:
            print(f'{Ha}(p:{results['p_value']:.2f}), however {effectDesc}')
    else:
        if("detailed" in log):
            print(f"Fail to Reject Null Hypothesis. Null Hypothesis '{Ho}' is True, p-value:{results['p_value']}
        else:
              print(f'{Ho}(p:{results['p_value']:.2f})')

    if("noplot" not in log):

        # We create two crosstabs
        crossNormalized12 = pd.crosstab(df[col1], df[col2], normalize='index') * 100
        crossNormalized21 = pd.crosstab(df[col2], df[col1], normalize='index') * 100

        # Create 1x3 subplot figure

        fig, axes = plt.subplots(1, 3, figsize= kwargs.get("figsize", (12, 5))) #pass figsize while calling

        colors = [
            "salmon", "skyblue",
            "lightgreen", "gold", "orchid", "coral", "khaki",
            "plum", "turquoise", "lightsteelblue", "tan", "lightpink",
            "mediumseagreen", "wheat", "lightsalmon", "aquamarine",
            "thistle", "palegoldenrod", "powderblue", "peru", "lightcyan",
            "violet", "darkseagreen", "peachpuff", "mistyrose"
        ]
        # determine how many categories you have
        num_categories = len(crossNormalized12.columns)

        # now slice the colors list accordingly
        color_list = colors[:num_categories]
        # --- First Plot: Col2 by Col1 ---
        bars = crossNormalized12.plot(kind='bar', stacked=False, ax=axes[0], color=color_list)
        axes[0].set_title(f"{col2} by {col1}")
        axes[0].set_ylabel(f"Percentage of {col2}")
        axes[0].set_xlabel(col1)
        axes[0].legend(title=col2)
        axes[0].tick_params(axis='x', rotation=0)
        annotate_bars(bars,'%.1f%%')

        # --- Second Plot: Col1 by Col2 ---
        bars = crossNormalized21.plot(kind='bar', stacked=False, ax=axes[1], color=color_list)
        axes[1].set_title(f"{col1} by {col2}")
        axes[1].set_ylabel(f"Percentage of {col1}")
        axes[1].set_xlabel(col2)
        axes[1].legend(title=col1)
        axes[1].tick_params(axis='x', rotation=0)
        annotate_bars(bars,'%.1f%%')

# pd.crosstab(df[col1], df[col2], normalize='index') * 100
        ct = pd.crosstab(df[col1], df[col2])
        sns.heatmap(ct, annot=True, fmt="d", cmap="Blues", ax=axes[2])
        plt.title(f"{col1} versus {col2}\nchi2={chi2_stat:.2f}, dof={dof}, p={p_value:.3g}, V={cramers_v:.3f}")
        plt.tight_layout()
        plt.show()

        plt.show()

    if("detailed" in log):
        print("="*50)
```

access_origin and access_type

In [114…

```
# _ = chi2_categorical_pair(df, "access_origin", "access_type", plot=True, annot=True)
perform_chi_square_test(df, "access_origin", "access_type")
```

```
Contingency Table:
access_type    all-access  desktop  mobile-web
access_origin
all-agents          39221    34596       35813
spider              34781        0           0


Chi-square Test Results:
Chi-square statistic: 43591.0816
p-value: 0.0000
Degrees of freedom: 2
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>access_origin and access_type are DEPENDENT' is True, p-value:
0.0. And, association is Strong(CramersV=0.55)
```



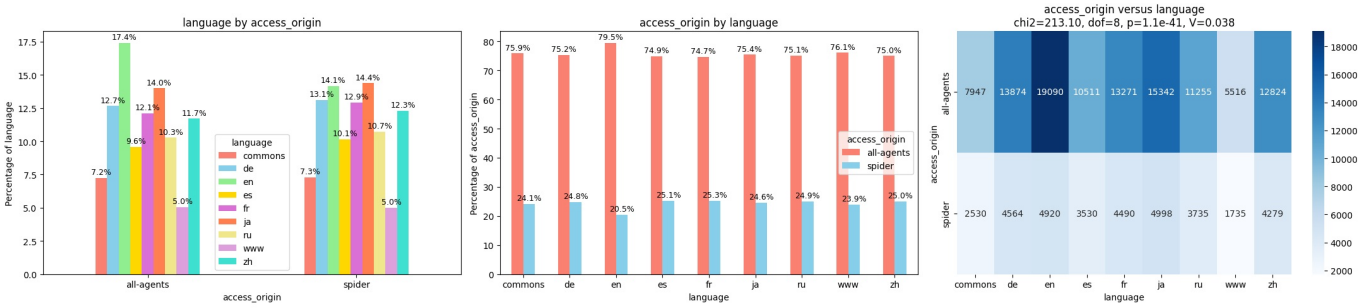
==========

- access_origin and access_type are DEPENDENT. p-value:0.0. And association is Strong(CramersV=0.55)

language and access_origin

In [114…
```
perform_chi_square_test(df, "access_origin", "language",figsize=(22, 5))
```

```
Contingency Table:
language       commons     de     en     es     fr     ja     ru   www     zh
access_origin
all-agents        7947  13874  19090  10511  13271  15342  11255  5516  12824
spider            2530   4564   4920   3530   4490   4998   3735  1735   4279


Chi-square Test Results:
Chi-square statistic: 213.1040
p-value: 0.0000
Degrees of freedom: 8
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>access_origin and language are DEPENDENT' is True, p-value:1.1
0122301337787 92e-41. However, association is Negligible(CramersV=0.04)
```



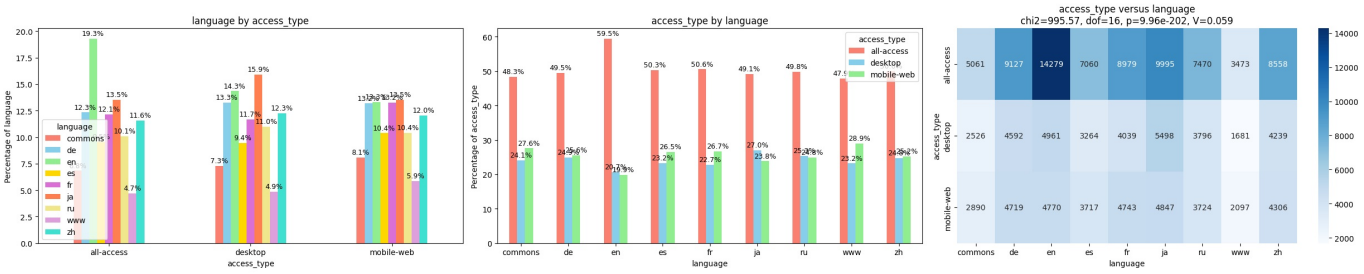
==========

- access_origin and language are DEPENDENT. p-value:1.1012230133778792e-41. However, association is Negligible(CramersV=0.04)

language and access_type

In [114…
```
perform_chi_square_test(df, "access_type", "language",figsize=(25, 5))
```

```
Contingency Table:
language     commons    de     en    es    fr    ja    ru   www    zh
access_type
all-access      5061  9127  14279  7060  8979  9995  7470  3473  8558
desktop         2526  4592   4961  3264  4039  5498  3796  1681  4239
mobile-web      2890  4719   4770  3717  4743  4847  3724  2097  4306


Chi-square Test Results:
Chi-square statistic: 995.5668
p-value: 0.0000
Degrees of freedom: 16
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>access_type and language are DEPENDENT' is True, p-value:9.963
57592036832e-202. However, association is Negligible(CramersV=0.06)
```



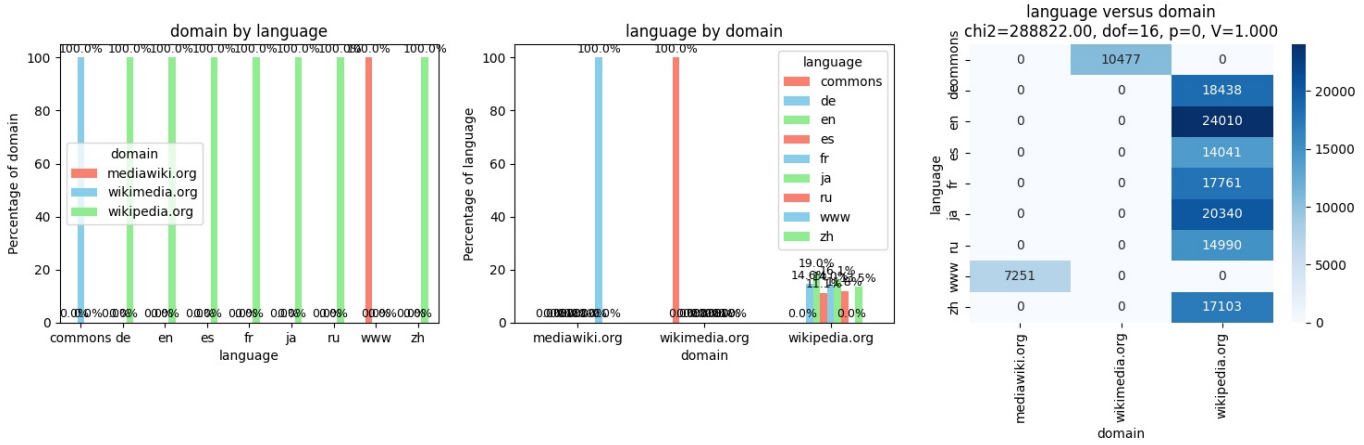
==========

- access_type and language are DEPENDENT' is True, p-value:9.96357592036832e-202. However, association is Negligible(CramersV=0.06)

language and domain

In [114…
```
perform_chi_square_test(df, "language", "domain",figsize=(15, 5))
```

```
Contingency Table:
domain    mediawiki.org  wikimedia.org  wikipedia.org
language
commons               0          10477              0
de                    0              0          18438
en                    0              0          24010
es                    0              0          14041
fr                    0              0          17761
ja                    0              0          20340
ru                    0              0          14990
www                7251              0              0
zh                    0              0          17103


Chi-square Test Results:
Chi-square statistic: 288822.0000
p-value: 0.0000
Degrees of freedom: 16
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>language and domain are DEPENDENT' is True, p-value:0.0. And,
association is Strong(CramersV=1.00)
```

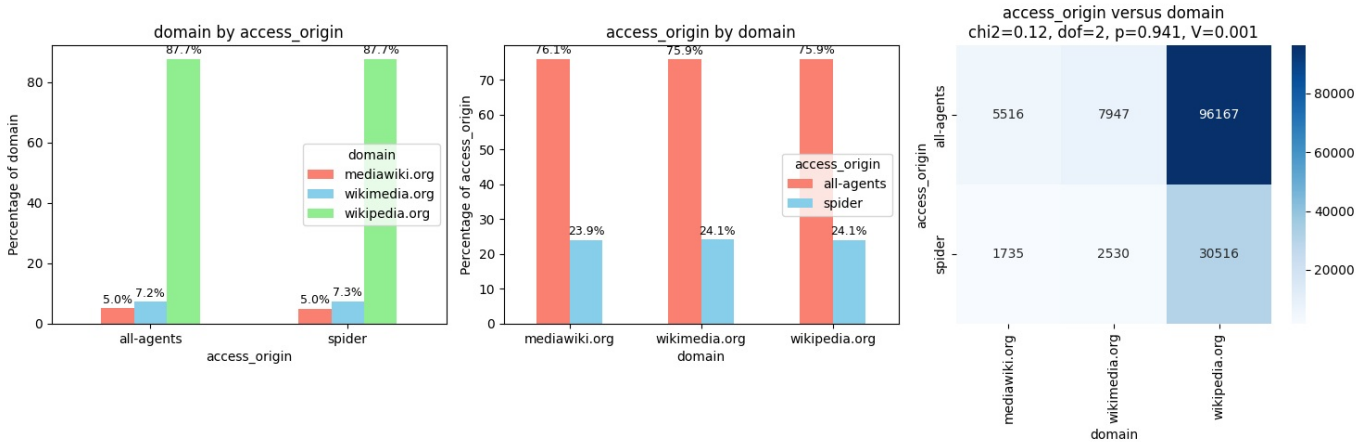


==========

- language and domain are DEPENDENT, p-value:0.0. And, association is Strong(CramersV=1.00)
- All www marked pages belong to mediawiki.org
- All commons marked pages belong to wikimedia.org
- All language marked pages belong to wikipedia.org
- This corroborates our understanding - wikipedia.org has the text content, so it has language-based pages, while wikimedia has audio, video and images, which are (supposedly) language agnostic (which may not be technically correct,but the case study has this assumption, there is no marker on the media files what language they are in.)

In [114…
```
perform_chi_square_test(df, "access_origin", "domain",figsize=(15, 5))
```

```
Contingency Table:
domain         mediawiki.org  wikimedia.org  wikipedia.org
access_origin
all-agents              5516           7947          96167
spider                  1735           2530          30516


Chi-square Test Results:
Chi-square statistic: 0.1218
p-value: 0.9409
Degrees of freedom: 2
Significant at α=0.05: No
Fail to Reject Null Hypothesis. Null Hypothesis '<li>access_origin and domain are independent.' is True, p-value
:0.9409393320466397
```



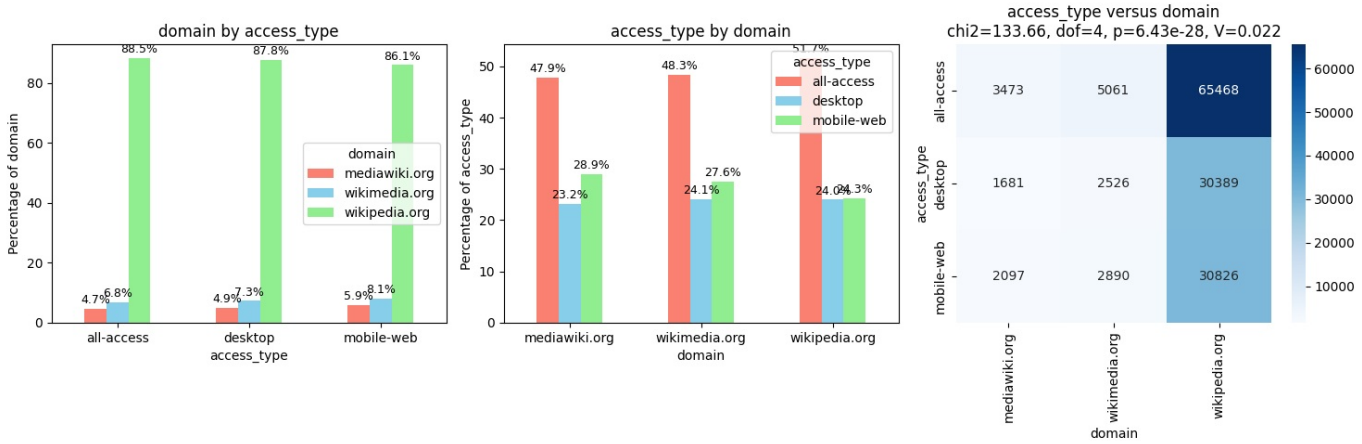
==========

- access_origin and domain are independent

access_type and domain

```
In [114… # _ = chi2_categorical_pair(df, "access_type", "domain", plot=True, annot=True)
perform_chi_square_test(df, "access_type", "domain",figsize=(15, 5))
```

```
Contingency Table:
domain       mediawiki.org  wikimedia.org  wikipedia.org
access_type
all-access            3473           5061          65468
desktop               1681           2526          30389
mobile-web            2097           2890          30826


Chi-square Test Results:
Chi-square statistic: 133.6559
p-value: 0.0000
Degrees of freedom: 4
Significant at α=0.05: Yes
Reject Null Hypothesis. Alternate Hypothesis '<li>access_type and domain are DEPENDENT' is True, p-value:6.43290
5337976179e-28. However, association is Negligible(CramersV=0.02)
```



==========

- access_type and domain are DEPENDENT' is True, p-value:6.432905337976179e-28. However, association is Negligible(CramersV=0.02)

Multivariate Analysis

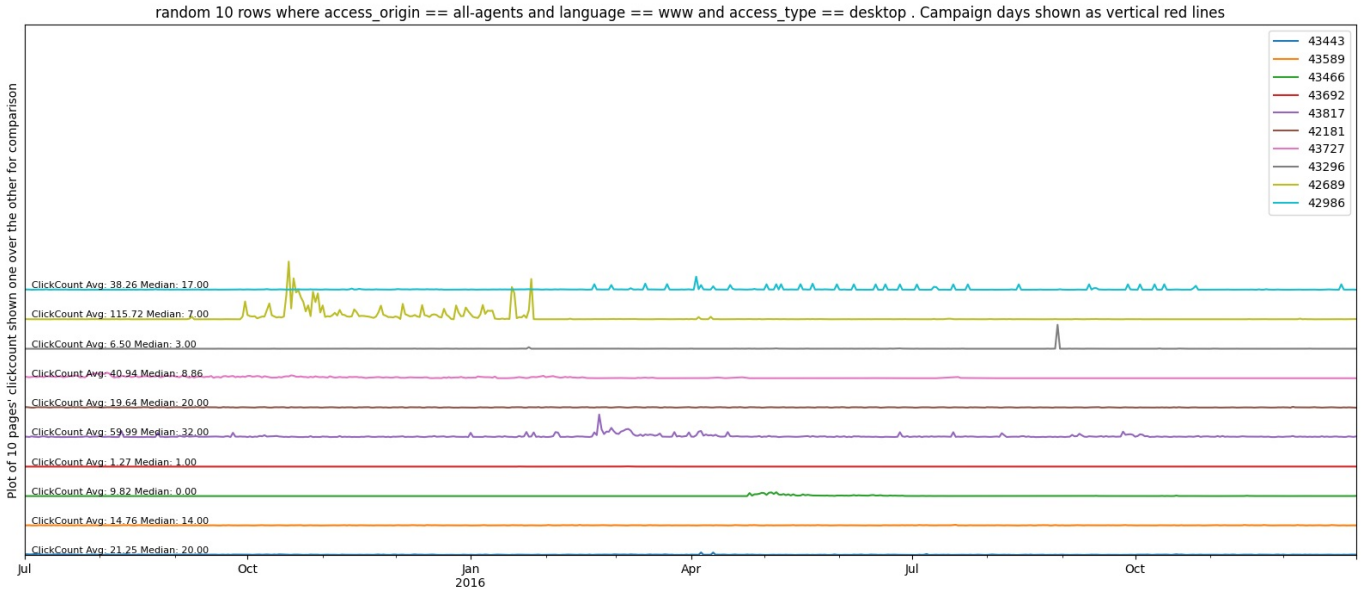
Language, access_origin, access_type and clickCount

We want to see if access_origin is non spider, then for various languages, does clickCount change based on access_type

We first check for language=www and access_type=desktop

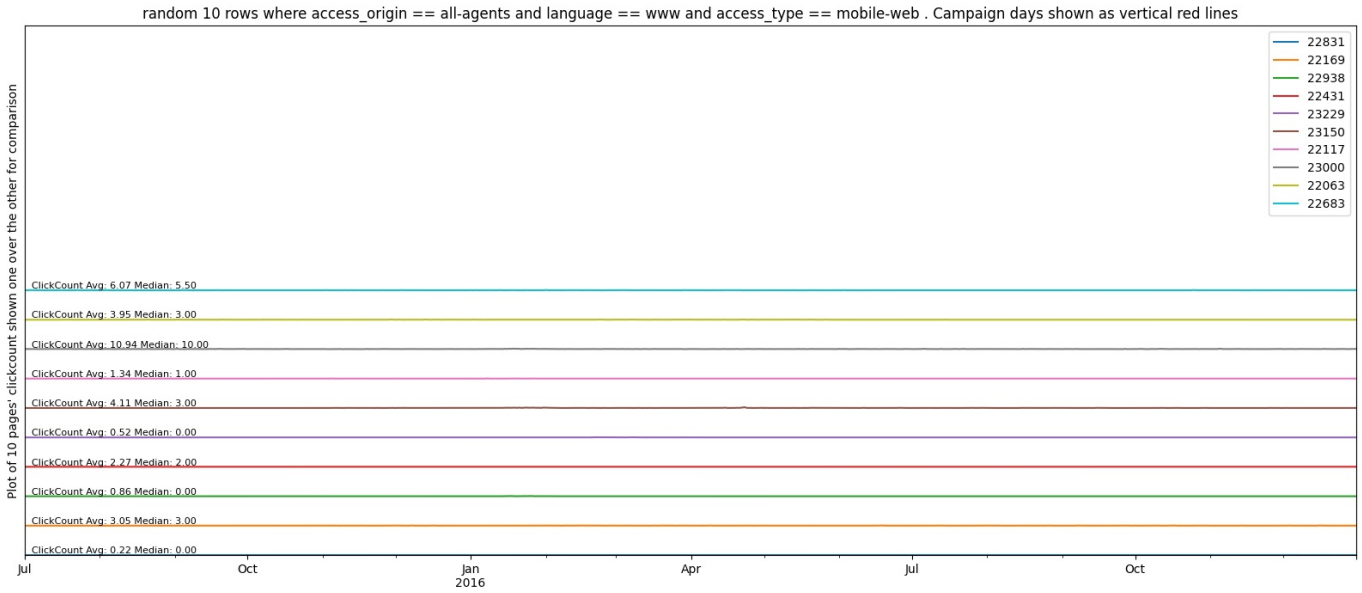
In [115…
```
rows,title=search_data(10,
                       colName1='access_origin',val1='all-agents',
                       colName2='language',val2='www',
                       colName3='access_type',val3='desktop')
plot_data_staggered(rows, title, campaign=True)
```

```
search_data():Searched random 10 rows where access_origin == all-agents and language == www and access_type == d
esktop
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 10 rows
```



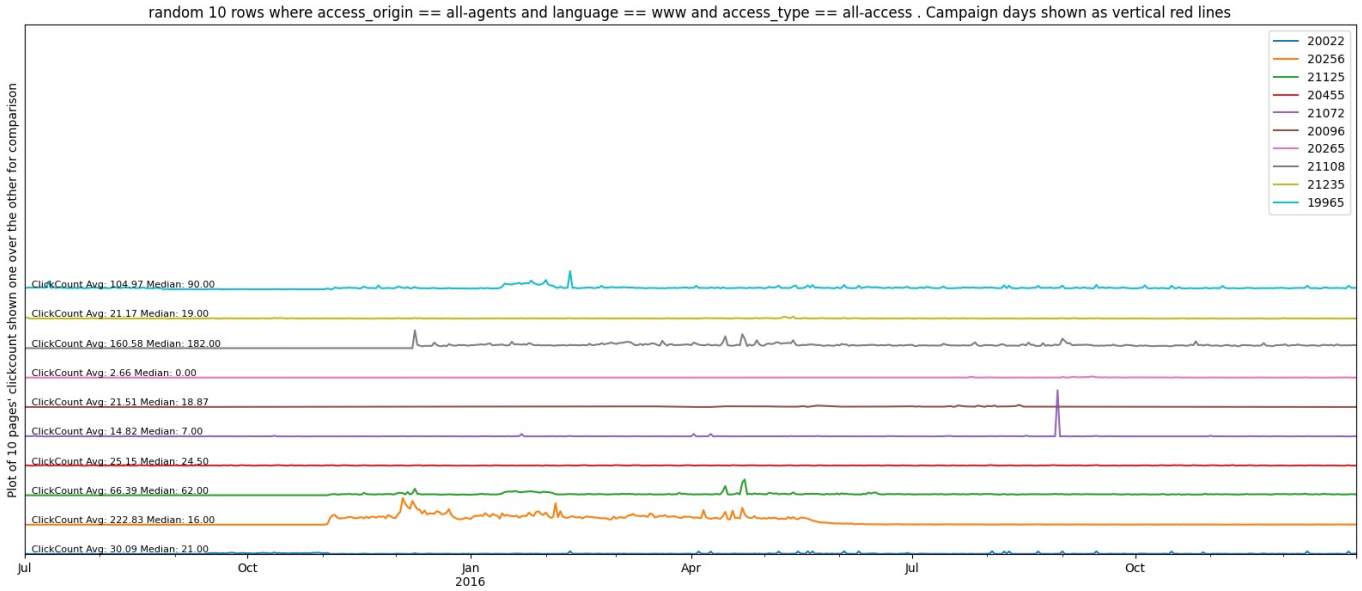
In [115…
```
rows,title=search_data(10,
                       colName1='access_origin',val1='all-agents',
                       colName2='language',val2='www',
                       colName3='access_type',val3='mobile-web')
plot_data_staggered(rows, title, campaign=True)
```

```
search_data():Searched random 10 rows where access_origin == all-agents and language == www and access_type == m
obile-web
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 10 rows
```



In [115…
```
rows,title=search_data(10,
                       colName1='access_origin',val1='all-agents',
                       colName2='language',val2='www',
                       colName3='access_type',val3='all-access')
plot_data_staggered(rows, title, campaign=True)
```

```
search_data():Searched random 10 rows where access_origin == all-agents and language == www and access_type == a
ll-access
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 10 rows
```



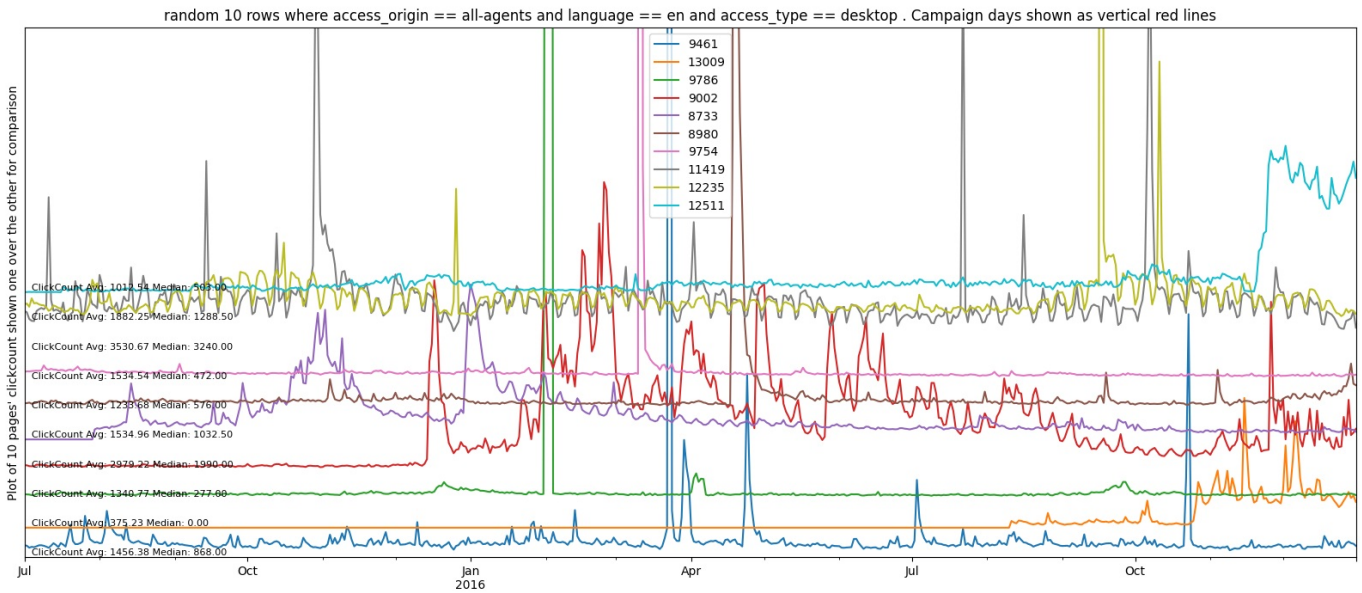
Conclusion - for language www there is no difference for differnt access_types.

Lets now check for a real language, say english

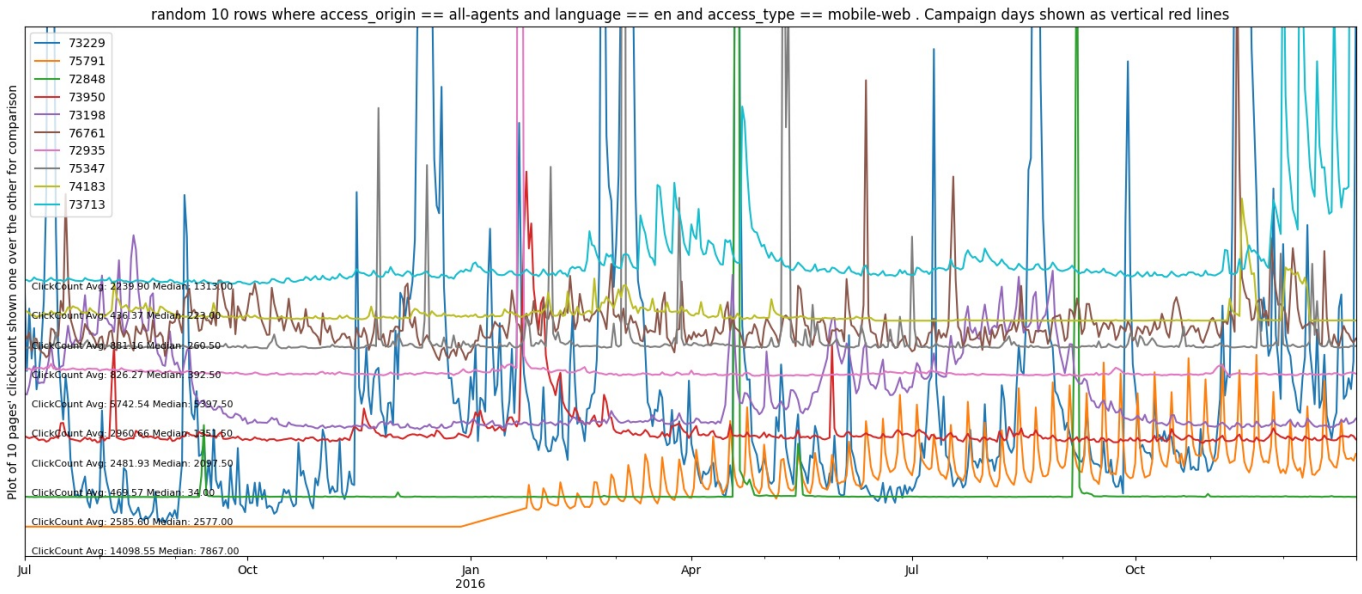
In [115…

```
lang='en'
rows,title=search_data(10,
                       colName1='access_origin',val1='all-agents',
                       colName2='language',val2=lang,
                       colName3='access_type',val3='desktop')
plot_data_staggered(rows, title, campaign=True)


rows,title=search_data(10,
                       colName1='access_origin',val1='all-agents',
                       colName2='language',val2=lang,
                       colName3='access_type',val3='mobile-web')
plot_data_staggered(rows, title, campaign=True)


rows,title=search_data(10,
                       colName1='access_origin',val1='all-agents',
                       colName2='language',val2=lang,
                       colName3='access_type',val3='all-access')
plot_data_staggered(rows, title, campaign=True)
```

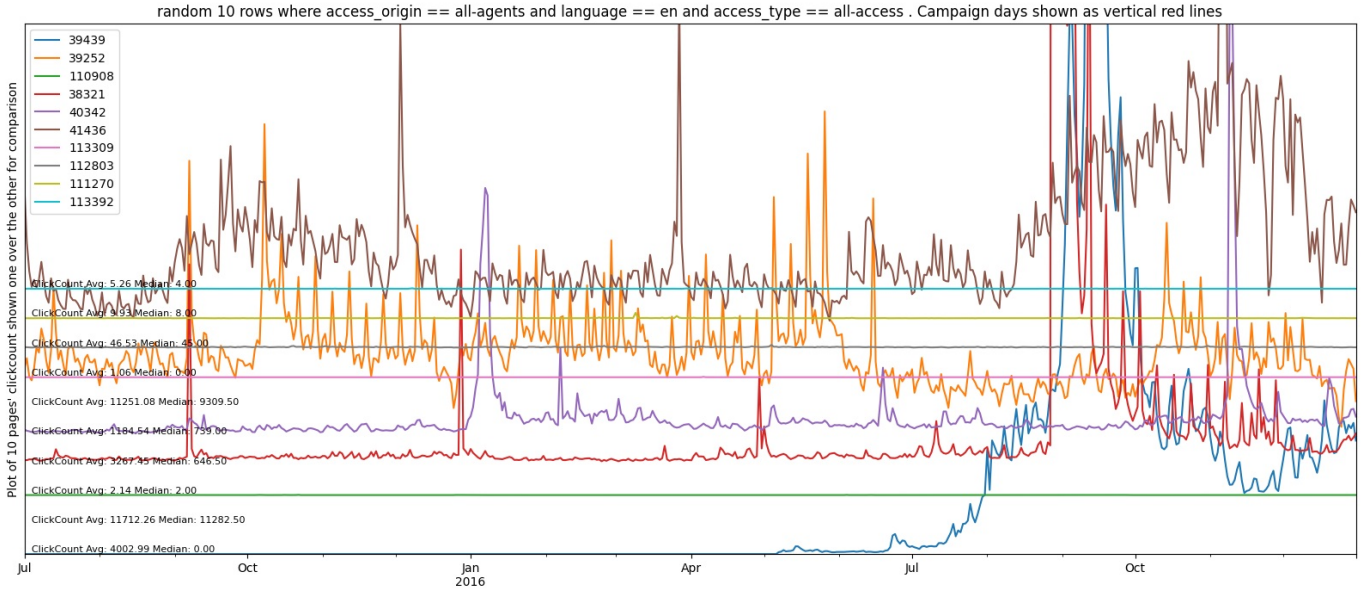
```
search_data():Searched random 10 rows where access_origin == all-agents and language == en and access_type == de
sktop
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 10 rows
```




```
search_data():Searched random 10 rows where access_origin == all-agents and language == en and access_type == mo
bile-web
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 10 rows
```




```
search_data():Searched random 10 rows where access_origin == all-agents and language == en and access_type == al
l-access
extractRowTransposeColumn():Transposing ALL rows from given dataframe that has 10 rows
```



- Results are similar for en too across different access_types, so **access type cannot be used for data segmentation** even for non spider pages

Segmentation logic

- We have figured out from all the EDA analysis that `access_origin` and `language`s are two good criteria for segmenting the data.
- We will create a separate model for each segment so that hopefully the model can predict better compared to if a single model is used across all data.
- The CARD SHAPE in the diagram show the MODEL NAMES

Aggregated Modeling Strategy:

- Rather than building separate models for each of the ~145,000 Wikipedia pages, we will aggregate pageviews at the segment level (e.g., all Spider traffic, all English non-Spider traffic).
- This approach offers three advantages:
  - (1) computational efficiency—training 10 models instead of 145,000;
  - (2) statistical robustness—aggregation smooths outliers and sparse pages with near-zero traffic;
  - (3) alignment with business objectives—segment-level forecasts support capacity planning and resource allocation decisions. Model parameters shall be validated on multiple sampled pages before applying to the full aggregated series.

Brief Summary of Multi-Series Forecasting Options

> **Alternative Approaches Considered:**

> - **Aggregated Model:** Sum all pages into one series per segment—simple, efficient, ideal for segment-level forecasts (chosen approach)
> - **Multi-Sample Parameter Tuning:** Test parameters on 50-100 random pages to find robust settings that generalize across pages
> - **Global ML Model:** Train one machine learning model (e.g., LightGBM) that learns patterns from all series simultaneously for page-level predictions
> - **Cluster-then-Forecast:** Group similar pages using clustering, then train separate models per cluster
> - **Deep Learning:** Use neural architectures (DeepAR, N-BEATS, Temporal Fusion Transformer) designed for large-scale multi-series forecasting

---

Approach	Description	Best For
Aggregated (Chosen)	Sum all pages → one model per segment	Segment-level forecasts
Multi-Sample Tuning	Validate params on 50-100 pages	Robust parameter selection
Global ML Model	One model learns from all series	Scalable page-level forecasts
Cluster + Forecast	Group similar pages, model per cluster	Heterogeneous page patterns
Deep Learning	Neural nets (DeepAR, N-BEATS)	Large-scale, complex patterns

---

- The graph shows that page views are different across various segments
- The EN segment is particularly unique.
- The campaign data for EN surely matches the time series.

In [115…
```
#Need to fix this.
# plot_transposed_data_staggered(timeSeriesFull_df[[modelNames[3]]], title="All segments", staggerByY=100, campa
```

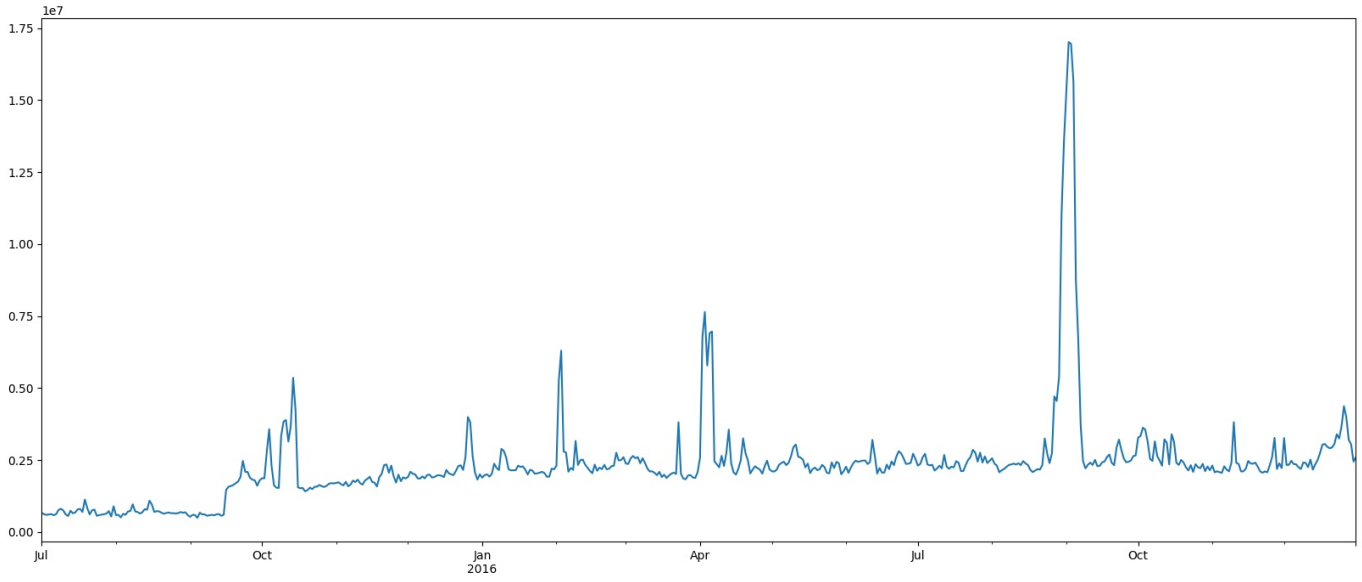
4.0 Modeling

4.1 Pipeline Flow Design

Take a sample data and analyse

In [115…
```
timeSeriesFull_df.iloc[:,0].plot()
```

Out[115… <Axes: >



Stationarity check (ADF test)

In [115…
```
from statsmodels.tsa.stattools import adfuller
```

```
print("Testing for ALL segments")

def performADFTestForAllSegments(full_df):
    results = []

    for i in range(len(full_df.columns)):
        adf_result = adfuller(full_df.iloc[:,i].dropna())
        status = "Stationary" if adf_result[1] < 0.05 else "Non-Stationary"
        results.append([modelNames[i], f'{adf_result[1]:.4e}', status])

    print(tabulate(results, headers=['Model', 'P-Value', 'Status'], tablefmt='simple'))

performADFTestForAllSegments(timeSeriesFull_df)
```

```
Testing for ALL segments
Model                        P-Value  Status
-------------------------  ----------  --------------
ModelSpider                1.2515e-05  Stationary
ModelNonSpiderLangCommons  0.059171    Non-Stationary
ModelNonSpiderLangDe       0.15129     Non-Stationary
ModelNonSpiderLangEn       0.18904     Non-Stationary
ModelNonSpiderLangEs       0.037999    Stationary
ModelNonSpiderLangFr       0.045776    Stationary
ModelNonSpiderLangJa       0.088203    Non-Stationary
ModelNonSpiderLangRu       0.0019096   Stationary
ModelNonSpiderLangWww      1.4259e-08  Stationary
ModelNonSpiderLangZh       0.43604     Non-Stationary
```

- **Interpreting the ADF test results**

The **Augmented Dickey–Fuller (ADF) test** checks for a unit root:

- **Null hypothesis (H_0):** series is **non-stationary**
- **Alternative (H_1):** series is **stationary**
- Typically, **p-value < 0.05** ⇒ **reject H_0** ⇒ **stationary**

Segment	p-value	Interpretation
ModelSpider	1.25e-05	✅ Stationary
NonSpiderLangCommons	0.059	❌ Non-stationary (borderline)
NonSpiderLangDe	0.151	❌ Non-stationary
NonSpiderLangEn	0.189	❌ Non-stationary
NonSpiderLangEs	0.038	✅ Stationary
NonSpiderLangFr	0.046	✅ Stationary
NonSpiderLangJa	0.083	❌ Non-stationary
NonSpiderLangRu	0.0019	✅ Stationary
NonSpiderLangWww	1.43e-08	✅ Stationary
NonSpiderLangZh	0.436	❌ Non-stationary

Key observations

1. **Heterogeneous stationarity across languages**
   - Some language segments are stationary (ES, FR, RU, WWW)
   - Others are clearly non-stationary (EN, DE, ZH)
2. **English & large Wikipedias**
   - EN, DE, ZH being non-stationary is very common
   - These usually show:
     - Strong trends
     - Structural breaks (growth, seasonality, events)
3. **Borderline cases**
   - `NonSpiderLangCommons (p=0.059)` and `JA (p=0.083)`
   - Likely **trend-stationary or seasonally non-stationary**

What this means for modeling

For stationary segments We can directly use:

- **ARMA**
- **ARIMA with d=0**
- **VAR (if multivariate)**

Examples:

- Spider
- ES, FR, RU, WWW

For non-stationary segments We should transform the data before modeling:

Common fixes

1. **First differencing**

```
y_diff = y.diff().dropna()
```
Then rerun ADF.

2. **Seasonal differencing** (very likely for click data)

```
y_seasonal_diff = y.diff(7)   # daily data
y.diff(30)                    # monthly seasonality
```

3. **Log + differencing**

```
y_log = np.log1p(y)
y_log_diff = y_log.diff().dropna()
```
✅ Wikipedia traffic almost always benefits from **log + seasonal differencing**.

ACF PACF

ACF and PACF can reveal the underlying dependence structure in a time series—specifically how current values relate to past values at different lags.

Model identification (AR / MA / ARIMA)

- PACF helps identify the order of an AR(p) model (sharp cutoff after lag p).
- ACF helps identify the order of an MA(q) model (sharp cutoff after lag q).

Detecting autocorrelation & seasonality

- Significant spikes at regular intervals indicate seasonal patterns (e.g., lag 7 for weekly seasonality).
- Helps confirm whether the data is truly independent over time or not.
- Checking stationarity and model adequacy
- Slowly decaying ACF suggests non-stationarity (need differencing).

ACF (Autocorrelation Function)

- Correlation between Y_t and Y_{t-k}
- Used mainly to identify q in **MA (q)**

PACF (Partial Autocorrelation Function)

- Direct correlation between Y_t and Y_{t-k}, removing intermediate effects
- Used mainly to identify p in **AR (p)**

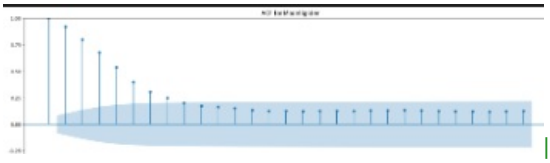
In [ ]:
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

def plotACFForAllSegments(full_df):
    for i in range(len(full_df.columns)):
            plot_acf(full_df.iloc[:,i].dropna(), title=f'ACF for {modelNames[i]}')

print("Plotting for ALL segments")
plotACFForAllSegments(timeSeriesFull_df)
```

- **How to Read the ACF Results Table**

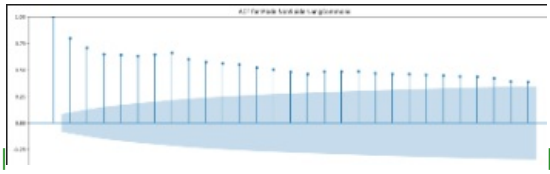
The ACF (Autocorrelation Function) answers one core question:

> **How strongly is the series correlated with its own past values at different lags?**

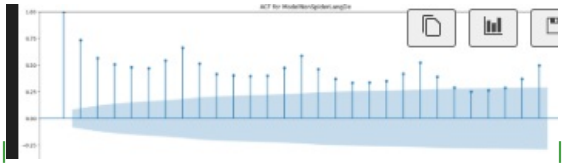
|Plot| Segment | Lag-1 Autocorrelation | ACF Decay Behavior | Oscillation / Cyclic Pattern | High-Lag Persistence | What ADF Result Said | |--|------|----------------------|------------------|----------------------------|--------------------|--------------------------| |



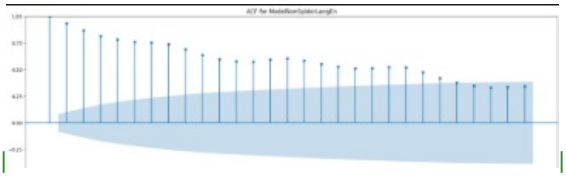
| **ModelSpider** | Very high | Smooth, monotonic decay | None | Moderate | ✅ Stationary | |



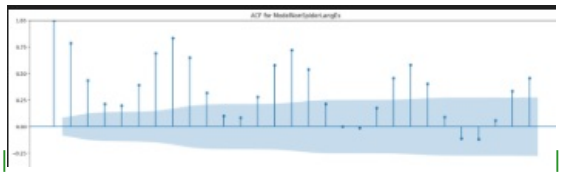
| **NonSpiderLangCommons** | Very high | Very slow decay | None | ❌ Non-stationary | |



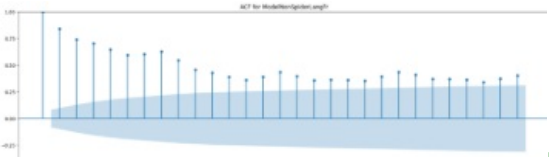
| **NonSpiderLangDe** | High | Slow decay | Present | Strong | ❌ Non-stationary | |



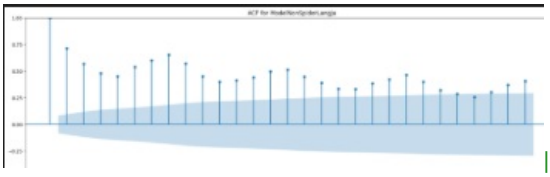
| **NonSpiderLangEn** | Very high | Extremely slow decay | Weak | ❌ Non-stationary | |



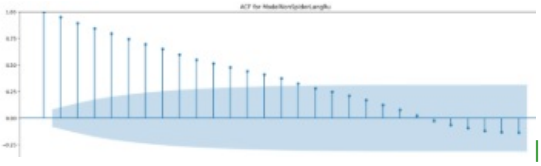
| **NonSpiderLangEs** | High | Non-monotonic decay | Strong | Moderate | ✅ Stationary | |



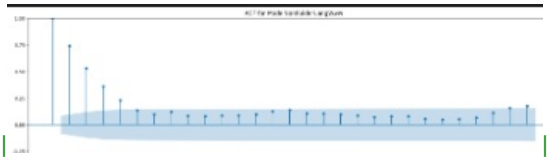
| **NonSpiderLangFr** | High | Gradual decay | Mild | Moderate | ✅ Stationary | |



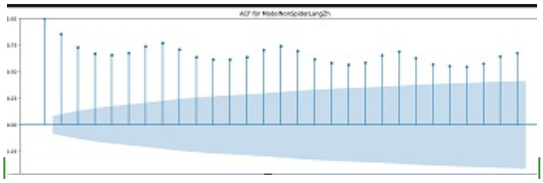
| **NonSpiderLangJa** | High | Slow decay | Moderate | Strong |❌ Non-stationary | |



| **NonSpiderLangRu** | Very high | Smooth decay | None | Moderate | ✅ Stationary | |



| **NonSpiderLangWww** | Moderate | Fast decay | None | Weak | ✅ Stationary | |



| **NonSpiderLangZh** | Very high | Very slow decay | Strong | ❌ Non-stationary |

---

1. **Lag-1 Autocorrelation**

What it is The autocorrelation at lag 1:

$$\rho(1) = \text{corr}(y_t, y_{t-1})$$

Why it's included

- It measures **immediate temporal dependence**
- It is the **single most informative ACF value**
- High lag-1 correlation implies strong inertia in the series

Why it matters

- Distinguishes:
  - noisy processes (low lag-1)
  - persistent processes (high lag-1)
- Helps compare segments **on the same scale**

Example interpretation:

- Lag-1 ≈ 0.9 → strong dependence
- Lag-1 ≈ 0.3 → weak memory

---

2. **ACF Decay Behavior**

What it describes ** The **shape of the ACF envelope as lag increases:

- Fast decay
- Slow decay
- Monotonic vs irregular

Why it's included The *rate* of decay contains information that **no single lag can capture**.

Why it matters Decay behavior tells us:

- Whether dependence is short-range or long-range
- Whether correlations die out quickly or persist

Typical patterns:

- **Fast decay** → short memory
- **Slow decay** → long memory or structural dependence

---

3. **Oscillation / Cyclic Pattern**

What it captures Whether the ACF:

- Alternates signs
- Shows repeating peaks
- Exhibits wave-like structure

Why it's included Oscillations indicate **systematic structure in correlations**, not random noise.

⚠ Important distinction:

- This **does NOT assert seasonality**
- It only states that correlations fluctuate in a patterned way

Why it matters

- Separates:
  - smooth AR-like decay
  - cyclic dependence
- Signals that **dependence is not purely monotonic**

---

4. **High-Lag Persistence**

What it means Whether autocorrelations:

- Remain statistically significant at large lags
- Or drop into the confidence band quickly

Why it's included High-lag behavior captures **global dependence**, not local structure.

Why it matters

- Long persistence suggests:
  - cumulative dependence
  - slow information decay
- Short persistence suggests:
  - rapid loss of memory

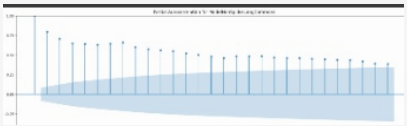
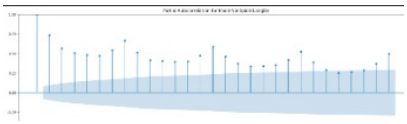
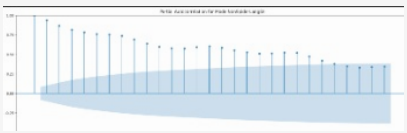
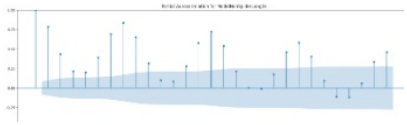
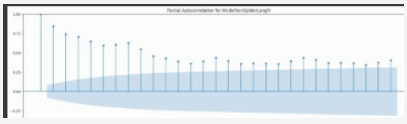
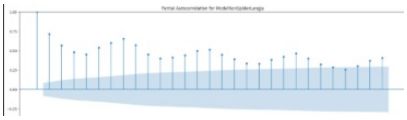
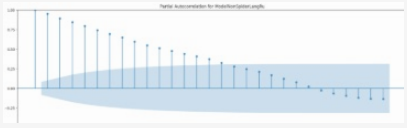
This column helps distinguish:

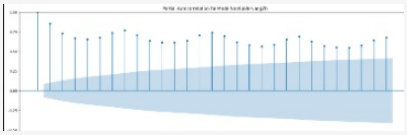
- series that “remember far back”
- series dominated by recent history only

✅ This is directly observable from the ACF confidence bounds.

In [ ]:
```
# plot_pacf(ts);

def plotPACFForAllSegments(full_df):
    for i in range(len(full_df.columns)):
            plot_acf(full_df.iloc[:,i].dropna(), title=f'Partial Autocorrelation for {modelNames[i]}')

print("Plotting for ALL segments")
plotPACFForAllSegments(timeSeriesFull_df)
```

- **PACF + ADF Summary Table**

Plot	Segment	ADF Result	PACF Pattern (Key Lags)	Conclusion
	**ModelSpider**	Stationary	Significant at multiple early lags, slow taper	Strong AR-type dependence
	**NonSpiderLangCommons**	Non-stationary	Significant across many lags	PACF dominated by non-stationarity
	**NonSpiderLangDe**	Non-stationary	Multiple significant, irregular spikes	Structure obscured by non-stationarity
	**NonSpiderLangEn**	Non-stationary	Very strong, persistent partial correlations	PACF not interpretable pre-differencing
	**NonSpiderLangEs**	Stationary	Isolated significant spikes, otherwise bounded	Finite AR structure plausible
	**NonSpiderLangFr**	Stationary	Gradual decay, no sharp cutoff	Weak to moderate AR dependence
	**NonSpiderLangJa**	Non-stationary	Persistent, oscillatory partial correlations	Differencing required before interpretation
	**NonSpiderLangRu**	Stationary	Strong early lags, tapering thereafter	Clear AR-type structure
	**NonSpiderLangWww**	Stationary	Significant at first few lags only	Short-memory AR behavior

	**NonSpiderLangZh**	Non-stationary	Strong partial correlations at many lags	PACF dominated by trend effects

---

PACF plots are interpretable for **ADF-stationary segments**, where they suggest finite autoregressive structure, while for **ADF-non-stationary segments** the PACF is dominated by persistence and requires differencing before meaningful order identification.

Next we will:

- Recompute **PACF after differencing** (only for non-stationary series)

Difference the identified columns

In [115…
```
timeSeriesFull_df
```

Out[115…

	ModelSpider	ModelNonSpiderLangCommons	ModelNonSpiderLangDe	ModelNonSpiderLangEn	ModelNonSpiderLangEs	Mod
2015-07-01	6.646480e+05	1.115089e+06	1.322512e+07	8.443447e+07	1.516379e+07	
2015-07-02	6.197182e+05	1.134739e+06	1.304097e+07	8.418241e+07	1.449318e+07	
2015-07-03	5.952534e+05	1.104986e+06	1.251964e+07	7.992619e+07	1.333192e+07	
2015-07-04	6.140900e+05	9.228855e+05	1.147857e+07	8.321397e+07	1.252267e+07	
2015-07-05	6.225934e+05	1.012272e+06	1.335621e+07	8.596247e+07	1.361801e+07	
...	...	...	...	...	...	
2016-12-27	3.983575e+06	2.461968e+06	2.020611e+07	1.446777e+08	1.496376e+07	
2016-12-28	3.200140e+06	2.757122e+06	1.922440e+07	1.405430e+08	1.548017e+07	
2016-12-29	3.044633e+06	2.464330e+06	1.852005e+07	1.498275e+08	1.479296e+07	
2016-12-30	2.447544e+06	2.664459e+06	1.767950e+07	1.247001e+08	1.141816e+07	
2016-12-31	2.599010e+06	2.334855e+06	1.661956e+07	1.229791e+08	1.090994e+07	

550 rows × 10 columns

In [116…
```
timeSeriesFull_differenced_df = timeSeriesFull_df.copy()

non_stationary_cols = [
    "ModelNonSpiderLangCommons",
    "ModelNonSpiderLangDe",
    "ModelNonSpiderLangEn",
    "ModelNonSpiderLangJa",
    "ModelNonSpiderLangZh"
]

periods=1
timeSeriesFull_differenced_df[non_stationary_cols] = timeSeriesFull_differenced_df[non_stationary_cols].diff(pe
timeSeriesFull_differenced_df = timeSeriesFull_differenced_df.dropna()
performADFTestForAllSegments(timeSeriesFull_differenced_df)
```

```
Model                        P-Value  Status
---------------------------  ---------  ----------
ModelSpider                  1.2963e-05  Stationary
ModelNonSpiderLangCommons    5.635e-23   Stationary
ModelNonSpiderLangDe         1.8144e-10  Stationary
ModelNonSpiderLangEn         1.4166e-12  Stationary
ModelNonSpiderLangEs         0.036535    Stationary
ModelNonSpiderLangFr         0.039369    Stationary
ModelNonSpiderLangJa         7.7179e-20  Stationary
ModelNonSpiderLangRu         0.0019986   Stationary
ModelNonSpiderLangWww        1.4353e-08  Stationary
ModelNonSpiderLangZh         1.6898e-11  Stationary
```

- So with period=1, we are able to make the non stationary series as stationary. Thus for these sereis, we will use d=1

Now that all series are stationary, lets plot ACF PACF again

Rule (Box–Jenkins) Both ACF and PACF used for identifying p and q must be computed on a stationary series.

In [ ]:
```
print("Plotting ACF for ALL segments")
plotACFForAllSegments(timeSeriesFull_differenced_df)

print("Plotting PACF for ALL segments")
plotPACFForAllSegments(timeSeriesFull_differenced_df)
```

- **How ARIMA parameters can be decided based on ACF and PACF results**

Rule for **d**

- `d = 0` if original series is ADF-stationary
- `d = 1` if original series is non-stationary and
  ✓ ADF on first-differenced series is stationary (your confirmation)

✓ No series uses `d > 1`

Gold-standard Box–Jenkins rules

Pattern	ACF	PACF	Interpretation
AR(p)	Tails off	Cuts off at p	AR-dominated
MA(q)	Cuts off at q	Tails off	MA-dominated
ARMA(p,q)	Both tail off	Both tail off	Mixed

Rule for p (from PACF)

- Clear cutoff at lag k → p = k
- Strong lag-1 only → p = 1
- No clear low-lag cutoff → p = 0

Rule for `q`

> **Choose `q` ONLY when the ACF shows a clear cutoff.**

If the ACF:

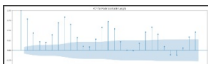
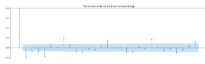
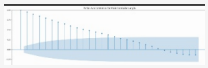
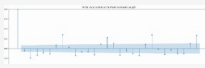
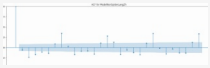
- Gradually decays → **q = 0**
- Drops to ~0 after lag `k` → **q = k**
- Has no clear cutoff → **prefer q = 0**

We only add `q = 1` **conservatively** when:

- Lag-1 is strongly significant
- Higher lags are not
- And PACF does not show a clean AR cutoff

Final ARIMA Identification Table (ACF + PACF Based)

Segment	d	PACFPlot	PACF Observation	ACF Plot	ACF Observation	ARIMA Search Space (PACF → p, d, ACF → q)
			Significant lags		Smooth, monotonic	p ∈ {1,2,3,4}, d =

ModelSpider	0		Significant lags 1–4, clear taper		Smooth, monotonic decay	p ∈ {1,2,3,4}, d = 0, q ∈ {0}
NonSpiderLangCommons	1		Strong negative lag-1 only		Clear cutoff at lag-1	p ∈ {0,1,2}, d = 1, q ∈ {0,1}
NonSpiderLangDe	1		No clear low-lag cutoff		No cutoff; isolated higher-lag spikes	p ∈ {0,1}, d = 1, q ∈ {0}
NonSpiderLangEn	1		No clear significant lags		No cutoff; weak tailing	p ∈ {0,1}, d = 1, q ∈ {0,1}
NonSpiderLangEs	0		Isolated significant lags. Later lags (≈ 7, 14, 21, 28) are large, repeating spikes, roughly equally spaced. They are periodic / cyclical dependence leaking into PACF. What a true high-order AR(p) PACF looks like - PACF is significant at lags 1…p, then drops to ~0 and stays there and No recurring structure. This was observed in ModelSpider above		Oscillatory, no cutoff	p ∈ {0,1,2}, d = 0, q ∈ {0}
NonSpiderLangFr	0		Gradual taper, no cutoff		Gradual decay	p ∈ {0,1,2}, d = 0, q ∈ {0}
NonSpiderLangJa	1		Strong negative lag-1 only		Clear cutoff at lag-1	p ∈ {0,1,2}, d = 1, q ∈ {0,1}
NonSpiderLangRu	0		Significant lags 1–2		Smooth decay	p ∈ {1,2,3}, d = 0, q ∈ {0}
NonSpiderLangWww	0		Significant lags 1–2		Fast decay, no cutoff (there is tailing, but no cutoff)	p ∈ {1,2,3}, d = 0, q ∈ {0}
NonSpiderLangZh	1		No clear low-lag structure		No cutoff; alternating small spikes	p ∈ {0,1}, d = 1, q ∈ {0}

Summary First differencing was applied to non-stationary series until stationarity was confirmed via ADF testing. Autoregressive order was selected based on PACF cutoff behavior, while moving-average order was informed by ACF tail behavior, resulting in low-order ARIMA specifications for all segments.

ACF and PACF plots are used for preliminary identification and to constrain the ARIMA parameter space. Final model selection will be performed using information criteria (AIC/BIC), with preference given to parsimonious models within a small AIC tolerance.

Setting up Train Test sets for model performance evaluation

let's create a function to estimate the performance of different models

In [116…

```
from sklearn.metrics import (
    mean_squared_error as mse,
    mean_absolute_error as mae,
    mean_absolute_percentage_error as mape
)

# Creating a function to print values of all these metrics.
```

```
def performance(actual, predicted,verbose=True):

    mape_val = round(mape(actual, predicted), 3)
    rmse_val = round(mse(actual, predicted)**0.5, 3)
    mae_val = round(mae(actual, predicted), 3)

    if(verbose):
        print('MAE :',mae_val)
        print('RMSE :', rmse_val)
        print('MAPE:', mape_val)
    return mape_val,rmse_val, mae_val
```

Train Test Split

Out of the 550 days of data, we decide to train on 490 days of data, and use the last 60 days for test data.

We can simply use **slicing** to obtain these sets.

In [ ]:

In [116…
```
test_size=int(len(timeSeriesFull_df)*0.2)  #20% test
train_max_date = timeSeriesFull_df.index[-test_size]
train_max_date
```

Out[116… `Timestamp('2016-09-13 00:00:00')`

In [116…
```
train_x = timeSeriesFull_df.loc[timeSeriesFull_df.index < train_max_date].copy()
test_x = timeSeriesFull_df.loc[timeSeriesFull_df.index >=train_max_date].copy()

test_x.head()
```

Out[116…

	ModelSpider	ModelNonSpiderLangCommons	ModelNonSpiderLangDe	ModelNonSpiderLangEn	ModelNonSpiderLangEs	Mod
2016-09-13	2.508016e+06	2.408625e+06	1.505150e+07	1.173980e+08	2.393244e+07	
2016-09-14	2.288274e+06	2.700041e+06	1.473732e+07	1.167235e+08	2.086071e+07	
2016-09-15	2.302830e+06	2.977628e+06	1.458530e+07	1.199807e+08	1.831930e+07	
2016-09-16	2.423128e+06	3.361518e+06	1.465944e+07	1.017883e+08	1.552162e+07	
2016-09-17	2.451582e+06	2.429943e+06	1.451717e+07	1.079524e+08	1.410321e+07	

In [116…
```
len(test_x)
```

Out[116… `110`

Fitting ARIMA Model ----------

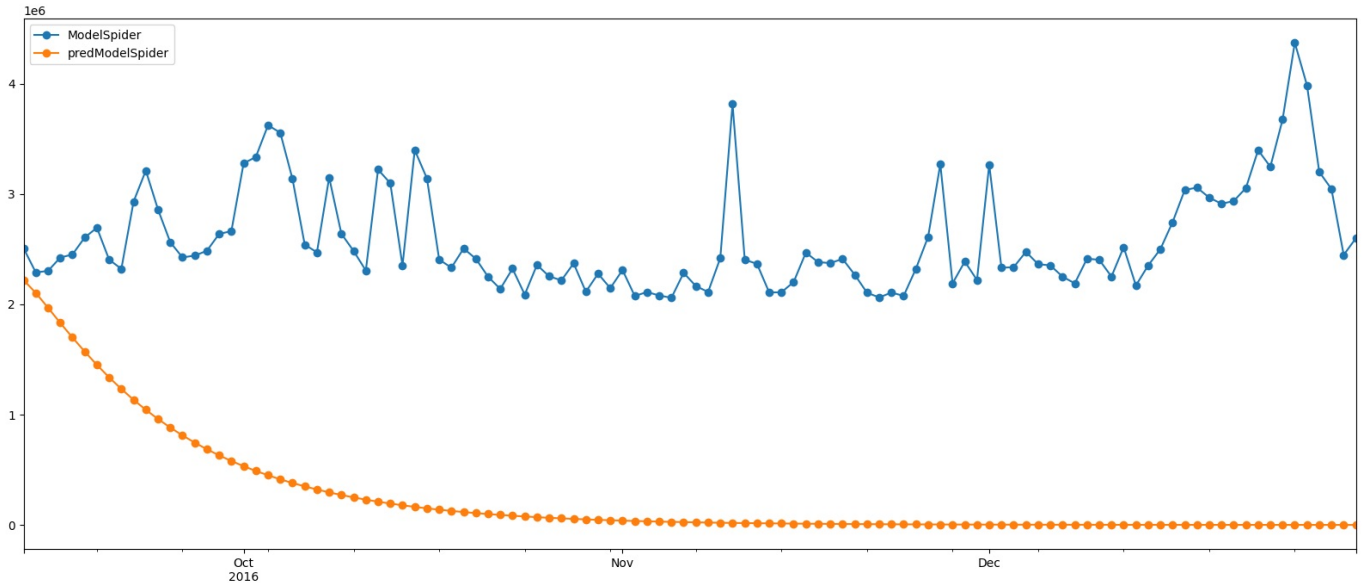
In [116…
```
from statsmodels.tsa.statespace.sarimax import SARIMAX

segmentName = modelNames[0]
model = SARIMAX(train_x[segmentName] ,order=(4,0,0),enforce_stationarity=False,
                                    enforce_invertibility=False)
model = model.fit(disp=False, maxiter=200)
```

Fit on One series first and do a Performance Review

In [116…
```
predColumnName = 'pred'+segmentName

test_x[predColumnName] = model.forecast(steps=test_size)
test_x[[segmentName,predColumnName]].plot(style='-o')
performance(test_x[segmentName], test_x[predColumnName])
```

```
MAE : 2321531.651
RMSE : 2419872.336
MAPE: 0.896
```

Out[116… `(0.896, 2419872.336, 2321531.651)`



- ARIMA is giving EXTREMELY poor results. Error (MAPE) is 89%
- Our data has clear seasonality.
- So should we spend time on ARIMA? To establish a baseline though, we will do a gridsearch using ARIMA for all models, and then try to improve on it.

Define the ARIMA search space

This follows from our ACF PACF analysis.

In [116…

```
ARIMA_SEARCH_SPACE = {
    "ModelSpider": {
        "p": [1, 2, 3, 4],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangCommons": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangDe": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangEn": {
        "p": [0, 1],
        "d": [1],
        "q": [0, 1],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangEs": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
```

```
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangFr": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangJa": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangRu": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangWww": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    },
    "ModelNonSpiderLangZh": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0]
    }
}

ARIMA_SEARCH_SPACE_old = {
    "ModelSpider": {
        "p": [1, 2, 3, 4],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangCommons": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangDe": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangEn": {
        "p": [0, 1],
```

```
        "d": [1],
        "q": [0, 1],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangEs": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangFr": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangJa": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangRu": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangWww": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    },
    "ModelNonSpiderLangZh": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0],
        "D": [0],
        "Q": [0],
        "s": [0],
        "numExog": 0
    }
}
```

Grid-search function (MAPE + AIC + BIC)

- We want to write ONE function that can do ARIMA, SARIMA and SARIMAX
- It takes the order ranges and ONE series
- Returns best result
- Saves the find in our modelDict object (MLOps to keep track of all experiments.)

In [116…

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
import warnings
import numpy as np
import pandas as pd
```

```python
from itertools import product
import time

def gridSearchSARIMAX(y, space, exog_df=pd.DataFrame(),
                      train_ratio=0.8, metric='mape', verbose=True):
    """
    Grid search for optimal SARIMAX parameters on a single series.

    Parameters:
    -----------
    y : array-like - Time series data
    p_range : list/range - AR orders to try (e.g., [0, 1, 2])
    d_range : list/range - Differencing orders (e.g., [0, 1])
    q_range : list/range - MA orders (e.g., [0, 1, 2])
    P_range : list/range - Seasonal AR orders
    D_range : list/range - Seasonal differencing orders
    Q_range : list/range - Seasonal MA orders
    s : int - Seasonal period (default 7 for weekly)
    exogSeries: Exogeneous regressor series.
    train_ratio : float - Train/test split ratio
    metric : str - 'mape', 'aic', 'bic', or 'rmse'
    verbose : bool - Print progress

    Returns:
    --------
    results_df : DataFrame with all combinations and scores
    best_params : dict with best parameters
    """



    # Train/test split
    n = len(y)
    train_size = int(n * train_ratio)
    y_train = y[:train_size]
    y_test = y[train_size:]
    if(exog_df is None or len(exog_df.columns)==0):
        exog_train=None
        exog_test=None
        exog_cols=''
    else:
        exog_train = exog_df[:train_size]
        exog_test = exog_df[train_size:]
        exog_cols= exog_train.columns

    # Generate all combinations
    import math;
    total_combinations = math.prod(len(v) for v in space.values())

    #Always printed text, even with Verbose=False
    print(f"Testing {total_combinations} combinations...")

    if verbose:
        # print(f"Testing {total_combinations} combinations...")
        print(f"Train size: {train_size}, Test size: {len(y_test)}")

    results = []
    tested = 0

    for p in space["p"]:
        for d in space["d"]:
            for q in space["q"]:
                for P in space["P"]:
                    for D in space["D"]:
                        for Q in space["Q"]:
                            for s in space["s"]:
                                tested += 1
                                try:
                                    with warnings.catch_warnings():
                                        warnings.simplefilter("ignore")

                                        # Fit model
                                        model = SARIMAX(endog=y_train,
                                                        order=(p, d, q),
                                                        seasonal_order=(P, D, Q, s),
                                                        exog=exog_train,
                                                        enforce_stationarity=False,
                                                        enforce_invertibility=False)

                                        fitted = model.fit(disp=False, maxiter=200)

                                        # Forecast
                                        forecast = fitted.forecast(steps=len(y_test), exog=exog_test)
```

```
                                    # Calculate metrics
                                    mape,rmse, mae = performance(y_test, forecast, verbose=False)
                                    results.append({
                                        'order': f'{p,d,q}',
                                        'seasonal_order':  f'{(P, D, Q, s)}',
                                        'exog': exog_cols,
                                        'aic': fitted.aic,
                                        'bic': fitted.bic,
                                        'mape': mape,
                                        'rmse': rmse,
                                        'mae': mae,
                                        'converged': True
                                    })

                                    if verbose and tested % 20 == 0:
                                        print(f"  Progress: {tested}/{total_combinations} ({100*tested/tota

                                except Exception as e:
                                    results.append({
                                        'order': f'({p,d,q})',
                                        'seasonal_order':  f'({(P, D, Q, s)})',
                                        'aic': np.nan, 'bic': np.nan,
                                        'mape': np.nan, 'rmse': np.nan, 'mae': np.nan,
                                        'converged': False,
                                        'error': str(e)
                                    })

    # Create results DataFrame
    results_df = pd.DataFrame(results)

    # Find best parameters
    if metric == 'mape':
        best_idx = results_df['mape'].idxmin()
    elif metric == 'aic':
        best_idx = results_df['aic'].idxmin()
    elif metric == 'bic':
        best_idx = results_df['bic'].idxmin()
    elif metric == 'rmse':
        best_idx = results_df['rmse'].idxmin()
    else:
        best_idx = results_df['mape'].idxmin()

    best_row = results_df.loc[best_idx]
    best_params = {
        'order': best_row['order'],
        'seasonal_order': best_row['seasonal_order'],
        'exog': exog_cols,
        'mape': best_row['mape'],
        'aic': best_row['aic'],
        'bic': best_row['bic'],
        'rmse': best_row['rmse']
    }
    #Always printed text, even with Verbose=False
    print("MAPE:",best_row['mape'])


    if verbose:
        print(f"\n✓ Best by {metric.upper()}:")
        print(f"   Order: {best_params['order']}")
        print(f"   Seasonal: {best_params['seasonal_order']}")
        print(f"   MAPE: {best_params['mape']:.2f}%")
        print(f"   AIC: {best_params['aic']:.2f}")

    return results_df, best_params
```

Lets try our grid search function on same series what we used in direct ARIMA to see if it works correctly

In [117…

```
#Lets try same series what we used in direct ARIMA
segmentName = modelNames[0]
# ARIMA_SEARCH_SPACE = {
#     "ModelSpider": {
#         "p": [1, 2, 3, 4],
#         "d": [0],
#         "q": [0]
#     }
r,b = gridSearchSARIMAX(timeSeriesFull_df[segmentName] ,
                        ARIMA_SEARCH_SPACE[segmentName])
r
```

```
Testing 4 combinations...
Train size: 440, Test size: 110
MAPE: 0.738

✅ Best by MAPE:
   Order: (1, 0, 0)
   Seasonal: (0, 0, 0, 0)
   MAPE: 0.74%
   AIC: 13080.64
```

Out[117…

	order	seasonal_order	exog	aic	bic	mape	rmse	mae	converged
0	(1, 0, 0)	(0, 0, 0, 0)		13080.635490	13088.804489	0.738	2063068.249	1922742.330	True
1	(2, 0, 0)	(0, 0, 0, 0)		13016.211772	13028.458428	0.859	2333037.461	2225966.750	True
2	(3, 0, 0)	(0, 0, 0, 0)		12989.504100	13005.823833	0.858	2331361.613	2224083.772	True
3	(4, 0, 0)	(0, 0, 0, 0)		12954.296060	12974.684272	0.896	2419872.336	2321531.651	True

- We tried (4,0,0) manually and using gridsearch, got 89%, so grid search code is correct.

gridSearchARIMAAllSegments()

In [117…

```python
def gridSearchSARIMAXAllSegments(timeSeriesCombined_df, modelNames, modelDict, searchSpace, exog_all_segments_df
                          train_ratio=0.8,metric='mape', searchTitle="GridSearch", comments="GridSearch",verbose
    """
    Run grid search on all segments.
    exogSeries_df- pass in a dataframe that has column named segment_exog_[i] for the segment for which we want
    We want to support unlimilted exogeneus columns.  So for that we have the [i]
    * If a segment has two exogeneous columns, then we will have <segment_name>_exog_1 and <segment_name>_exog_2
    * If a segment has one exogeneous columns, then we will have <segment_name>_exog_1
    * If a segment has no exogeneous columns, then we will have no column that has <segment_name>_exog*
    * Number of columns will come from searchSpace grid.



    Returns:
    --------
    all_results : dict with results for each segment
    best_orders : dict with best parameters for each segment
    """

    all_results = {}
    best_orders = {}
    exog_segment_df=pd.DataFrame()
    exog_segment_cols=[]

    for i, segmentName in enumerate(modelNames):
        print(f"\n{'='*60}")
        print(f"GRID SEARCH: {segmentName}")
        print(f"{'='*60}")

        if(exog_all_segments_df is not None):
            # find all exogenous columns for this segment
            exog_segment_cols = [col for col in exog_all_segments_df.columns if col.startswith(f"{segmentName}_e
            exog_segment_df = exog_all_segments_df[exog_segment_cols]

        else:
            exog_segment_df=None

        # Run grid search
        start_time = time.time()
        results_df, best_params = gridSearchSARIMAX(timeSeriesCombined_df[segmentName] ,
                                searchSpace[segmentName],exog_df=exog_segment_df,
                                train_ratio=train_ratio, metric=metric, verbose=verbose)

        elapsed = time.time() - start_time
        print(f"  ⏱  Time: {elapsed:.1f} seconds")

        # Store results
        all_results[segmentName] = results_df
        best_orders[segmentName] = best_params

        # Log best model to modelDict
        addModelAttempt(
            modelDict, segmentName,
            searchTitle,
            (best_params['order'], best_params['seasonal_order'],exog_segment_cols),
            best_params['mape'],
            comments=f"{comments} | AIC:{best_params['aic']:.1f}",
            verbose=verbose
```

```
        )

    return all_results, best_orders
```

In [ ]:
```
printModelDictSummaryDETAILED(showAttempts=True)
```

Model Dictionary Summary

Model Name	Data Rows	# Attempts	Best Model	Best Params	Best MAPE

| ModelSpider | 34913 | 0 | None | - | - | ModelNonSpiderLangCommons | 8005 | 0 | None | - | - | ModelNonSpiderLangDe | 13960 | 0 | None | - | - | ModelNonSpiderLangEn | 19177 | 0 | None | - | - | ModelNonSpiderLangEs | 10532 | 0 | None | - | - | ModelNonSpiderLangFr | 13302 | 0 | None | - | - | ModelNonSpiderLangJa | 15424 | 0 | None | - | - | ModelNonSpiderLangRu | 11280 | 0 | None | - | - | ModelNonSpiderLangWww | 5551 | 0 | None | - | - | ModelNonSpiderLangZh | 12919 | 0 | None | - | - | **TOTAL** | **145063** | **0** | - | - | - |

Validation

Metric	Count
Total rows across models	145063
Original dataframe rows	145063
Difference	0

✅ **MATCH:** Total rows equals original dataframe!

Best MAPE Ranking

| Rank | Segment Name | Best MAPE |Source |------|-----------|----------|------| | - | No models fitted yet | - |

In [117…
```
r,b=gridSearchSARIMAXAllSegments(timeSeriesCombined_df=timeSeriesFull_df,
                                 modelNames=modelNames,
                                 modelDict=modelDict,
                                 searchSpace=ARIMA_SEARCH_SPACE,
                                 exog_all_segments_df=None,
                                 train_ratio=0.8,
                                 metric='mape',
                                 searchTitle="ARIMA GridSearch",
                                 comments="Used ARIMA_SEARCH_SPACE",
                                 verbose=False)
```

```
============================================================
GRID SEARCH: ModelSpider
============================================================
Testing 4 combinations...
MAPE: 0.738
  □  Time: 0.2 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangCommons
============================================================
Testing 6 combinations...
MAPE: 0.147
  □  Time: 0.2 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangDe
============================================================
Testing 2 combinations...
MAPE: 0.074
  □  Time: 0.0 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangEn
============================================================
Testing 4 combinations...
MAPE: 0.069
  □  Time: 0.1 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangEs
============================================================
Testing 3 combinations...
MAPE: 0.181
  □  Time: 0.1 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangFr
============================================================
Testing 3 combinations...
MAPE: 0.117
  □  Time: 0.1 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangJa
============================================================
Testing 6 combinations...
MAPE: 0.088
  □  Time: 0.4 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangRu
============================================================
Testing 3 combinations...
MAPE: 0.321
  □  Time: 0.1 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangWww
============================================================
Testing 3 combinations...
MAPE: 0.786
  □  Time: 0.1 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangZh
============================================================
Testing 2 combinations...
MAPE: 0.065
  □  Time: 0.0 seconds
```

In [ ]:
```
printModelDictSummaryDETAILED(showAttempts=True)
```

Model Dictionary Summary

Model Name	Data Rows	# Attempts	Best Model	Best Params	Best MAPE

| ModelSpider | 34913 | 1 | ARIMA GridSearch | ('(1, 0, 0)', '(0, 0, 0, 0)', []) | 0.7380% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 73.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangCommons | 8005 | 1 | ARIMA GridSearch | ('(0, 1, 0)', '(0, 0, 0, 0)', []) | 0.1470% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 14.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangDe | 13960 | 1 | ARIMA GridSearch | ('(0, 1, 0)', '(0, 0, 0, 0)', []) | 0.0740% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 7.4% | Used ARIMA_SEARCH_SPACE | AIC:... | |

ModelNonSpiderLangEn | 19177 | 1 | ARIMA GridSearch | ('(0, 1, 0)', '(0, 0, 0, 0)', []) | 0.0690% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.9% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangEs | 10532 | 1 | ARIMA GridSearch | ('(1, 0, 0)', '(0, 0, 0, 0)', []) | 0.1810% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 18.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangFr | 13302 | 1 | ARIMA GridSearch | ('(2, 0, 0)', '(0, 0, 0, 0)', []) | 0.1170% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(2, 0, 0)', '(0, 0, 0, 0)', []) | | | | 11.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangJa | 15424 | 1 | ARIMA GridSearch | ('(0, 1, 1)', '(0, 0, 0, 0)', []) | 0.0880% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(0, 1, 1)', '(0, 0, 0, 0)', []) | | | | 8.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangRu | 11280 | 1 | ARIMA GridSearch | ('(1, 0, 0)', '(0, 0, 0, 0)', []) | 0.3210% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 32.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangWww | 5551 | 1 | ARIMA GridSearch | ('(1, 0, 0)', '(0, 0, 0, 0)', []) | 0.7860% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 78.6% | Used ARIMA_SEARCH_SPACE | AIC:... | | ModelNonSpiderLangZh | 12919 | 1 | ARIMA GridSearch | ('(1, 1, 0)', '(0, 0, 0, 0)', []) | 0.0650% | ↳ Attempts: | | | | | | | 1. ARIMA GridSearch('(1, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.5% | Used ARIMA_SEARCH_SPACE | AIC:... | | **TOTAL** | **145063** | **10** | - | - | - |

Validation

Metric	Count
Total rows across models	145063
Original dataframe rows	145063
Difference	0

✅ **MATCH:** Total rows equals original dataframe!

Best MAPE Ranking

| Rank | Segment Name | Best MAPE |Source |------|------------|----------|------| | 1 | ModelNonSpiderLangZh | 6.5% |ARIMA GridSearch('(1, 1, 0)', '(0, 0, 0, 0)', []) | 2 | ModelNonSpiderLangEn | 6.9% |ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | 3 | ModelNonSpiderLangDe | 7.4% |ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | 4 | ModelNonSpiderLangJa | 8.8% |ARIMA GridSearch('(0, 1, 1)', '(0, 0, 0, 0)', []) | 5 | ModelNonSpiderLangFr | 11.7% |ARIMA GridSearch('(2, 0, 0)', '(0, 0, 0, 0)', []) | 6 | ModelNonSpiderLangCommons | 14.7% |ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | 7 | ModelNonSpiderLangEs | 18.1% |ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | 8 | ModelNonSpiderLangRu | 32.1% |ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | 9 | ModelSpider | 73.8% |ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | 10 | ModelNonSpiderLangWww | 78.6% |ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', [])

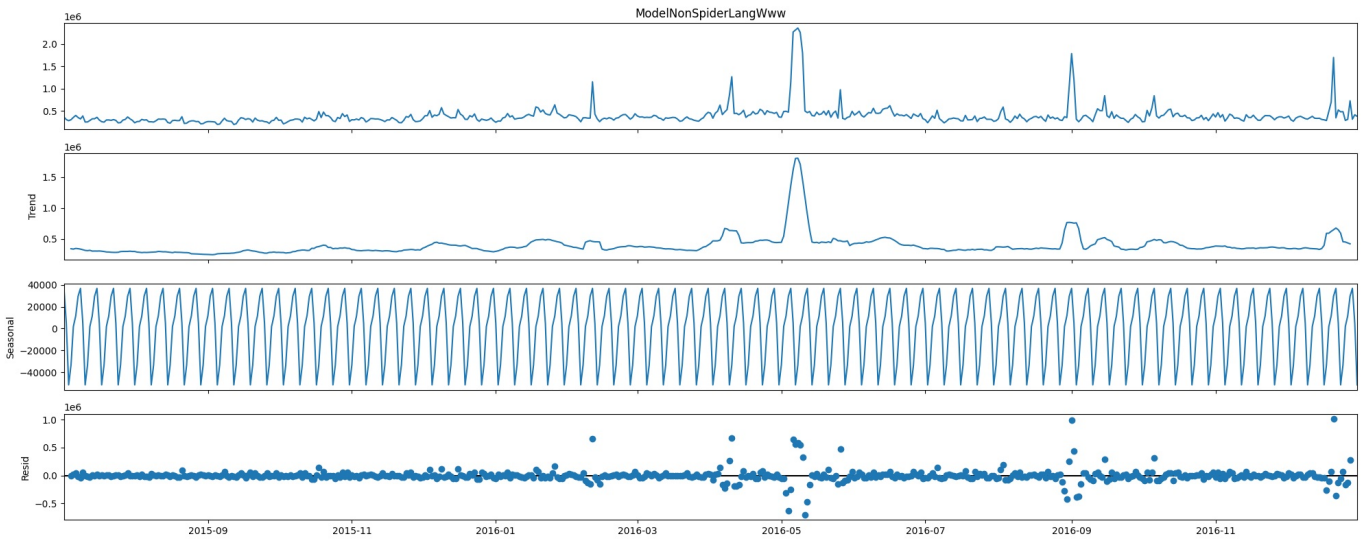
Checking for Seasonality

- ARIMA did not utilize seasonality.
- But if its there, it can be taken into consideration as it will help us improve our model
- Lets decompose our series to see trend , seasonality

Plot Seasonal decomposition of one sample

In [117…

```
import statsmodels.api as sm


decompose_model = sm.tsa.seasonal_decompose(timeSeriesFull_differenced_df[modelNames[8]], model='additive')
decompose_model.plot();
```



- We are having STRONG seasonality !!
- But ADF test indicated stationary series. Is there a contradiction here?
- The key point is:

ADF stationarity ≠ “no trend or no seasonality”

1 What the ADF test actually checks (very important)

ADF tests **only one thing**:

> **Is there a unit root (stochastic trend)?**

- **p < 0.05 ⇒ no unit root**
- That means the series is **mean-reverting**
- It does **NOT** mean:
  - no trend
  - no seasonality
  - no spikes
  - no structure

A series **can be stationary AND seasonal**

A series **can be stationary AND have level shifts / spikes**

Here is why a series can be "stationary" according to the ADF test while still having visible patterns:

---

1. Stationary AND Seasonal

A series is stationary if its mean, variance, and autocorrelation structure do not change over time.

- **The Logic:** If a series has a seasonal pattern (e.g., electricity demand peaks every 24 hours) but that pattern is **constant and predictable**, the series is often considered "seasonally stationary."
- **Mean Reversion:** Even with seasonality, the series constantly pulls back toward a long-term average or a repeating cycle. It doesn't "drift away" forever like a random walk would.

2. Stationary AND Level Shifts / Spikes

The ADF test checks if a shock to the system lasts forever.

- **The "Shock" Test:** In a non-stationary series (like a Random Walk), if the price jumps by 10, $that$10 increase is permanent. In a stationary series, if there is a massive "spike," the system eventually pulls the value back to its mean.
- **Structure:** You can have a series that is mostly flat but has occasional, violent spikes. As long as those spikes are short-lived and the series returns to its average, the ADF test will still call it stationary.

3. The Difference: Stochastic vs. Deterministic Trends

This is the most important distinction for understanding that image:

- **Stochastic Trend (Unit Root):** The "drift" is random. The series wanders aimlessly and never returns to a baseline. **ADF catches this.**
- **Deterministic Trend:** The series follows a predictable path (like a straight line up or a sine wave). While it's not "strictly" stationary, it is "trend-stationary." You can subtract the trend/seasonality, and you are left with a perfectly stationary series.

ARIMA versus SARIMA

➡ **SARIMA can give better** results for this data.

Determining Seasonality length

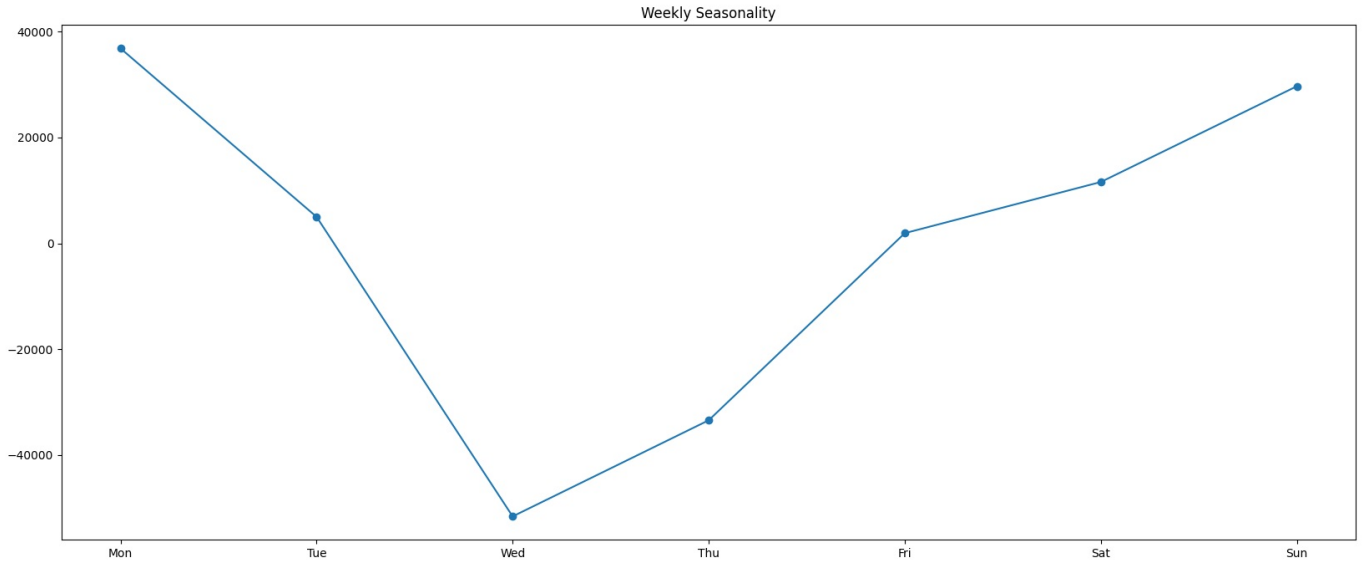
- Before we try fit a SARIMA model, lets see if we can "see" the seasonality in data
- The Seasonal decompose plot shows a very very compressed view, so while its clear there is a seasonality, we cannot really "read it off" from the plot just yet
- The decompose_model provides access to seasonal component using `decompose_model.seasonal`. But this is just a repeated seasonal signal, we still need to infer/caculate the seasonality length
- Lets look at our series more closely to "see" the seasonality trend

In [117…

```
#Extract the time series called seasonal from the decompose model.
seasonal = decompose_model.seasonal

#Lets plot just one week
one_week = seasonal.iloc[:7]
```

```
one_week.index = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']
one_week.plot(marker='o', title='Weekly Seasonality')
```

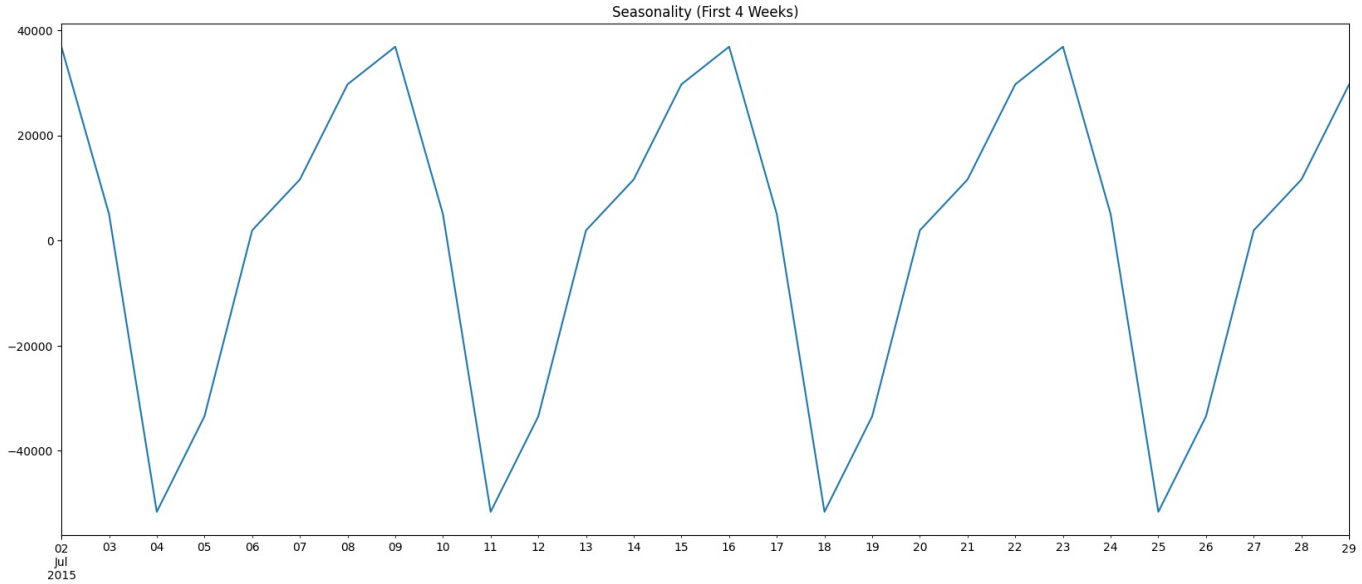
Out[117… <Axes: title={'center': 'Weekly Seasonality'}>



- **Interesting result.** We can "see" there is some movement.
- But does it repeat.
- Lets plot a broader window, say 28 days.

In [117…
```
seasonal.iloc[:28].plot(title='Seasonality (First 4 Weeks)')
```

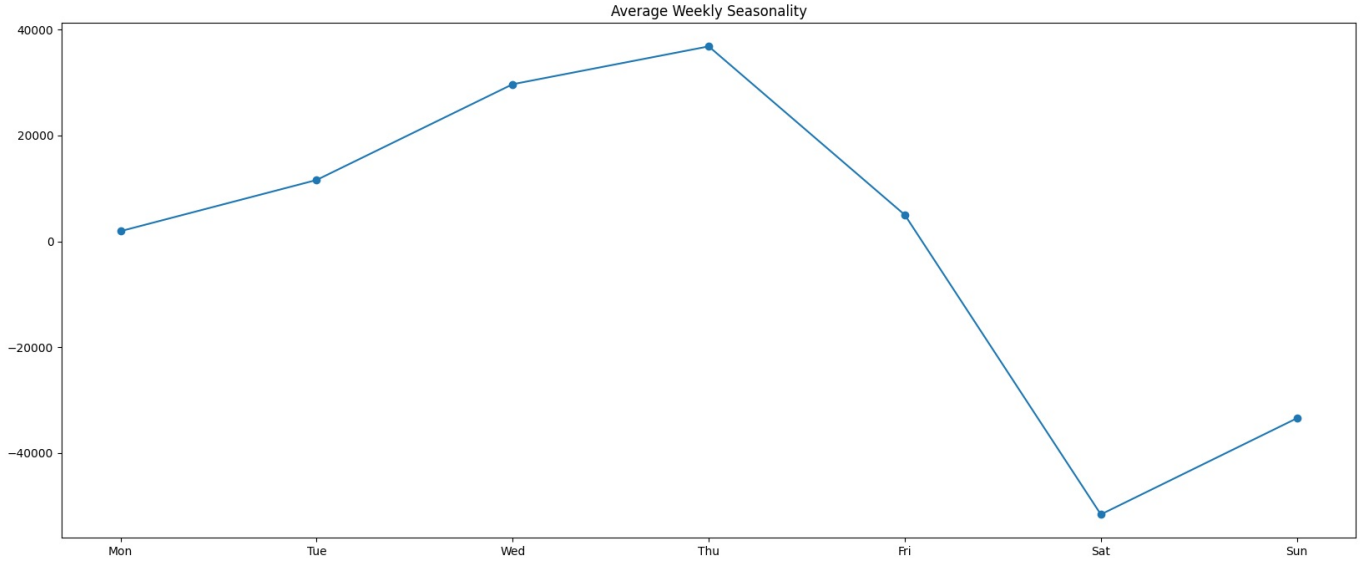
Out[117… <Axes: title={'center': 'Seasonality (First 4 Weeks)'}>



- In 28 days too, we can see a trend that repeats 4 times. **This is strong hint that our seasonality is 7 days.**
- This is visual proof, but not mathematical. We are not concluding it just yet.
- But we do have solid ground to say that the cycle could be 7 days (Wikipedia pages are things people will read every week)
- A mathematical proof will be if the pattern is same across all weeks. What if we saw all 7 day weeks and averaged out the values.
- That is what we do next. Lets average across all cycles to remove any noise

In [117…
```
weekly_seasonality = (
    seasonal
    .groupby(seasonal.index.dayofweek)
    .mean()
)

weekly_seasonality.index = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']

weekly_seasonality.plot(marker='o', title='Average Weekly Seasonality')
```

Out[117… <Axes: title={'center': 'Average Weekly Seasonality'}>



- Clear weekly seasonality exists ✓ The curve is not flat and shows a strong, consistent pattern by day of week.
- The identical weekly seasonal profile across different STL periods indicates that weekly seasonality is the dominant and stable component of the time series.

Intepreting this graph

- This graph is not raw data.
- It is:
  - The average effect of each day of the week, after removing trend and noise
- How it's computed (conceptually):
  - STL extracts the seasonal component
  - Every Monday seasonal value is grouped together
  - Same for Tuesday, Wednesday, …
  - We take the mean for each weekday
- So the plot answers this question:
  - "If everything else is equal, how does this specific day of the week affect the value?"

Mid-week peak (Wed–Thu)

Observations

- **Thursday is the strongest positive day**
- Wednesday is a close second
- Tuesday is moderately positive
- Monday is barely above baseline

Interpretation

- Demand / activity **builds through the workweek**
- Peaks **before the weekend**
- Suggests:
  - Business-driven behavior
  - Planning / decision-making mid-week

✓ **Modeling implication**

- SARIMA with weekly seasonality is appropriate
- Exogenous regressors like "weekday dummies" would help

```
pd.get_dummies(ts.index.dayofweek, drop_first=True)
```

This will:

- Explicitly model the weekend crash
- Reduce seasonal burden on SARIMA terms

Magnitude is large → seasonality dominates noise

The seasonal effect ranges roughly:

- **+35k at peak**
- **−50k at trough**

That's a **~85k swing weekly**.

➡ Seasonality is not subtle — it's a **primary driver**.

Business-level interpretation

- Users are **work-week oriented**
- Thursday is the most valuable day
- Saturday is nearly "dead time"
- Forecast errors will spike on weekends if not modeled properly

Plot Seasonal Decomposition for all Segments

One question - should we pass period or not.

Short answer (rule)

✅ **You SHOULD pass `period`**
❌ Not passing it does NOT let the model "figure it out"

In fact:

> **If you don't pass `period`, `seasonal_decompose` is either guessing poorly or will fail.**

Why this is the case

`seasonal_decompose` in **statsmodels** is **not a learning algorithm**.
It does **not infer seasonality** from the data. It does **NOT** "try multiple periods and choose the best one".

Instead, it:

- **assumes a fixed season length**
- then mechanically splits the series into:
  - trend (via moving average)
  - seasonal (average of aligned positions)
  - residual

So when you pass `period`, you're not *biasing* it — you're giving it **required structural information**.

Analogy (important intuition)

Think of `period` like this:

> Passing `period=7` is like saying
> *"This is daily data, check if there's a weekly pattern."*

Not:

> *"Force weekly seasonality no matter what."*

If there is **no weekly seasonality**, the seasonal component will be close to zero.

Best practice - Detect candidate periods outside decomposition

Use **domain knowledge first**:

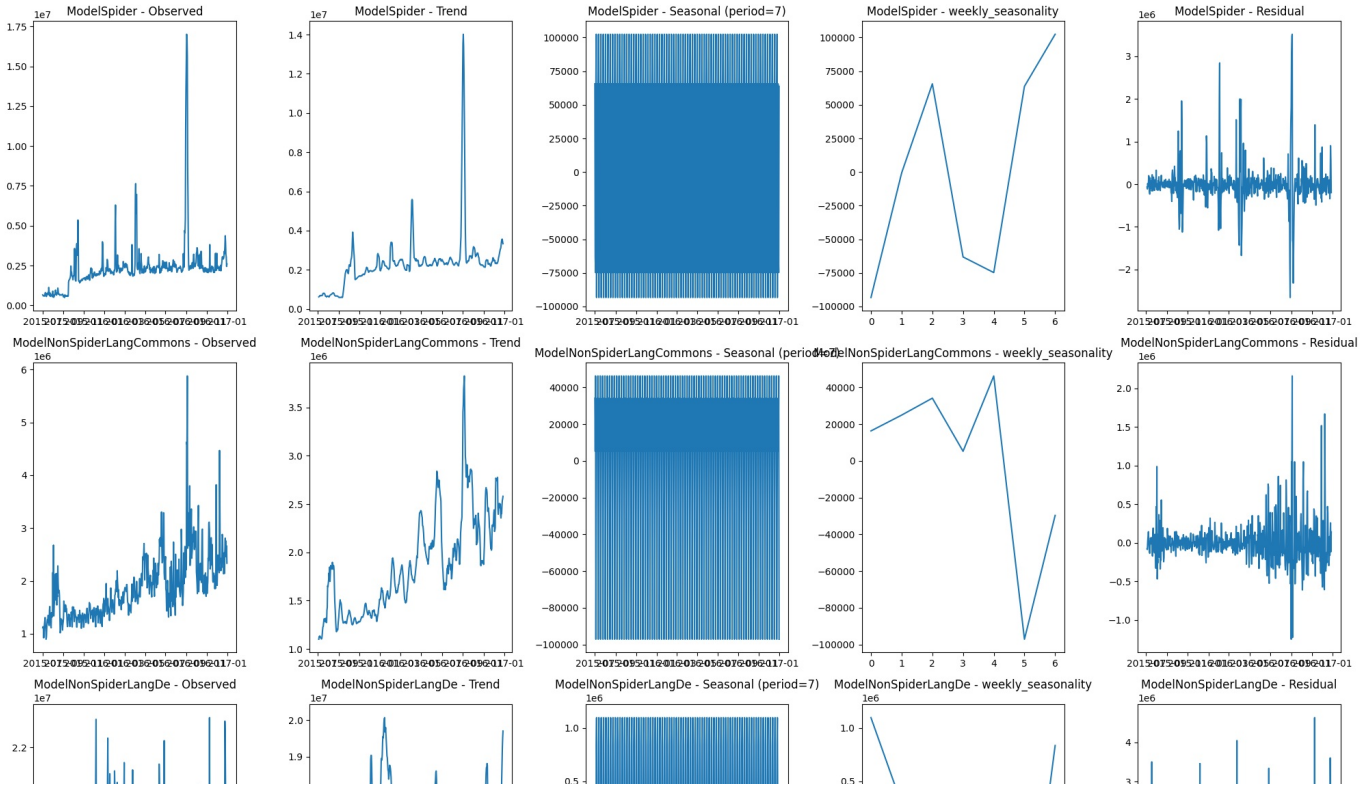
- Daily data → try `7`
- Monthly data → try `12`
- Hourly data → try `24`, `168`

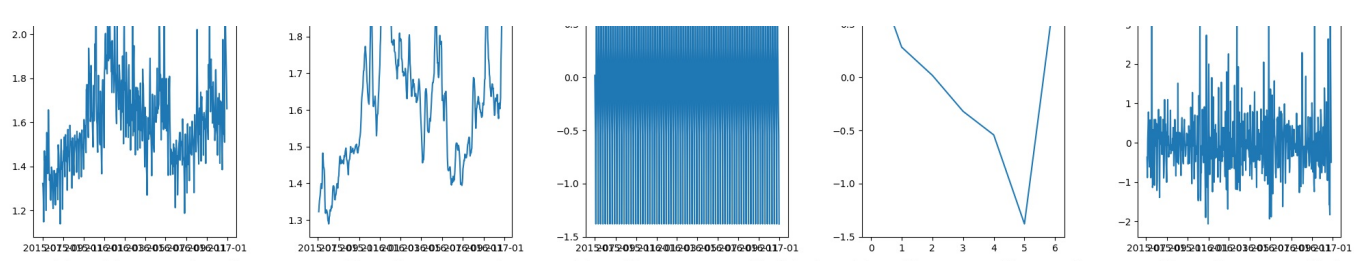
**We did this.

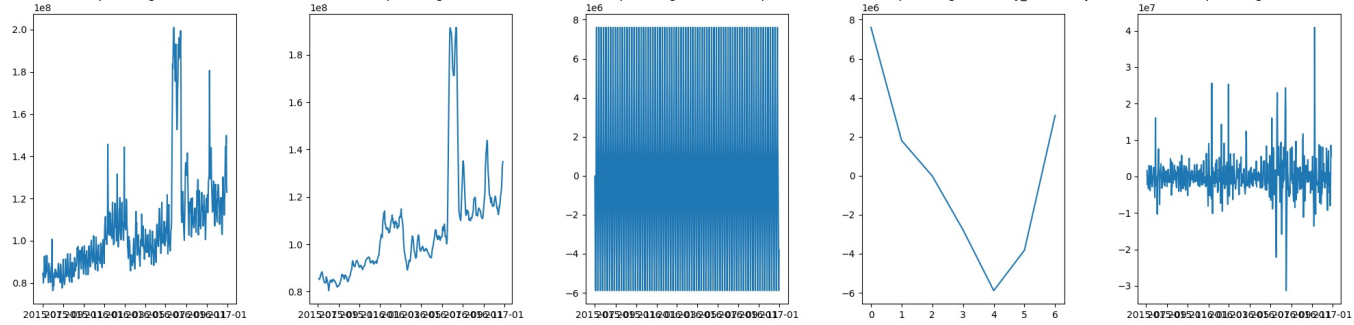
- From ACF and PACF plot, we can infer period = 7.
- Also our data is daily data.
- Also, in next section, we plotted weekly means and again can confirm there is 7 day seasonality

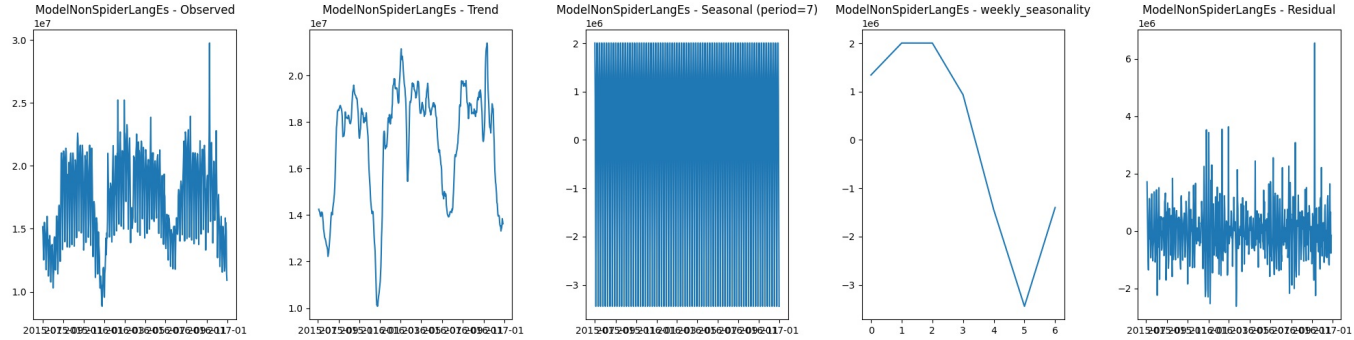
In [117…

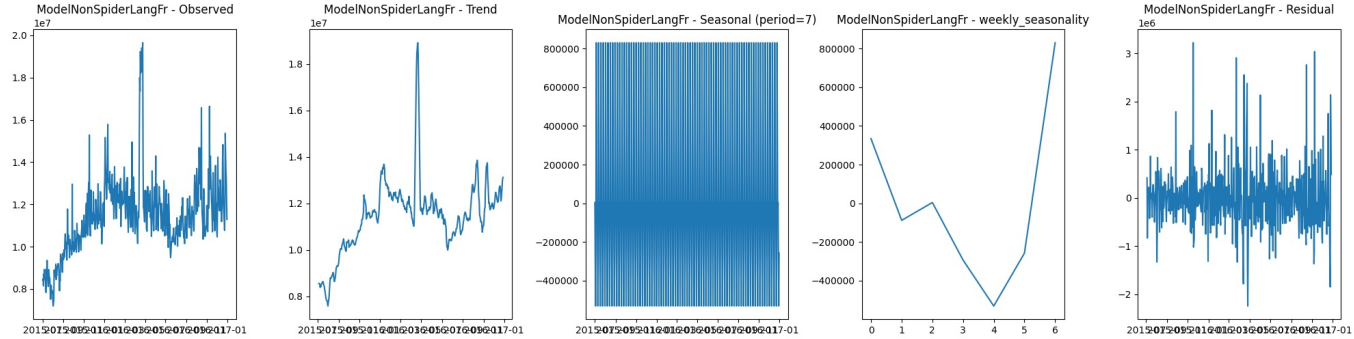
```
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt

def plotSeasonalDecomposition(timeSeriesCombined_df, modelNames, period=7):
    """
    Plot seasonal decomposition for all segments.
    """
    fig, axes = plt.subplots(len(timeSeriesCombined_df.columns), 5, figsize=(20, 5*len(timeSeriesCombined_df.co

    for i,model in enumerate(modelNames):
        ts = timeSeriesCombined_df[model]

        try:
            decomposition = seasonal_decompose(ts, model='additive', period=period)

            axes[i, 0].plot(decomposition.observed)
            axes[i, 0].set_title(f'{model} - Observed')

            axes[i, 1].plot(decomposition.trend)
            axes[i, 1].set_title(f'{model} - Trend')

            axes[i, 2].plot(decomposition.seasonal)
            axes[i, 2].set_title(f'{model} - Seasonal (period={period})')

            weekly_seasonality = (
                decomposition.seasonal
                .groupby(decomposition.seasonal.index.dayofweek)
                .mean()
            )
            # weekly_seasonality.index = ['Mon','Tue','Wed','Thu','Fri','Sat','Sun']
            # weekly_seasonality.plot(marker='o', title='Average Weekly Seasonality', axes=axes[i, 3])


            axes[i, 3].plot(weekly_seasonality)
            axes[i, 3].set_title(f'{model} - weekly_seasonality')

            axes[i, 4].plot(decomposition.resid)
            axes[i, 4].set_title(f'{model} - Residual')

        except Exception as e:
            axes[i, 0].text(0.5, 0.5, f'Error: {e}', ha='center')
            axes[i, 0].set_title(f'{model} - Error')

    plt.tight_layout()
    plt.show()

#We do seasonal_decompose on the original (non-differenced) series, not the differenced one.
plotSeasonalDecomposition(timeSeriesFull_df, modelNames, period=7)
```

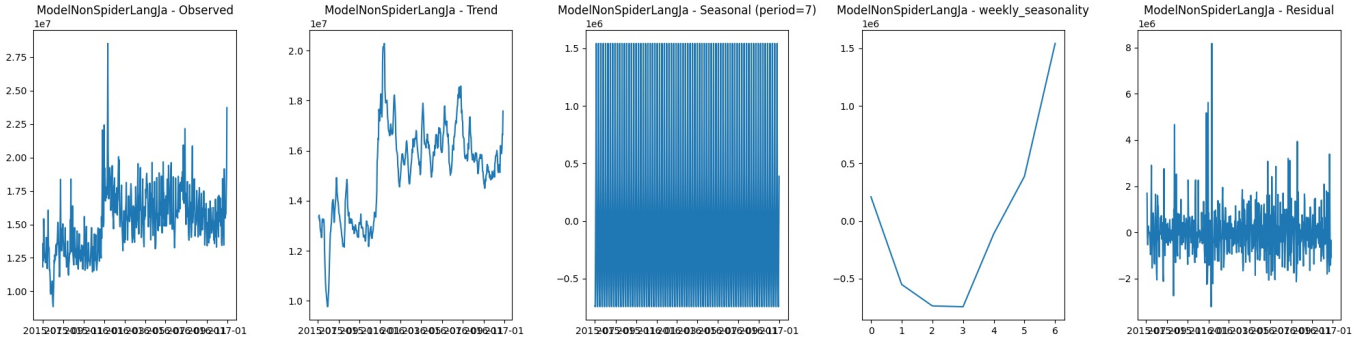


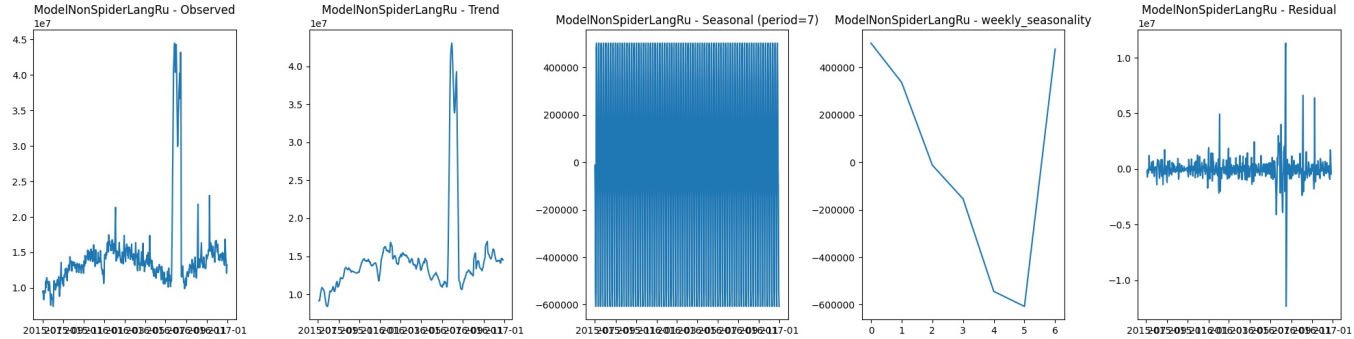

ModelNonSpiderLangEn - Observed
ModelNonSpiderLangEn - Trend
ModelNonSpiderLangEn - Seasonal (period=7)
ModelNonSpiderLangEn - weekly_seasonality
ModelNonSpiderLangEn - Residual

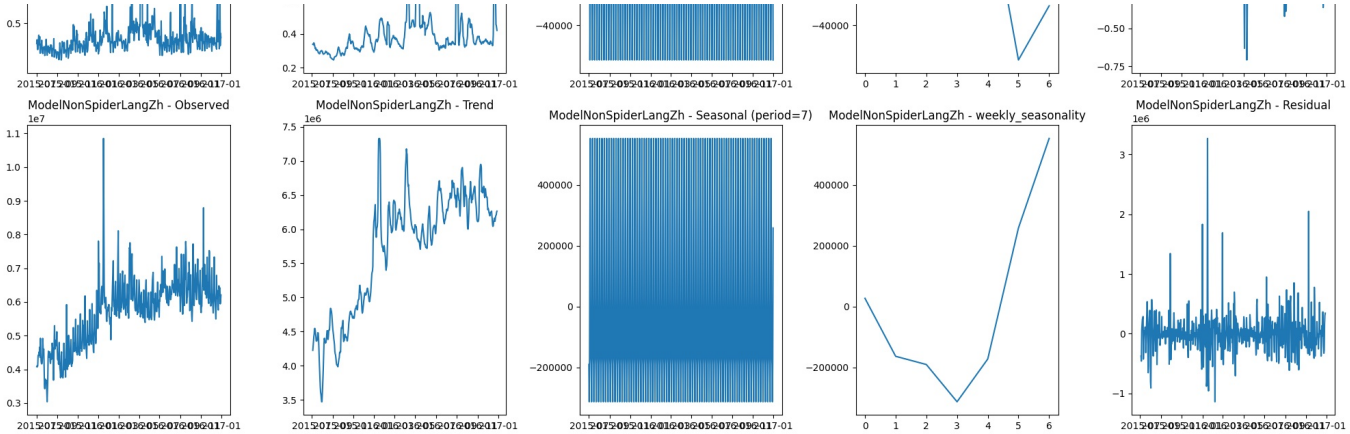

ModelNonSpiderLangEs - Observed
ModelNonSpiderLangEs - Trend
ModelNonSpiderLangEs - Seasonal (period=7)
ModelNonSpiderLangEs - weekly_seasonality
ModelNonSpiderLangEs - Residual


ModelNonSpiderLangFr - Observed
ModelNonSpiderLangFr - Trend
ModelNonSpiderLangFr - Seasonal (period=7)
ModelNonSpiderLangFr - weekly_seasonality
ModelNonSpiderLangFr - Residual


ModelNonSpiderLangJa - Observed
ModelNonSpiderLangJa - Trend
ModelNonSpiderLangJa - Seasonal (period=7)
ModelNonSpiderLangJa - weekly_seasonality
ModelNonSpiderLangJa - Residual


ModelNonSpiderLangRu - Observed
ModelNonSpiderLangRu - Trend
ModelNonSpiderLangRu - Seasonal (period=7)
ModelNonSpiderLangRu - weekly_seasonality
ModelNonSpiderLangRu - Residual


ModelNonSpiderLangWww - Observed
ModelNonSpiderLangWww - Trend
ModelNonSpiderLangWww - Seasonal (period=7)
ModelNonSpiderLangWww - weekly_seasonality
ModelNonSpiderLangWww - Residual



Big Picture (Across All Segments)

- ✅ 1. Weekly seasonality exists in **every segment**
  - Even though the *shape* differs, **none of the weekly_seasonality plots are flat**.
  - ➡ This justifies:
    - `s = 7` **globally**
    - At least considering `D = 1` for most segments
- ✅ 2. Seasonality is **asymmetric**, not sinusoidal
  - Across segments:
    - Gradual buildup during weekdays
    - Sharp drop around Fri–Sat
    - Partial recovery on Sunday
  - ➡ This is **behavioral seasonality**, not a smooth cycle.
  - **Implication**
    - SARIMA alone may struggle
    - SARIMAX + weekday exogenous variables will perform better
- Residual Diagnostics (What to Notice)
  - Across most segments:
    - Residuals are centered around zero ✅
    - Variance spikes remain ❌
  - Meaning
    - Weekly seasonality explains structure
    - But **exogenous shocks remain**

Segment-Level Insights

1️⃣ ModelSpider (Crawler traffic)

What we see

- Very strong weekday vs weekend contrast
- Large positive midweek
- Deep negative on Sat/Sun
- Residuals still show spikes → external events

Interpretation

- Crawlers are **work-week heavy**
- Likely throttled or inactive on weekends

✅ **Modeling choice**

- `s = 7`, `D = 1`
- Consider **NO exog** (seasonality is extremely regular)
- Keep AR terms slightly higher

2️⃣ NonSpiderLangCommons / En / De / Fr / Es (Human traffic)

What we see

- Clean trend + strong weekly seasonality
- Midweek peak (Wed/Thu)

- Saturday trough is dominant
- Seasonal amplitude is **large relative to noise**

Interpretation

- Human usage is **business-day driven**
- Predictable, stable behavior

✅ **Modeling choice**

- SARIMAX with:
  - `s = 7`, `D = 1`
  - Weekday dummies as exog
- Lower AR/MA complexity needed once exog is added

---

3 Asian languages (Ja, Zh)

What we see

- Still weekly seasonality
- Shape slightly shifted (earlier/later peaks)
- Weekend drop exists but may differ in depth

Interpretation

- Cultural/workweek differences
- Same cycle length, different profile

✅ **Modeling choice**

- `s = 7`, `D = 1`
- Weekday exog still valid
- Do **not** assume same coefficients as EN/FR

---

4 WWW / Ru (noisier segments)

What we see

- Weekly shape exists but noisier
- Trend is less smooth
- Residuals have more spikes

Interpretation

- Mixed traffic sources
- External shocks (events, bots, releases)

✅ **Modeling choice**

- SARIMAX with:
  - `s = 7`
  - Possibly `D = 0` (test both)
- Exog helps stabilize forecasts

Summary

> "All segments show strong but asymmetric weekly seasonality with a sharp weekend drop, justifying `s=7` globally and favoring SARIMAX models with weekday exogenous regressors over purely seasonal AR terms."

> ✅ Per-segment tuning

Segment	Type	D	Exog
Spider	1	No	Human language	1	Yes (weekday)	Noisy/mixed	0–1 (test)	Yes

ACF and PACF for seasonally differenced data

Do we need to do it?

- We already know seasonality. From our decomposition:

    - s = 7 ✅
    - Weekly pattern ✅
    - Often needs D = 1 ✅
    - ACF/PACF won't tell you anything new here.
- ACF PACF are mostly exploratory. I could have used them them when:
    - We want to shrink the grid
    - We're modeling one series only
    - We need a quick exploratory check

We will straightaway do a grid search after carefully setting the grid search space.

Fitting SARIMA model ----------

Seasonal AR/MA kept minimal (P≤1, Q=0) because:

- Weekly seasonality removed via D=1. That removes:
    - the mean weekly pattern
    - most weekly persistence
    - What remains is weak residual seasonality.
    - **➡ One seasonal AR term (P=1) is enough to mop up what's left.**
    - Higher P causes over-coupling across weeks
        - P=2 means we are saying this week depends on the last TWO weeks
        - But our plots show strong weekday pattern
- No strong residual seasonal autocorrelation,
- Q competes with exogenous regressors.
    - We are already planning for addding weekday effects (explicit) and maybe events later
    - Seasonal MA then becomes redundant.
    - **➡so Q=0**
- Why not let grid search decide everything?
    - Because SARIMA is **not convex**:
        - Many local minima
        - Many invalid models
        - AIC comparisons become noisy
        - Constraining the grid is a modeling decision, not laziness.
        - However if improvement is not there for some segments, we can consider expanding the grid in a controlled manner for those segments

In [118…

```
SARIMA_SEARCH_SPACE = {
    "ModelSpider": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [0],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangCommons": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangDe": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangEn": {
        "p": [0, 1],
        "d": [1],
        "q": [0, 1],
        "P": [0, 1],
```

```
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangEs": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangFr": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangJa": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangRu": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangWww": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    },
    "ModelNonSpiderLangZh": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7]
    }
}


SARIMA_SEARCH_SPACEold = {
    "ModelSpider": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [0],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangCommons": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangDe": {
```

```
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangEn": {
        "p": [0, 1],
        "d": [1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangEs": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangFr": {
        "p": [0, 1, 2],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangJa": {
        "p": [0, 1, 2],
        "d": [1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangRu": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangWww": {
        "p": [1, 2, 3],
        "d": [0],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    },
    "ModelNonSpiderLangZh": {
        "p": [0, 1],
        "d": [1],
        "q": [0],
        "P": [0, 1],
        "D": [1],
        "Q": [0],
        "s": [7],
        "numExog": 0
    }
}
```


We already have the function. Just need to pass new grid space.

In [118…

```
r,b=gridSearchSARIMAXAllSegments(timeSeriesCombined_df=timeSeriesFull_df,
                                 modelNames=modelNames,
                                 modelDict=modelDict,
                                 searchSpace=SARIMA_SEARCH_SPACE,
                                 exog_all_segments_df=None,
                                 train_ratio=0.8,
                                 metric='mape',
                                 searchTitle="SARIMA GridSearch",
                                 comments="Used SARIMA_SEARCH_SPACE",
                                 verbose=False)
```

```
============================================================
GRID SEARCH: ModelSpider
============================================================
Testing 6 combinations...
MAPE: 0.649
  Time: 0.3 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangCommons
============================================================
Testing 12 combinations...
MAPE: 0.294
  Time: 1.8 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangDe
============================================================
Testing 4 combinations...
MAPE: 0.074
  Time: 0.1 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangEn
============================================================
Testing 8 combinations...
MAPE: 0.885
  Time: 0.5 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangEs
============================================================
Testing 6 combinations...
MAPE: 0.139
  Time: 0.3 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangFr
============================================================
Testing 6 combinations...
MAPE: 0.076
  Time: 0.3 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangJa
============================================================
Testing 12 combinations...
MAPE: 0.072
  Time: 0.8 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangRu
============================================================
Testing 6 combinations...
MAPE: 0.071
  Time: 0.5 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangWww
============================================================
Testing 6 combinations...
MAPE: 0.302
  Time: 0.4 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangZh
============================================================
Testing 4 combinations...
MAPE: 0.457
  Time: 0.1 seconds
```

In [ ]:

```
printModelDictSummaryDETAILED(modelDict)
```

Model Dictionary Summary

Model Name	Data Rows	# Attempts	Best Model	Best Params	Best MAPE

| ModelSpider | 34913 | 2 | SARIMA GridSearch | ('(1, 0, 0)', '(1, 0, 0, 7)', []) | 0.6490% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 73.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 0, 0)', '(1, 0, 0, 7)', []) | | | | 64.9% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangCommons | 8005 | 2 | ARIMA GridSearch | ('(0, 1, 0)', '(0, 0, 0, 0)', []) | 0.1470% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 14.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 1, 0)', '(1, 1, 0, 7)', []) | | | | 29.4% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangDe | 13960 | 2 | ARIMA GridSearch | ('(0, 1, 0)', '(0, 0, 0, 0)', []) | 0.0740% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 7.4% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(1, 1, 0, 7)', []) | | | | 7.4% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangEn | 19177 | 2 | ARIMA GridSearch | ('(0, 1, 0)', '(0, 0, 0, 0)', []) | 0.0690% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.9% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 1, 1)', '(1, 1, 0, 7)', []) | | | | 88.5% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangEs | 10532 | 2 | SARIMA GridSearch | ('(2, 0, 0)', '(1, 1, 0, 7)', []) | 0.1390% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 18.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(2, 0, 0)', '(1, 1, 0, 7)', []) | | | | 13.9% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangFr | 13302 | 2 | SARIMA GridSearch | ('(0, 0, 0)', '(0, 1, 0, 7)', []) | 0.0760% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(2, 0, 0)', '(0, 0, 0, 0)', []) | | | | 11.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 0, 0)', '(0, 1, 0, 7)', []) | | | | 7.6% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangJa | 15424 | 2 | SARIMA GridSearch | ('(1, 1, 0)', '(0, 1, 0, 7)', []) | 0.0720% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 1)', '(0, 0, 0, 0)', []) | | | | 8.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(0, 1, 0, 7)', []) | | | | 7.2% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangRu | 11280 | 2 | SARIMA GridSearch | ('(1, 0, 0)', '(1, 1, 0, 7)', []) | 0.0710% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 32.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', []) | | | | 7.1% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangWww | 5551 | 2 | SARIMA GridSearch | ('(3, 0, 0)', '(0, 1, 0, 7)', []) | 0.3020% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 78.6% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(3, 0, 0)', '(0, 1, 0, 7)', []) | | | | 30.2% | Used SARIMA_SEARCH_SPACE | AIC... | | ModelNonSpiderLangZh | 12919 | 2 | ARIMA GridSearch | ('(1, 1, 0)', '(0, 0, 0, 0)', []) | 0.0650% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.5% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(1, 1, 0, 7)', []) | | | | 45.7% | Used SARIMA_SEARCH_SPACE | AIC... | | **TOTAL** | **145063** | **20** | - | - | - |

Validation

Metric	Count
Total rows across models	145063
Original dataframe rows	145063
Difference	0

✅ **MATCH:** Total rows equals original dataframe!

Best MAPE Ranking

| Rank | Segment Name | Best MAPE |Source |------|------------|----------|------| | 1 | ModelNonSpiderLangZh | 6.5% |ARIMA GridSearch('(1, 1, 0)', '(0, 0, 0, 0)', []) | 2 | ModelNonSpiderLangEn | 6.9% |ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | 3 | ModelNonSpiderLangRu | 7.1% |SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', []) | 4 | ModelNonSpiderLangJa | 7.2% |SARIMA GridSearch('(1, 1, 0)', '(0, 1, 0, 7)', []) | 5 | ModelNonSpiderLangDe | 7.4% |ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | 6 | ModelNonSpiderLangFr | 7.6% |SARIMA GridSearch('(0, 0, 0)', '(0, 1, 0, 7)', []) | 7 | ModelNonSpiderLangEs | 13.9% |SARIMA GridSearch('(2, 0, 0)', '(1, 1, 0, 7)', []) | 8 | ModelNonSpiderLangCommons | 14.7% |ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | 9 | ModelNonSpiderLangWww | 30.2% |SARIMA GridSearch('(3, 0, 0)', '(0, 1, 0, 7)', []) | 10 | ModelSpider | 64.9% |SARIMA GridSearch('(1, 0, 0)', '(1, 0, 0, 7)', [])

Fitting SARIMAX model ----------

- We have got campaign information for English pages as exogeneous variable.
- We decided to have weekday/weekend exogeneous column for most of our segments in previous analysis
- To see if this would even be useful, a visual inspection can be done
- We plot a vertical line on the graph whenever there is a campaign

Buiding Exogeneous column for each segment

In [118…

```
import pandas as pd

def build_weekend_exog_df(index, segment_names):
    is_weekend = index.dayofweek.isin([5, 6]).astype(int)

    exog_df = pd.DataFrame(
```

```
        {
            f"{segment}_exog_weekend": is_weekend
            for segment in segment_names
        },
        index=index
    )

    return exog_df

exog_df = build_weekend_exog_df(timeSeriesFull_df.index, modelNames)
exog_df
```

Out[118…

	ModelSpider_exog_weekend	ModelNonSpiderLangCommons_exog_weekend	ModelNonSpiderLangDe_exog_weekend	ModelNor
2015-07-01	0	0	0	
2015-07-02	0	0	0	
2015-07-03	0	0	0	
2015-07-04	1	1	1	
2015-07-05	1	1	1	
...	...	...	...	
2016-12-27	0	0	0	
2016-12-28	0	0	0	
2016-12-29	0	0	0	
2016-12-30	0	0	0	
2016-12-31	1	1	1	

550 rows × 10 columns

In [118…

```
# We give 1 in the name, so English now will have TWO exogeneous columns.
exog_df = addExogeneousColumnToADataFrame(exog_df, exog_campaign_series,'ModelNonSpiderLangEn_exog_campaign')
exog_df.columns
```

```
addExogeneousColumnToADataFrame():Adding the exogeneous variable (campaign data) as a column
```

Out[118…

```
Index(['ModelSpider_exog_weekend', 'ModelNonSpiderLangCommons_exog_weekend',
       'ModelNonSpiderLangDe_exog_weekend',
       'ModelNonSpiderLangEn_exog_weekend',
       'ModelNonSpiderLangEs_exog_weekend',
       'ModelNonSpiderLangFr_exog_weekend',
       'ModelNonSpiderLangJa_exog_weekend',
       'ModelNonSpiderLangRu_exog_weekend',
       'ModelNonSpiderLangWww_exog_weekend',
       'ModelNonSpiderLangZh_exog_weekend',
       'ModelNonSpiderLangEn_exog_campaign'],
      dtype='object')
```

In [118…

```
SARIMAX_SEARCH_SPACE = {
    "ModelSpider": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangCommons": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangDe": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
```

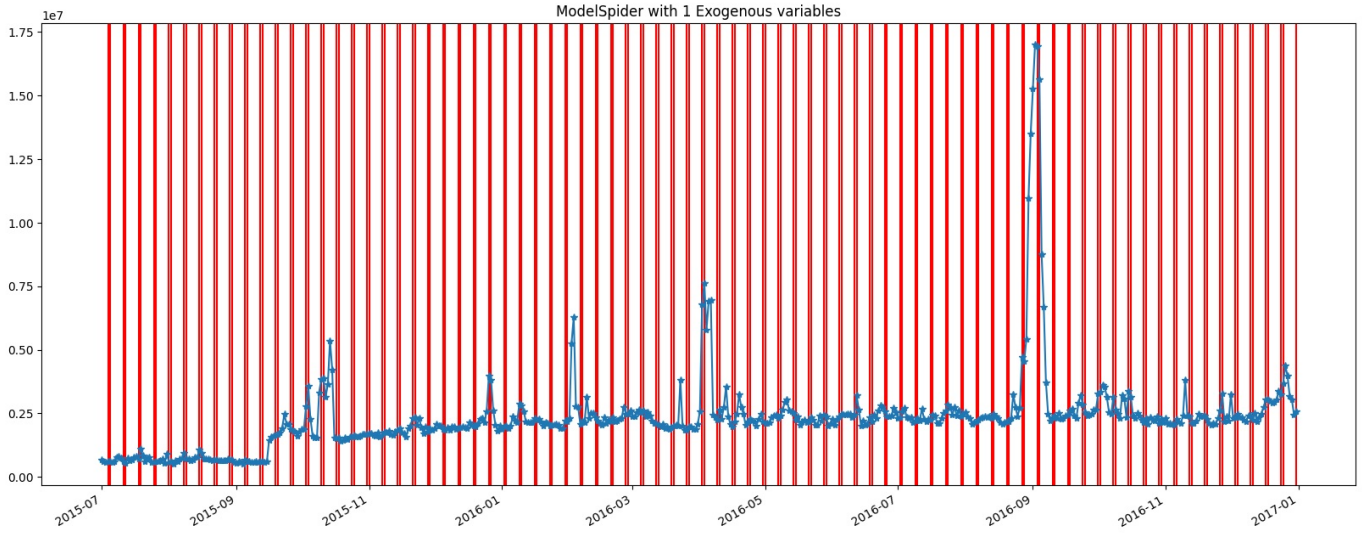
```
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangEn": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangEs": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangFr": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangJa": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangRu": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangWww": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    },
    "ModelNonSpiderLangZh": {
        "p": [1, 2, 3, 4],
        "d": [0, 1],
        "q": [0, 1],
        "P": [0, 1],
        "D": [0, 1],
        "Q": [0, 1],
        "s": [7]
    }
}
```

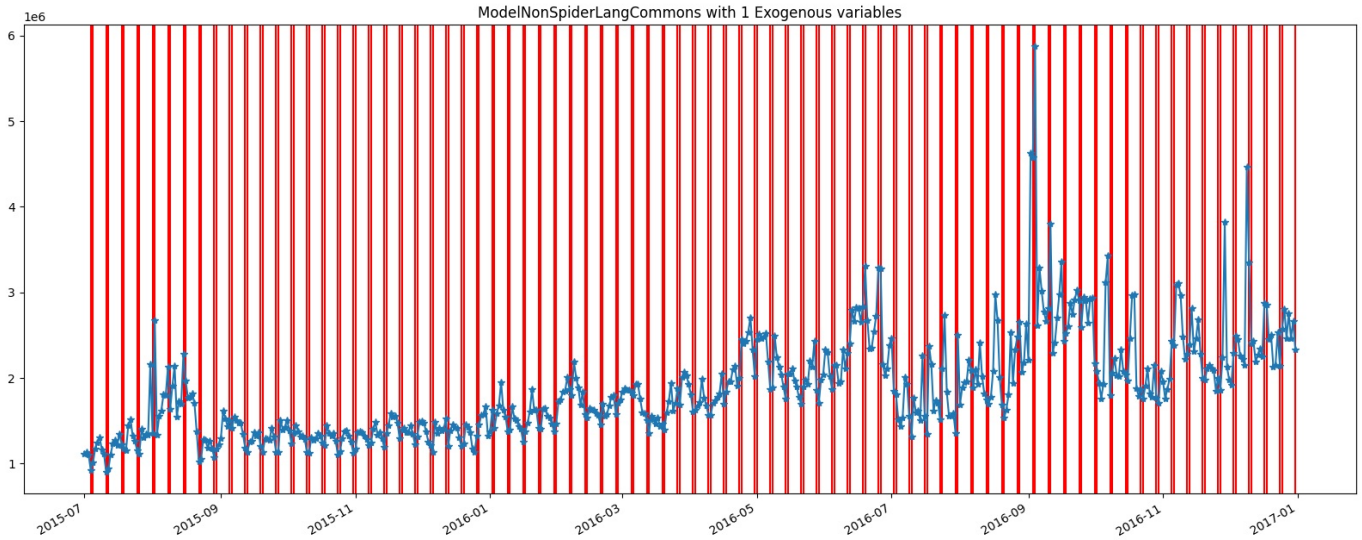
Visual examination of Exogeneous Variable impact on Time Series

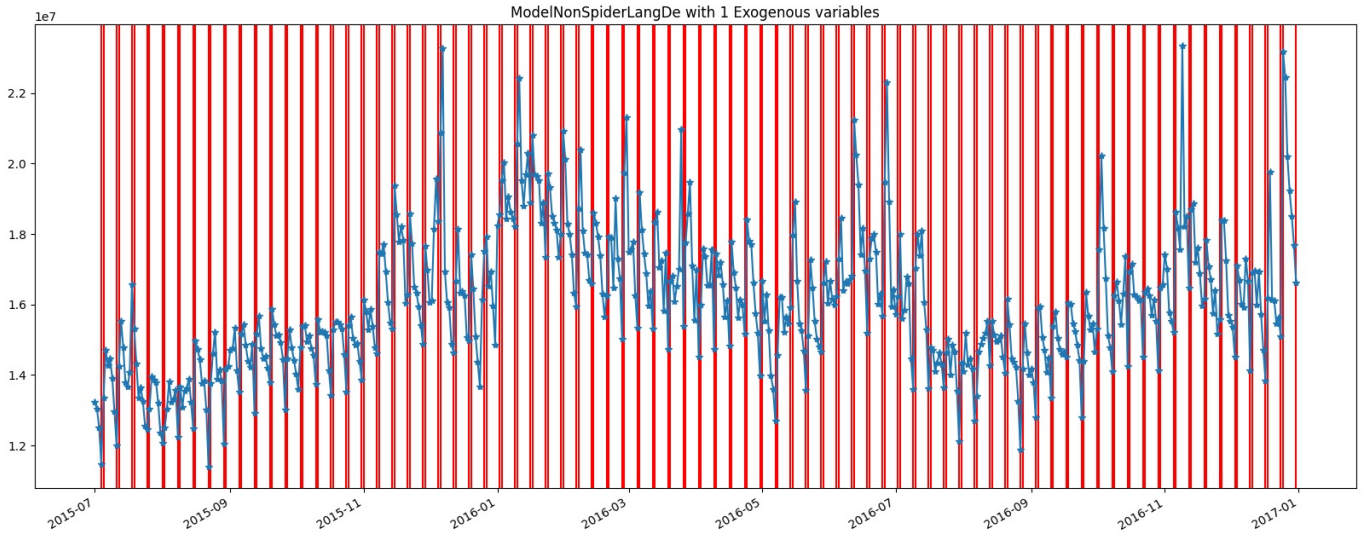
In [118…

```
# First we collect all indexes where campaign are there

# def plot_data_with_campaign(data,exogData, segment_names, space):
#     colors = ['red', 'orange', 'purple']
#     for segment in segment_names:
#         for exog in range(space[segment]['numExog']):
#             campaign = exogData[exogData[f'{segment}_exog_{exog}']==1].index
#             for day in campaign:
#                 plt.axvline(x=day, color=colors[exog % len(colors)])
```

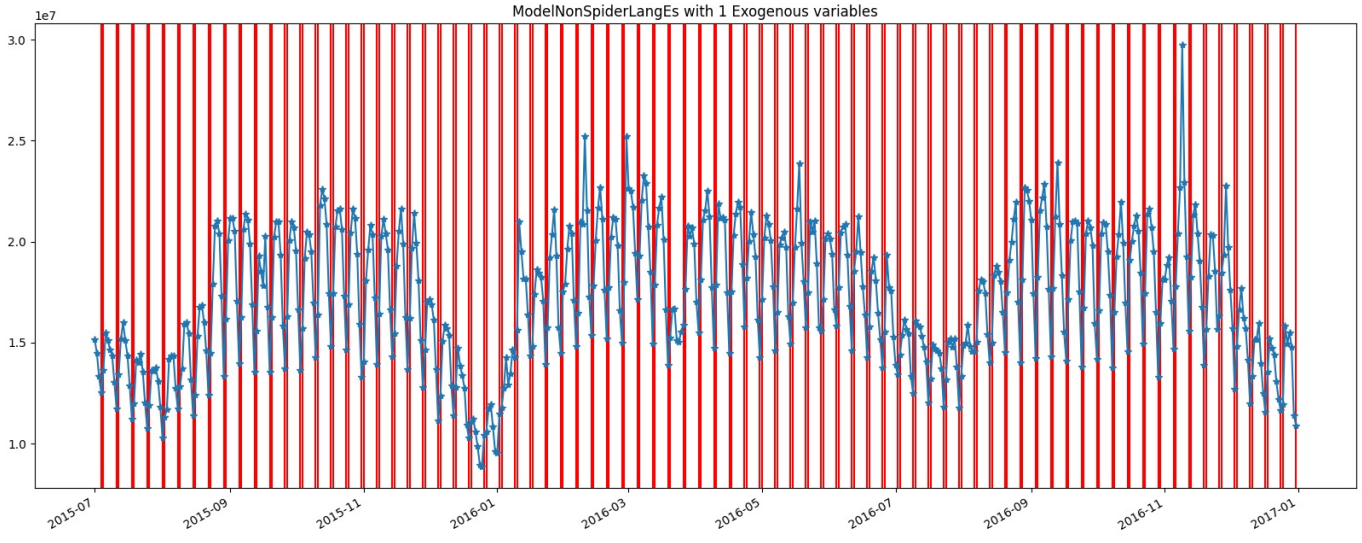
```
#         data[segment].plot(style='-*')
#         plt.title(f"{segment} with {space[segment]['numExog']} Exogeneous variable")
#         plt.show()

def plot_data_with_campaign(data, exogData, segment_names):
    colors = ['red', 'orange', 'purple']

    for segment in segment_names:
        # find all exogenous columns for this segment
        exog_cols = [col for col in exogData.columns if col.startswith(f"{segment}_exog")]

        for i, col in enumerate(exog_cols):
            campaign_days = exogData[exogData[col] == 1].index
            for day in campaign_days:
                plt.axvline(x=day, color=colors[i % len(colors)])

        data[segment].plot(style='-*')
        plt.title(f"{segment} with {len(exog_cols)} Exogenous variables")
        plt.show()

plot_data_with_campaign(timeSeriesFull_df, exog_df, modelNames)
```

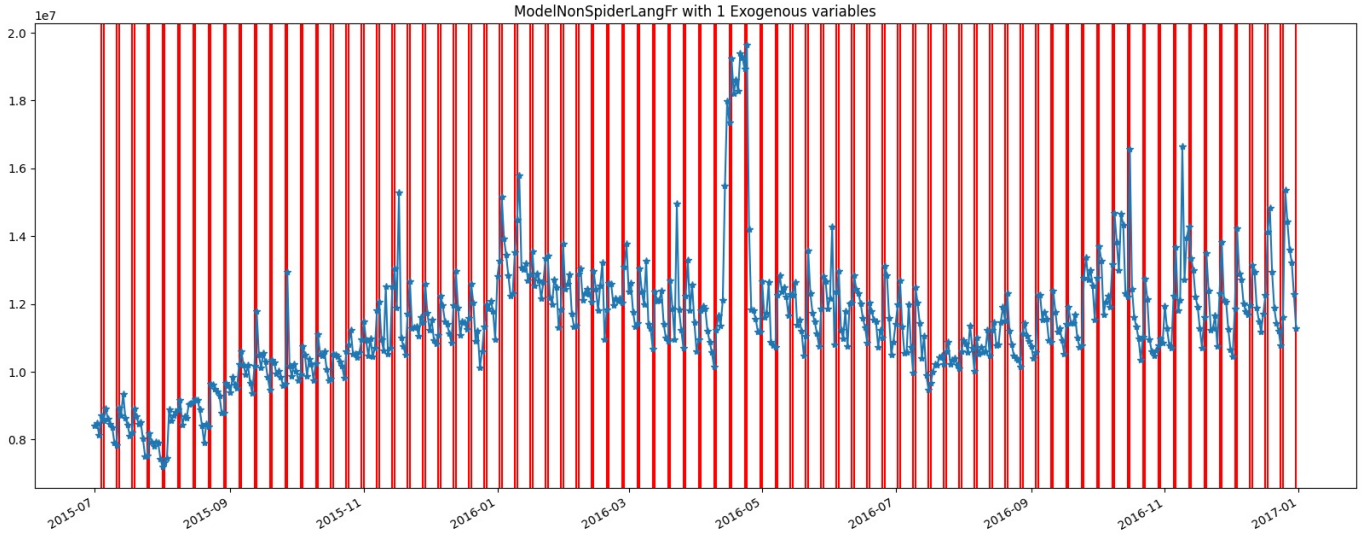


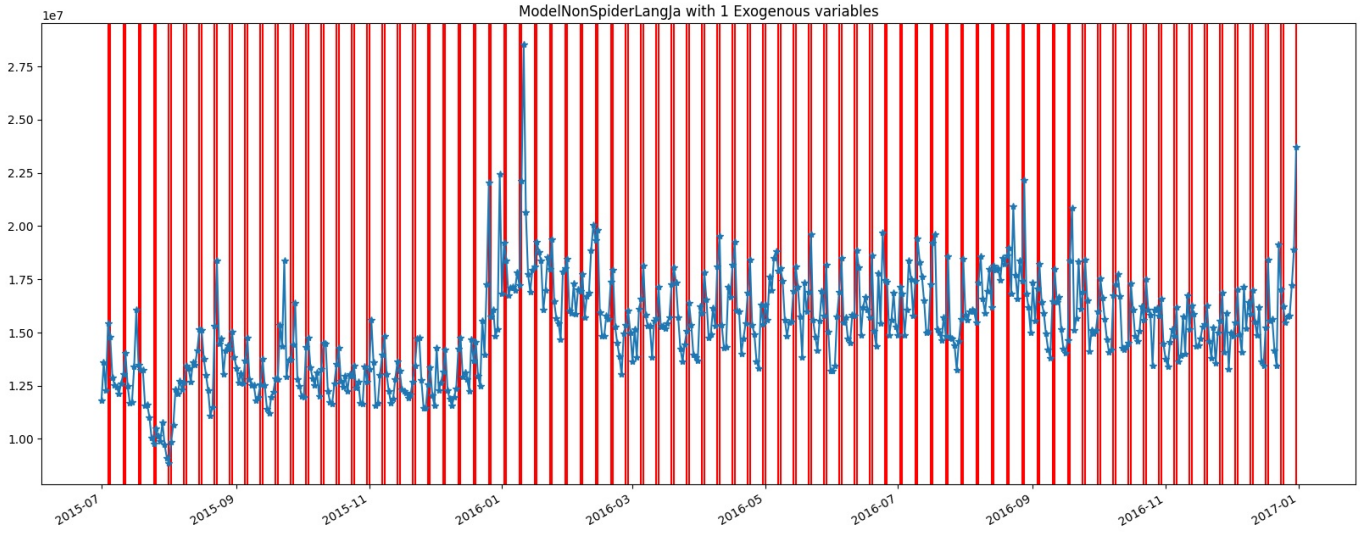


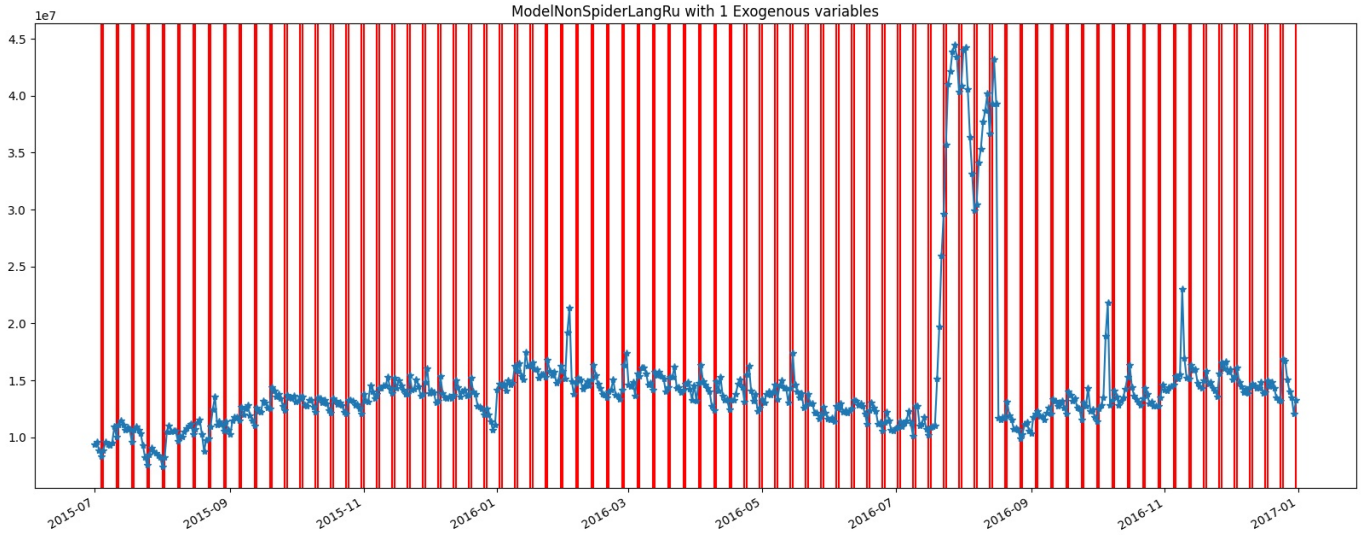

ModelNonSpiderLangDe with 1 Exogenous variables
1e7
2.2
2.0
1.8
1.6
1.4
1.2
2015-07
2015-09
2015-11
2016-01
2016-03
2016-05
2016-07
2016-09
2016-11
2017-01

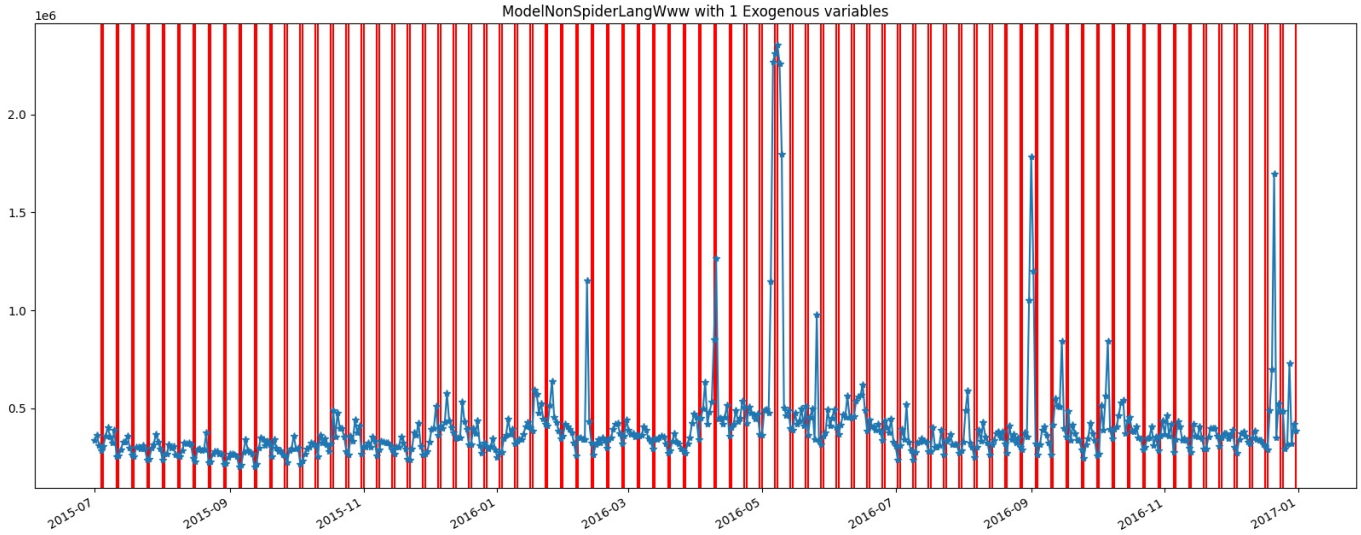

ModelNonSpiderLangEn with 2 Exogenous variables
1e8
2.0
1.8
1.6
1.4
1.2
1.0
0.8
2015-07
2015-09
2015-11
2016-01
2016-03
2016-05
2016-07
2016-09
2016-11
2017-01

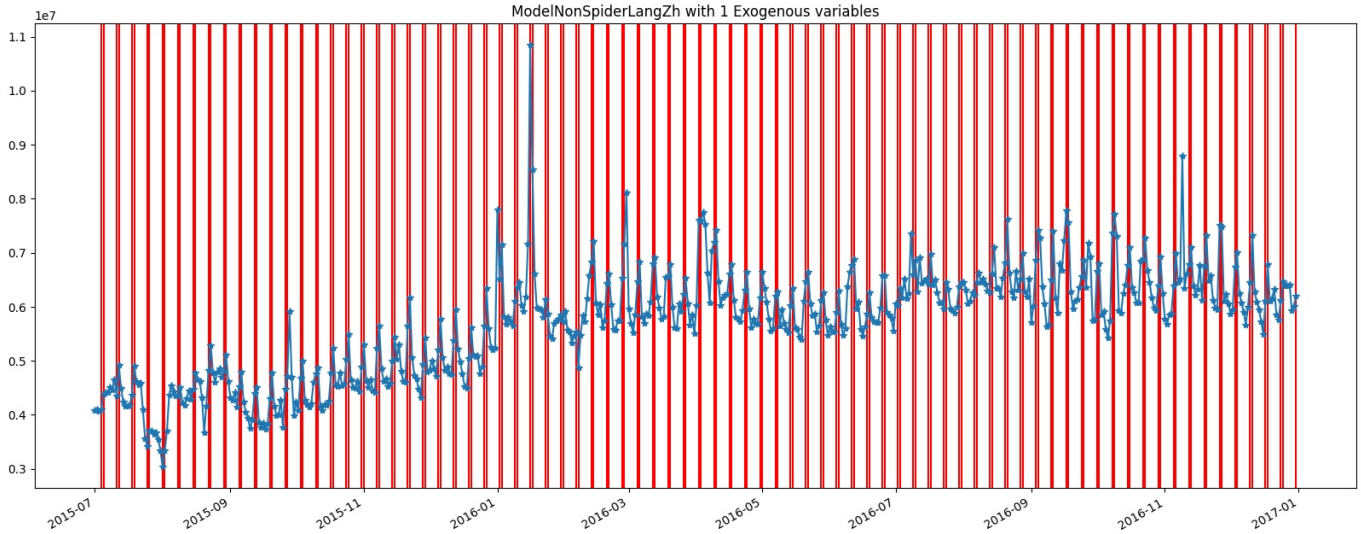

ModelNonSpiderLangEs with 1 Exogenous variables
1e7
3.0
2.5
2.0
1.5
1.0
2015-07
2015-09
2015-11
2016-01
2016-03
2016-05
2016-07
2016-09
2016-11
2017-01


ModelNonSpiderLangFr with 1 Exogenous variables
1e7
2.0
1.8
1.6
1.4
1.2
1.0
0.8
2015-07
2015-09
2015-11
2016-01
2016-03
2016-05
2016-07
2016-09
2016-11
2017-01


ModelNonSpiderLangJa with 1 Exogenous variables
1e7
2.75
2.50
2.25
2.00
1.75
1.50
1.25
1.00
2015-07
2015-09
2015-11
2016-01
2016-03
2016-05
2016-07
2016-09
2016-11
2017-01


ModelNonSpiderLangRu with 1 Exogenous variables
1e7
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
2015-07
2015-09
2015-11
2016-01
2016-03
2016-05
2016-07
2016-09
2016-11
2017-01



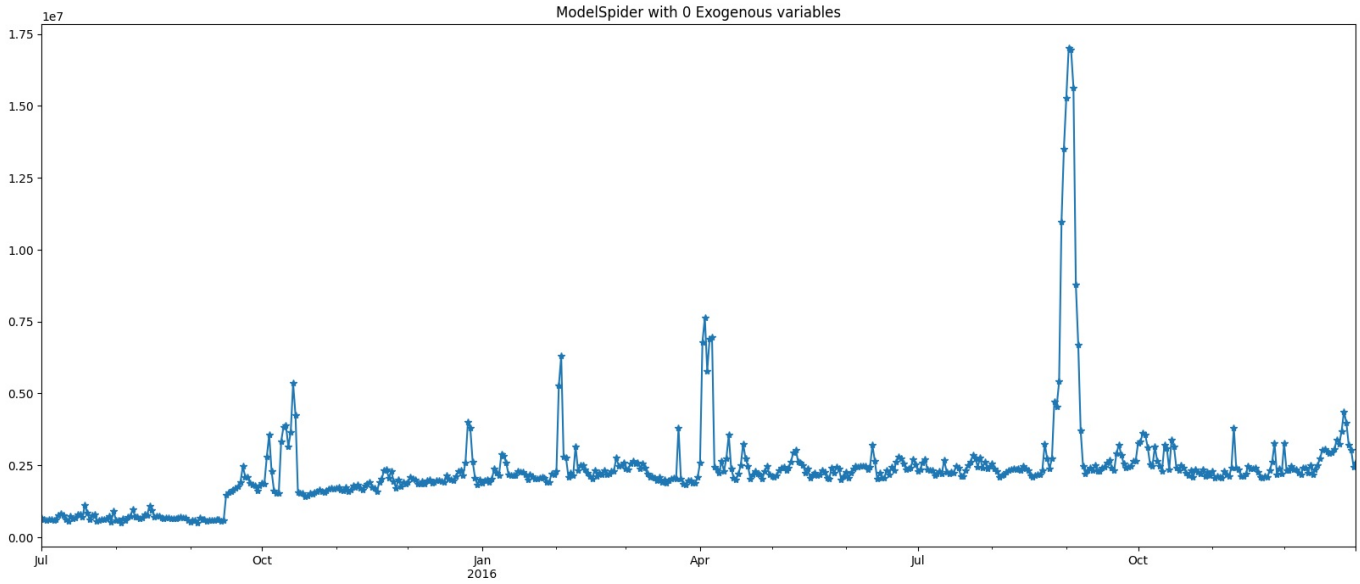


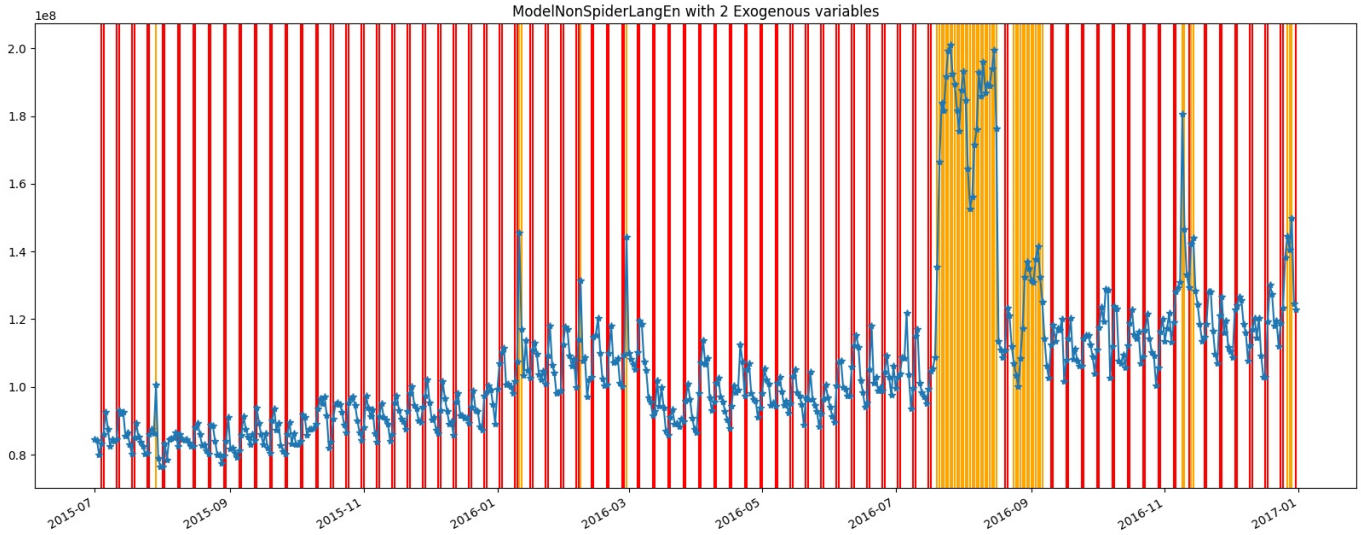
- We have got VERY good results visully...The peaks seem to match the weekend weeday exogeneous shock very nicely across the data.
- Data is suggesting that while the given exogeeous data was for only English, it may be useful for russian too.
- Spider data is not having any effect. We will remove the column for spider.

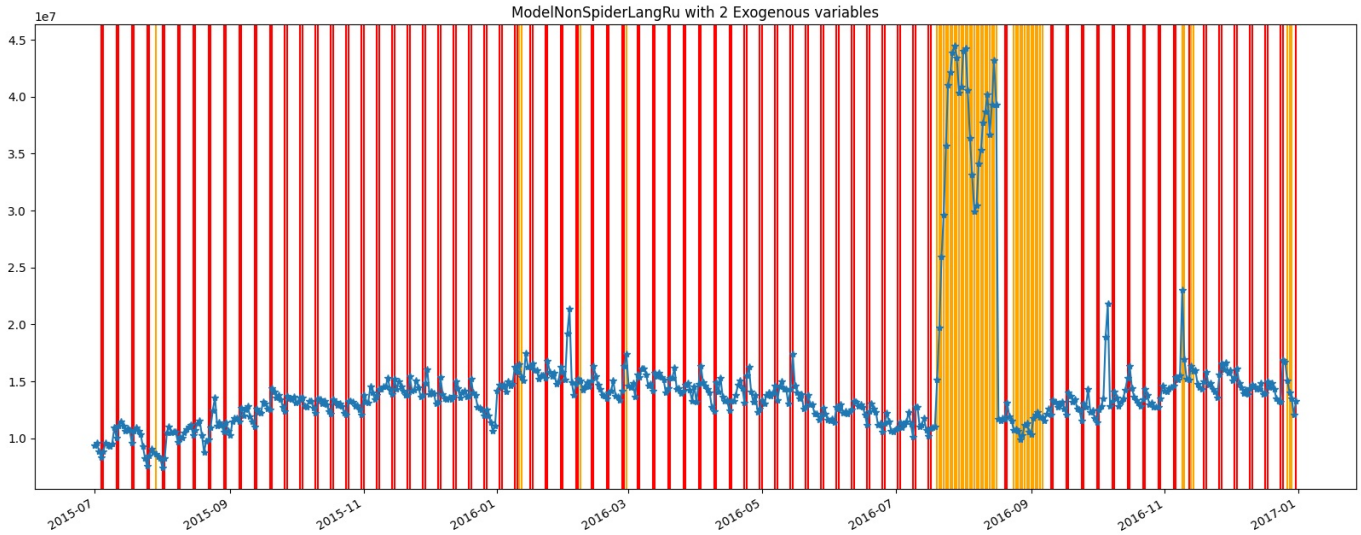
In [118…
```
exog_df = addExogeneousColumnToADataFrame(exog_df, exog_campaign_series,'ModelNonSpiderLangRu_exog_campaign')
exog_df.drop(columns=['ModelSpider_exog_weekend'], inplace=True)
```

```
addExogeneousColumnToADataFrame():Adding the exogeneous variable (campaign data) as a column
```

In [118…
```
plot_data_with_campaign(timeSeriesFull_df[['ModelSpider','ModelNonSpiderLangEn','ModelNonSpiderLangRu']], exog_
```







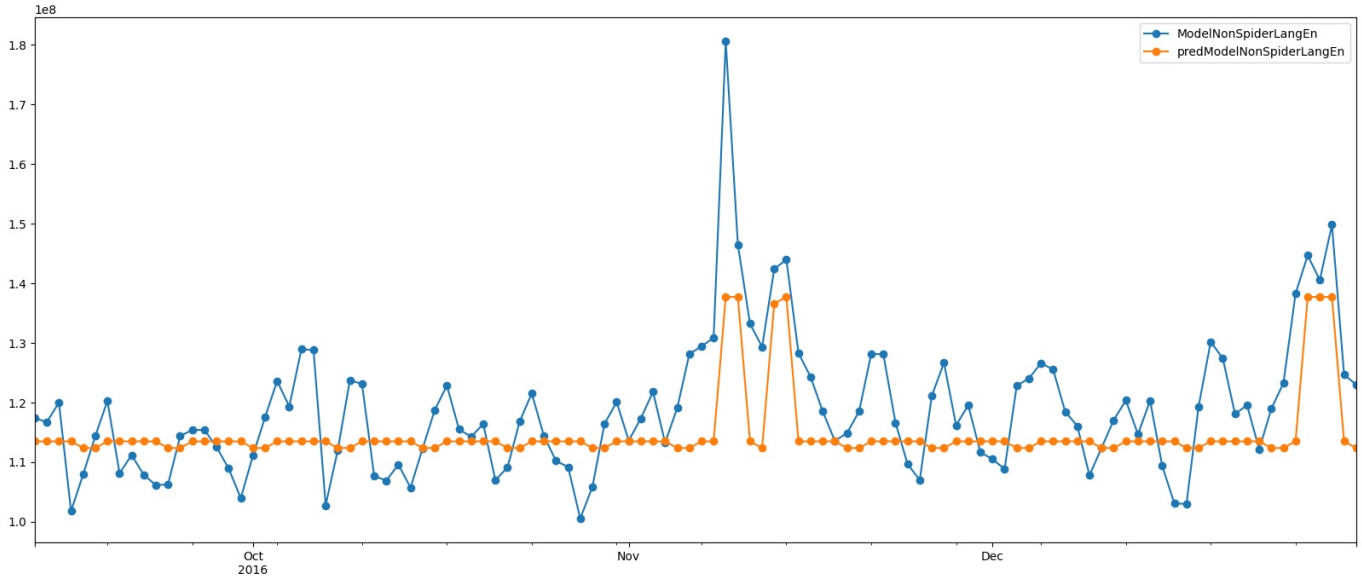
- Russian matched well with English exogeneous regressor.
- Exogeneous removed from Spider

In [118…

```
train_x = timeSeriesFull_df.loc[timeSeriesFull_df.index < train_max_date].copy()
test_x = timeSeriesFull_df.loc[timeSeriesFull_df.index >=train_max_date].copy()
exog_train = exog_df.loc[exog_df.index < train_max_date].copy()
exog_test =exog_df.loc[exog_df.index >= train_max_date].copy()
segmentName = modelNames[3]

exog_segment_cols = [col for col in exog_df.columns if col.startswith(f"{segmentName}_exog")]



model = SARIMAX(endog=train_x[segmentName],
                        order=(0, 1, 0),
                        seasonal_order=(0, 0, 0, 7),
                        exog=exog_train[exog_segment_cols],
```

```
                        enforce_stationarity=False,
                        enforce_invertibility=False)
fitted = model.fit(disp=False, maxiter=200)

# Forecast
predColumnName = 'pred'+segmentName
test_x[predColumnName] = fitted.forecast(steps=len(test_x[segmentName]), exog=exog_test[exog_segment_cols])
test_x[[segmentName,predColumnName]].plot(style='-o')
# Calculate metrics
mape_val,rmse_val, mae_val = performance(test_x[segmentName], test_x[predColumnName], verbose=True)
```

```
MAE : 7362033.451
RMSE : 9480060.725
MAPE: 0.06
```



Running gridSearchSARIMAXAllSegments

In [119…

```
r,b=gridSearchSARIMAXAllSegments(timeSeriesCombined_df = timeSeriesFull_df,
                                 modelNames=modelNames,
                                 modelDict=modelDict,
                                 searchSpace=SARIMAX_SEARCH_SPACE,
                                 exog_all_segments_df=exog_df,
                                 train_ratio=0.8,
                                 metric='mape',
                                 searchTitle="SARIMAX GridSearch",
                                 comments="Used SARIMAX_SEARCH_SPACE",
                                 verbose=False)
```

```
============================================================
GRID SEARCH: ModelSpider
============================================================
Testing 128 combinations...
MAPE: 0.115
  ⏱  Time: 24.2 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangCommons
============================================================
Testing 128 combinations...
MAPE: 0.141
  ⏱  Time: 29.0 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangDe
============================================================
Testing 128 combinations...
MAPE: 0.055
  ⏱  Time: 22.7 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangEn
============================================================
Testing 128 combinations...
MAPE: 0.056
  ⏱  Time: 31.7 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangEs
============================================================
Testing 128 combinations...
MAPE: 0.089
  ⏱  Time: 22.0 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangFr
============================================================
Testing 128 combinations...
MAPE: 0.071
  ⏱  Time: 24.3 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangJa
============================================================
Testing 128 combinations...
MAPE: 0.064
  ⏱  Time: 23.0 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangRu
============================================================
Testing 128 combinations...
MAPE: 0.076
  ⏱  Time: 26.8 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangWww
============================================================
Testing 128 combinations...
MAPE: 0.25
  ⏱  Time: 26.7 seconds

============================================================
GRID SEARCH: ModelNonSpiderLangZh
============================================================
Testing 128 combinations...
MAPE: 0.047
  ⏱  Time: 22.6 seconds
```

In [ ]:
```
printModelDictSummaryDETAILED(modelDict)
```

Model Dictionary Summary

Model Name	Data Rows	# Attempts	Best Model	Best Params	Best MAPE
ModelSpider	34913	3	SARIMAX GridSearch	('(2, 1, 0)', '(0, 0, 1, 7)', [])	0.1150%
↳ Attempts:					
1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', [])					73.8%
Used ARIMA_SEARCH_SPACE	AIC:...				
2. SARIMA GridSearch('(1, 0, 0)', '(1, 0, 0, 7)', [])					64.9%
Used SARIMA_SEARCH_SPACE	AIC...				
3. SARIMAX GridSearch('(2, 1, 0)', '(0, 0, 1, 7)', [])					11.5%
Used SARIMAX_SEARCH_SPACE	AI...				
ModelNonSpiderLangCommons	8005	3	SARIMAX GridSearch	('(3, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangCommons_exog_weekend'])	0.1410%
↳ Attempts:					
1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', [])					

14.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 1, 0)', '(1, 1, 0, 7)', []) | | | | 29.4% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(3, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangCommons_exog_weekend']) | | | | 14.1% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangDe | 13960 | 3 | SARIMAX GridSearch | ('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']) | 0.0550% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 7.4% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(1, 1, 0, 7)', []) | | | | 7.4% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']) | | | | 5.5% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangEn | 19177 | 3 | SARIMAX GridSearch | ('(1, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']) | 0.0560% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.9% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 1, 1)', '(1, 1, 0, 7)', []) | | | | 88.5% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']) | | | | 5.6% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangEs | 10532 | 3 | SARIMAX GridSearch | ('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangEs_exog_weekend']) | 0.0890% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 18.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(2, 0, 0)', '(1, 1, 0, 7)', []) | | | | 13.9% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangEs_exog_weekend']) | | | | 8.9% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangFr | 13302 | 3 | SARIMAX GridSearch | ('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']) | 0.0710% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(2, 0, 0)', '(0, 0, 0, 0)', []) | | | | 11.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 0, 0)', '(0, 1, 0, 7)', []) | | | | 7.6% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']) | | | | 7.1% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangJa | 15424 | 3 | SARIMAX GridSearch | ('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']) | 0.0640% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 1)', '(0, 0, 0, 0)', []) | | | | 8.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(0, 1, 0, 7)', []) | | | | 7.2% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']) | | | | 6.4% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangRu | 11280 | 3 | SARIMA GridSearch | ('(1, 0, 0)', '(1, 1, 0, 7)', []) | 0.0710% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 32.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', []) | | | | 7.1% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 0)', '(1, 0, 1, 7)', ['ModelNonSpiderLangRu_exog_weekend', 'ModelNonSpiderLangRu_exog_campaign']) | | | | 7.6% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangWww | 5551 | 3 | SARIMAX GridSearch | ('(2, 1, 1)', '(1, 0, 1, 7)', ['ModelNonSpiderLangWww_exog_weekend']) | 0.2500% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 78.6% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(3, 0, 0)', '(0, 1, 0, 7)', []) | | | | 30.2% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(2, 1, 1)', '(1, 0, 1, 7)', ['ModelNonSpiderLangWww_exog_weekend']) | | | | 25.0% | Used SARIMAX_SEARCH_SPACE | AI... | | ModelNonSpiderLangZh | 12919 | 3 | SARIMAX GridSearch | ('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']) | 0.0470% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.5% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(1, 1, 0, 7)', []) | | | | 45.7% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']) | | | | 4.7% | Used SARIMAX_SEARCH_SPACE | AI... | | **TOTAL** | **145063** | **30** | - | - | - |

Validation

Metric	Count
Total rows across models	145063
Original dataframe rows	145063
Difference	0

✅ **MATCH:** Total rows equals original dataframe!

Best MAPE Ranking

| Rank | Segment Name | Best MAPE |Source |------|------------|----------|------| | 1 | ModelNonSpiderLangZh | 4.7% |SARIMAX GridSearch('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']) | 2 | ModelNonSpiderLangDe | 5.5% |SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']) | 3 | ModelNonSpiderLangEn | 5.6% |SARIMAX GridSearch('(1, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']) | 4 | ModelNonSpiderLangJa | 6.4% |SARIMAX GridSearch('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']) | 5 | ModelNonSpiderLangFr | 7.1% |SARIMAX GridSearch('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']) | 6 | ModelNonSpiderLangRu | 7.1% |SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', []) | 7 | ModelNonSpiderLangEs | 8.9% |SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangEs_exog_weekend']) | 8 | ModelSpider | 11.5% |SARIMAX GridSearch('(2, 1, 0)', '(0, 0, 1, 7)', []) | 9 | ModelNonSpiderLangCommons | 14.1% |SARIMAX GridSearch('(3, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangCommons_exog_weekend']) | 10 | ModelNonSpiderLangWww | 25.0% |SARIMAX GridSearch('(2, 1, 1)', '(1, 0, 1, 7)', ['ModelNonSpiderLangWww_exog_weekend'])

Fitting Prophet ----------

Prophet helper functions

`fitProphetWithRegressors()`

In [119…

```python
from prophet import Prophet

def fitProphetWithRegressors(prophet_df, exog_cols,
                             train_ratio=0.8,
                             seasonality_mode='multiplicative',
                             changepoint_prior_scale=0.1,
                             seasonality_prior_scale=0.1,
                             weekly_seasonality=True,
                             yearly_seasonality=False,
                             verbose=True):
    """
    Fit Prophet model with exogenous regressors.

    Parameters:
    -----------
    prophet_df : DataFrame with ds, y and any exogeneous columns.
    exog_cols : str or list - exogenous variable column(s)
    train_ratio : float
    seasonality_mode : 'additive' or 'multiplicative'
    changepoint_prior_scale : float
    seasonality_prior_scale : float
    weekly_seasonality : bool
    yearly_seasonality : bool
    verbose : bool

    Returns:
    --------
    results : dict with model, forecast, metrics
    """
    # Remove NaN
    prophet_df = prophet_df.dropna()

    # Train/test split
    n = len(prophet_df)
    train_size = int(n * train_ratio)

    train_df = prophet_df.iloc[:train_size].copy()
    test_df = prophet_df.iloc[train_size:].copy()

    if verbose:
        print(f"Regressors: {exog_cols}")
        print(f"Train size: {len(train_df)}, Test size: {len(test_df)}")
        print(f"Params: seasonality_mode={seasonality_mode}, "
              f"changepoint_prior={changepoint_prior_scale}, "
              f"seasonality_prior={seasonality_prior_scale}")

    try:
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")

            # Initialize Prophet
            model = Prophet(
                seasonality_mode=seasonality_mode,
                changepoint_prior_scale=changepoint_prior_scale,
                seasonality_prior_scale=seasonality_prior_scale,
                weekly_seasonality=weekly_seasonality,
                yearly_seasonality=yearly_seasonality
            )

            # Add regressors
            for col in exog_cols:
                model.add_regressor(col)

            # Fit
            model.fit(train_df)

            # Create future dataframe with regressors
            future = model.make_future_dataframe(periods=len(test_df), freq='D')

            # Add regressor values to future
            full_regressors = prophet_df[exog_cols].values
            for i, col in enumerate(exog_cols):
                future[col] = np.concatenate([
                    full_regressors[:, i],
                    np.zeros(max(0, len(future) - len(full_regressors)))  # Pad if needed
                ])[:len(future)]

            # Predict
            forecast = model.predict(future)

            # Extract test predictions
```

```
            y_pred = forecast.iloc[train_size:]['yhat'].values[:len(test_df)]
            y_test = test_df['y'].values

            mape,rmse, mae = performance(y_test, y_pred, verbose=False)

            if verbose:
                print(f"\n✅ Prophet with Regressors Results:")
                print(f"   MAPE: {mape:.2f}%")
                print(f"   RMSE: {rmse:.2f}")
                print(f"   MAE: {mae:.2f}")

            results = {
                'model': model,
                'forecast': forecast,
                'y_test': y_test,
                'y_pred': y_pred,
                'mape': mape,
                'rmse': rmse,
                'mae': mae,
                'train_size': train_size,

                'params': {
                    'seasonality_mode': seasonality_mode,
                    'changepoint_prior_scale': changepoint_prior_scale,
                    'seasonality_prior_scale': seasonality_prior_scale,
                    'weekly_seasonality': weekly_seasonality,
                    'yearly_seasonality': yearly_seasonality,
                    'exog': exog_cols
                }
            }

            return results

    except Exception as e:
        print(f"❌ Error: {str(e)}")
        import traceback
        traceback.print_exc()
        return {'error': str(e)}
```

gridSearchProphetWithRegressors()

In [119…

```
def gridSearchProphetWithRegressors(prophet_df, exog_cols, train_ratio=0.8, verbose=True):
    """
    Grid search over Prophet hyperparameters.

    Returns:
    --------
    results_df : DataFrame with all combinations
    best_params : dict with best parameters
    """

    # Parameter grid
    param_grid_comprehensive = {
        'seasonality_mode': ['additive', 'multiplicative'],
        'changepoint_prior_scale': [0.001, 0.01, 0.05, 0.1, 0.5],
        'seasonality_prior_scale': [0.1, 1.0, 10.0],
        'weekly_seasonality': [True],  # True = auto, or custom Fourier order
        'yearly_seasonality': [False, True]
    }

    param_grid_fast = {
        'seasonality_mode': ['additive'],
        'changepoint_prior_scale': [0.001, 0.01, 0.05, 0.1],
        'seasonality_prior_scale': [0.1, 1.0],
        'weekly_seasonality': [True],  # True = auto, or custom Fourier order
        'yearly_seasonality': [True]
    }

    param_grid = param_grid_comprehensive

    # Generate all combinations
    from itertools import product

    keys = param_grid.keys()
    combinations = list(product(*param_grid.values()))

    total = len(combinations)
    if verbose:
        print(f"Testing {total} combinations...")

    results = []
```

```
    for idx, combo in enumerate(combinations):
        params = dict(zip(keys, combo))

        try:
            with warnings.catch_warnings():
                warnings.simplefilter("ignore")

                result = fitProphetWithRegressors(
                    prophet_df, exog_cols,
                    train_ratio=train_ratio,
                    verbose=False,
                    **params
                )
                call_params = {
                **params,
                'exog': exog_cols
                }
                results.append({
                    **call_params,
                    'mape': result['mape'],
                    'rmse': result['rmse'],
                    'mae': result['mae'],
                    'converged': True
                })

        except Exception as e:
            results.append({
                **params,
                'mape': np.nan,
                'rmse': np.nan,
                'mae': np.nan,
                'converged': False,
                'error': str(e)
            })

        if verbose and (idx + 1) % 20 == 0:
            print(f"  Progress: {idx+1}/{total}")

    results_df = pd.DataFrame(results)

    # Find best
    valid = results_df[results_df['converged'] == True]
    if len(valid) > 0:
        best_idx = valid['mape'].idxmin()
        best_row = results_df.loc[best_idx]
        best_params = best_row.to_dict()
    else:
        best_params = None

    if verbose and best_params:
        print(f"\n✓ Best MAPE: {best_params['mape']:.2f}%")
        print(f"   Params: seasonality_mode={best_params['seasonality_mode']}, "
              f"changepoint_prior={best_params['changepoint_prior_scale']}")

    return results_df, best_params
```

gridSearchProphetAllSegments()

In [119…

```
def gridSearchProphetAllSegments(timeSeriesCombined_df, modelNames, modelDict,
                                 exog_all_segments_df=None,
                                 train_ratio=0.8,
                                 comments="Prophet Grid Search_FAST"):
    """
    Run Prophet grid search on all segments.
    """

    all_results = {}
    best_params_all = {}

    for i, segmentName in enumerate(modelNames):
        print(f"\n{'='*60}")
        print(f"PROPHET GRID SEARCH: {segmentName} ({i+1}/{len(modelNames)})")
        print(f"{'='*60}")

        try:
            # Create Prophet DataFrame
            prophet_df = pd.DataFrame({
                'ds': pd.to_datetime(timeSeriesCombined_df.index),
                'y': timeSeriesCombined_df[segmentName].values
            })
            exog_segment_cols=[]
```

```
            if(exog_all_segments_df is not None):
                # find all exogenous columns for this segment
                exog_segment_cols = [col for col in exog_all_segments_df.columns if col.startswith(f"{segmentNa
                exog_segment_df = exog_all_segments_df[exog_segment_cols].copy().reset_index().rename(columns={
                # Now both DataFrames have a ds column.
                prophet_df = prophet_df.merge(exog_segment_df, on="ds", how="left")


            # Run grid search
            start_time = time.time()

            results_df, best_params = gridSearchProphetWithRegressors(
                prophet_df, exog_cols = exog_segment_cols, train_ratio=train_ratio, verbose=True
            )

            elapsed = time.time() - start_time
            print(f"  ⏱  Time: {elapsed:.1f} seconds")

            all_results[segmentName] = results_df

            if best_params:
                best_params_all[segmentName] = best_params

                # Log to modelDict
                addModelAttempt(
                    modelDict, segmentName,
                    "Prophet-GridSearch",
                    str({k: v for k, v in best_params.items()
                         if k not in ['mape', 'rmse', 'mae', 'converged']}),
                    best_params['mape'],
                    comments=comments
                )

        except Exception as e:
            print(f"  ✗ Error: {str(e)}")

    return all_results, best_params_all
```

Fit Prophet after Grid Search

In [ ]:
```
# Grid search for best hyperparameters
print("="*60)
print("PROPHET GRID SEARCH")
print("="*60)

prophet_grid_results, prophet_best_params = gridSearchProphetAllSegments(
   timeSeriesCombined_df=timeSeriesFull_df,
    modelNames=modelNames,
    modelDict=modelDict,
    exog_all_segments_df=exog_df
)
```

printModelDictSummaryDETAILED

In [ ]:
```
printModelDictSummaryDETAILED(showAttempts=True)
```

Model Dictionary Summary

Model Name	Data Rows	# Attempts	Best Model	Best Params	Best MAPE

| ModelSpider | 34913 | 4 | SARIMAX GridSearch | ('(2, 1, 0)', '(0, 0, 1, 7)', []) | 0.1150% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 73.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 0, 0)', '(1, 0, 0, 7)', []) | | | | 64.9% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(2, 1, 0)', '(0, 0, 1, 7)', []) | | | | 11.5% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.001, 'seasonality_prior_scale': 10.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': []} | | | | 53.1% | Prophet Grid Search_FAST | | ModelNonSpiderLangCommons | 8005 | 4 | Prophet-GridSearch | {'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.01, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangCommons_exog_weekend']} | 0.1390% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 14.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 1, 0)', '(1, 1, 0, 7)', []) | | | | 29.4% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(3, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangCommons_exog_weekend']) | | | | 14.1% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.01, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangCommons_exog_weekend']} | | | | 13.9% | Prophet Grid Search_FAST | | ModelNonSpiderLangDe | 13960 | 4 | SARIMAX GridSearch | ('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']) | 0.0550% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 7.4% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(1, 1, 0, 7)', []) | | | | 7.4% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1,

1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']) | | | | 5.5% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.01, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangDe_exog_weekend']} | | | | 7.2% | Prophet Grid Search_FAST | | ModelNonSpiderLangEn | 19177 | 4 | Prophet-GridSearch | {'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.1, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']} | 0.0480% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.9% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 1, 1)', '(1, 1, 0, 7)', []) | | | | 88.5% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 0)', '(1, 0, 0, 7)', ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']) | | | | 5.6% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.1, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']} | | | | 4.8% | Prophet Grid Search_FAST | | ModelNonSpiderLangEs | 10532 | 4 | Prophet-GridSearch | {'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.001, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEs_exog_weekend']} | 0.0750% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 18.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(2, 0, 0)', '(1, 1, 0, 7)', []) | | | | 13.9% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangEs_exog_weekend']) | | | | 8.9% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.001, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEs_exog_weekend']} | | | | 7.5% | Prophet Grid Search_FAST | | ModelNonSpiderLangFr | 13302 | 4 | SARIMAX GridSearch | ('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']) | 0.0710% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(2, 0, 0)', '(0, 0, 0, 0)', []) | | | | 11.7% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(0, 0, 0)', '(0, 1, 0, 7)', []) | | | | 7.6% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']) | | | | 7.1% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.1, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangFr_exog_weekend']} | | | | 8.1% | Prophet Grid Search_FAST | | ModelNonSpiderLangJa | 15424 | 4 | SARIMAX GridSearch | ('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']) | 0.0640% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(0, 1, 1)', '(0, 0, 0, 0)', []) | | | | 8.8% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(0, 1, 0, 7)', []) | | | | 7.2% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']) | | | | 6.4% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.5, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangJa_exog_weekend']} | | | | 6.8% | Prophet Grid Search_FAST | | ModelNonSpiderLangRu | 11280 | 4 | SARIMA GridSearch | ('(1, 0, 0)', '(1, 1, 0, 7)', []) | 0.0710% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 32.1% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', []) | | | | 7.1% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(1, 1, 0)', '(1, 0, 1, 7)', ['ModelNonSpiderLangRu_exog_weekend', 'ModelNonSpiderLangRu_exog_campaign']) | | | | 7.6% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'multiplicative', 'changepoint_prior_scale': 0.5, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangRu_exog_weekend', 'ModelNonSpiderLangRu_exog_campaign']} | | | | 9.3% | Prophet Grid Search_FAST | | ModelNonSpiderLangWww | 5551 | 4 | Prophet-GridSearch | {'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.05, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': False, 'exog': ['ModelNonSpiderLangWww_exog_weekend']} | 0.1510% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 0, 0)', '(0, 0, 0, 0)', []) | | | | 78.6% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(3, 0, 0)', '(0, 1, 0, 7)', []) | | | | 30.2% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(2, 1, 1)', '(1, 0, 1, 7)', ['ModelNonSpiderLangWww_exog_weekend']) | | | | 25.0% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.05, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': False, 'exog': ['ModelNonSpiderLangWww_exog_weekend']} | | | | 15.1% | Prophet Grid Search_FAST | | ModelNonSpiderLangZh | 12919 | 4 | SARIMAX GridSearch | ('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']) | 0.0470% | ↳ Attempts: | | | | | | 1. ARIMA GridSearch('(1, 1, 0)', '(0, 0, 0, 0)', []) | | | | 6.5% | Used ARIMA_SEARCH_SPACE | AIC:... | | 2. SARIMA GridSearch('(1, 1, 0)', '(1, 1, 0, 7)', []) | | | | 45.7% | Used SARIMA_SEARCH_SPACE | AIC... | | 3. SARIMAX GridSearch('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']) | | | | 4.7% | Used SARIMAX_SEARCH_SPACE | AI... | | 4. Prophet-GridSearch{'seasonality_mode': 'multiplicative', 'changepoint_prior_scale': 0.5, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangZh_exog_weekend']} | | | | 5.4% | Prophet Grid Search_FAST | | **TOTAL** | **145063** | **40** | - | - | - |

Validation

Metric	Count
Total rows across models	145063
Original dataframe rows	145063
Difference	0

✅ **MATCH:** Total rows equals original dataframe!

Best MAPE Ranking

| Rank | Segment Name | Best MAPE |Source |------|-----------|----------|------| | 1 | ModelNonSpiderLangZh | 4.7% |SARIMAX

GridSearch('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']) | 2 | ModelNonSpiderLangEn | 4.8% |Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.1, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']} | 3 | ModelNonSpiderLangDe | 5.5% |SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']) | 4 | ModelNonSpiderLangJa | 6.4% |SARIMAX GridSearch('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']) | 5 | ModelNonSpiderLangFr | 7.1% |SARIMAX GridSearch('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']) | 6 | ModelNonSpiderLangRu | 7.1% |SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', []) | 7 | ModelNonSpiderLangEs | 7.5% |Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.001, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEs_exog_weekend']} | 8 | ModelSpider | 11.5% |SARIMAX GridSearch('(2, 1, 0)', '(0, 0, 1, 7)', []) | 9 | ModelNonSpiderLangCommons | 13.9% |Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.01, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangCommons_exog_weekend']} | 10 | ModelNonSpiderLangWww | 15.1% |Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.05, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': False, 'exog': ['ModelNonSpiderLangWww_exog_weekend']}

Definitely, the campaign data is aligning to a good extent to the spikes in data, although some spikes are unexplained.
This may vary with language or any other parameter. Lets check few more records

Final Model Creation

`extract_best_models()` `fit_sarimax_full()` `fit_prophet_full`

In [119…

```python
from statsmodels.tsa.statespace.sarimax import SARIMAX
from prophet import Prophet
import ast
import warnings
warnings.filterwarnings('ignore')

# ============================================================
# STEP 1: EXTRACT BEST MODELS FROM modelDict
# ============================================================

def extract_best_models(modelDict, verbose=False):
    """
    Extract best model info from your modelDict structure

    Structure:
    modelDict[segment] = {
        'data': DataFrame,
        'modelsTried': [...],
        'bestModel': {'modelName': ..., 'modelParams': ..., 'mape': ...}
    }
    """

    best_models = {}

    print("="*70)
    print("EXTRACTING BEST MODELS FROM modelDict")
    print("="*70 + "\n")

    for segment_name, segment_info in modelDict.items():
        best = segment_info['bestModel']

        model_name = best['modelName']
        model_params = best['modelParams']
        mape = best['mape']

        # Parse model type
        if 'Prophet' in model_name:
            model_type = 'Prophet'
            # Parse params string to dict if needed
            if isinstance(model_params, str):
                params = ast.literal_eval(model_params)
            else:
                params = model_params

            best_models[segment_name] = {
                'model_type': 'Prophet',
                'model_name': model_name,
                'params': params,
                'mape': mape
            }
        elif 'SARIMAX' in model_name:
            model_type = 'SARIMAX'
            # model_params is tuple: ((p,d,q), (P,D,Q,s), (exog))
            order = model_params[0]
```

```
            seasonal = model_params[1]
            exog_cols = model_params[2]

            best_models[segment_name] = {
                'model_type': 'SARIMAX',
                'model_name': model_name,
                'order': order,
                'seasonal': seasonal,
                'exog':exog_cols,
                'mape': mape
            }

        elif 'SARIMA' in model_name:
            model_type = 'SARIMA'
            # model_params is tuple: ((p,d,q), (P,D,Q,s))
            order = model_params[0]
            seasonal = model_params[1]

            best_models[segment_name] = {
                'model_type': 'SARIMA',
                'model_name': model_name,
                'order': order,
                'seasonal': seasonal,
                'mape': mape
            }

        elif 'ARIMA' in model_name:
            model_type = 'ARIMA'
            # model_params is tuple: (p,d,q)
            order = model_params

            best_models[segment_name] = {
                'model_type': 'ARIMA',
                'model_name': model_name,
                'order': order,
                'seasonal': (0, 0, 0, 0),  # No seasonality
                'mape': mape
            }

        else:
            print(f"⚠ Unknown model type for {segment_name}: {model_name}")
            continue
        summary=f"{segment_name}-{model_name},{model_params},MAPE: {100*mape:.2f}%"
        best_models[segment_name]['summary']=summary
        if(verbose):
            print(f"✓ {summary}")

    return best_models



# ============================================================
# STEP 3: FIT MODELS ON FULL DATA
# ============================================================

def fit_sarimax_full(ts_data, order, seasonal, exog):
    """Fit SARIMA on full data"""

    y = ts_data.values
    exog= exog if exog.shape[1]>0 else None
    model = SARIMAX(
        y,
        order=order,
        seasonal_order=seasonal,
        exog=exog,
        enforce_stationarity=False,
        enforce_invertibility=False
    )

    fitted = model.fit(disp=False, maxiter=500)

    return fitted


def fit_prophet_full(ts_data, params,exog=None):
    """Fit Prophet on full data"""
    #ts_data and exog are for this segment, but have date in index.


    df_prophet = ts_data.copy().reset_index()
    df_prophet.columns = ['ds', 'y']

    # Create Prophet DataFrame
```

```
    prophet_df = pd.DataFrame({
        'ds': pd.to_datetime(ts_data.index),
        'y': ts_data.values
    })
    if(exog is not None):
        exog=exog.copy().reset_index().rename(columns={"index": "ds"})
        # Now both DataFrames have a ds column.
        prophet_df = prophet_df.merge(exog, on="ds", how="left")

    # Extract parameters with defaults
    model = Prophet(
        seasonality_mode=params.get('seasonality_mode', 'additive'),
        changepoint_prior_scale=params.get('changepoint_prior_scale', 0.05),
        seasonality_prior_scale=params.get('seasonality_prior_scale', 10),
        yearly_seasonality=params.get('yearly_seasonality', True),
        weekly_seasonality=params.get('weekly_seasonality', True),
        daily_seasonality=False
    )

    model.fit(prophet_df)

    return model
```

- forecast_df is a DataFrame containing the predicted future values for each segment. It's the output of the forecasting step.

In [119…

```
# ============================================================
# STEP 4: GENERATE FORECASTS
# ============================================================

def forecast_sarimax(fitted_model, steps, last_date, exog_future):
    """Generate SARIMA forecast"""

    if fitted_model.model.k_exog > 0:
        forecast_result = fitted_model.get_forecast(steps=steps, exog=exog_future)
    else:
        forecast_result = fitted_model.get_forecast(steps=steps)


    forecast_mean = forecast_result.predicted_mean
    conf_int = forecast_result.conf_int(alpha=0.05)

    future_dates = pd.date_range(
        start=last_date + pd.Timedelta(days=1),
        periods=steps,
        freq='D'
    )

    forecast_df = pd.DataFrame({
        'date': future_dates,
        'forecast': forecast_mean,
        'lower_ci': conf_int[:, 0],
        'upper_ci': conf_int[:, 1]
    })

    # Clip negatives
    forecast_df['forecast'] = forecast_df['forecast'].clip(lower=0)
    forecast_df['lower_ci'] = forecast_df['lower_ci'].clip(lower=0)

    return forecast_df


def forecast_prophet(fitted_model, steps):
    """Generate Prophet forecast"""

    future = fitted_model.make_future_dataframe(periods=steps, freq='D')
    prediction = fitted_model.predict(future)

    forecast_df = prediction[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(steps).copy()
    forecast_df.columns = ['date', 'forecast', 'lower_ci', 'upper_ci']
    forecast_df = forecast_df.reset_index(drop=True)

    # Clip negatives
    forecast_df['forecast'] = forecast_df['forecast'].clip(lower=0)
    forecast_df['lower_ci'] = forecast_df['lower_ci'].clip(lower=0)

    return forecast_df
```

`plot_full_actual_vs_predicted()`

In [119…

```
import pandas as pd
```

```python
import matplotlib.pyplot as plt
from prophet import Prophet
from statsmodels.tsa.statespace.sarimax import SARIMAXResults
import re


def plot_full_actual_vs_predicted(
    model,
    ts_data,
    exog=None,
    plot_exog_lines=True,
    model_type=None,
    summary=None

):
    """
    Plot actual vs predicted values over full data range.

    Parameters
    ----------
    model : fitted model object
        Prophet or statsmodels (ARIMA/SARIMAX/SARIMA)
    ts_data : pd.Series
        Actual time series with DatetimeIndex
    exog : pd.DataFrame or None
        Exogenous regressors aligned on index
    model_type : str or None
        One of {'prophet', 'sarimax', 'arima'}.
        If None, inferred automatically.
    title : str or None
        Plot title
    """

    # ------------------------------
    # Infer model type if not given
    # ------------------------------
    if model_type is None:
        if isinstance(model, Prophet):
            model_type = 'prophet'
        elif isinstance(model, SARIMAXResults):
            model_type = 'sarimax'
        else:
            raise ValueError("Unable to infer model type; please specify model_type")

    # ------------------------------
    # Prophet flow
    # ------------------------------
    if model_type.upper() == 'PROPHET':

        future = pd.DataFrame({
            'ds': pd.to_datetime(ts_data.index)
        })

        if exog is not None:
            exog_future = exog.copy().reset_index().rename(columns={"index": "ds"})
            future = future.merge(exog_future, on='ds', how='left')

        forecast = model.predict(future)

        pred_index = forecast['ds']
        y_pred = forecast['yhat']
        y_lower = forecast['yhat_lower']
        y_upper = forecast['yhat_upper']

    # ------------------------------
    # SARIMAX / ARIMA flow
    # ------------------------------
    elif model_type in ('SARIMAX', 'SARIMA','ARIMA'):

        if exog is not None and len(exog.columns)>0:
            forecast_res = model.get_prediction(
                start=0,
                end=len(ts_data) - 1,
                exog=exog
            )
        else:
            forecast_res = model.get_prediction(
                start=0,
                end=len(ts_data) - 1,
            )

        pred_df = forecast_res.summary_frame()
```

```
        pred_index = ts_data.index
        y_pred = pred_df['mean']
        y_lower = pred_df['mean_ci_lower']
        y_upper = pred_df['mean_ci_upper']

    else:
        raise ValueError(f"Unsupported model_type: {model_type}")

    # ------------------------------
    # Plot
    # ------------------------------
    plt.figure(figsize=(14, 6))

    plt.plot(ts_data.index, ts_data.values, label='Actual')
    plt.plot(pred_index, y_pred, label='Predicted')

    if y_lower is not None and y_upper is not None:
        plt.fill_between(
            pred_index,
            y_lower,
            y_upper,
            alpha=0.25,
            label='Uncertainty Interval'
        )
    # ------------------------------
    # Plot vertical lines for exog
    # ------------------------------
    exog_line_alpha=0.15
    exog_line_width=1
    first = True
    if plot_exog_lines and exog is not None and exog.shape[1] > 0:
        for col in exog.columns:
            active_dates = exog.index[exog[col] != 0]

            for d in active_dates:
                plt.axvline(
                    x=d,
                    linestyle='--',
                    color='red',
                    alpha=exog_line_alpha,
                    linewidth=exog_line_width,
                    label=f'Exogeneous event indicator' if first else None
                )
                first = False
    plt.text(
        0.01, 0.99,
        re.sub(r",(?![^()]*\))", "\n", summary),
        transform=plt.gca().transAxes,
        fontsize=10,
        verticalalignment='top',
        bbox=dict(
            boxstyle='round',
            facecolor='white',
            alpha=0.8
        )
    )
    plt.title(f'{summary.split("-", 1)[0]}: Actual vs Predicted (Uses {model_type.upper()})')
    plt.xlabel('Date')
    plt.ylabel('Value')
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_full_actual_vs_predicted2(
    model,
    ts_data,
    exog=None,
    model_type=None,
    summary=None
):
    print(f'***************{ summary}**')
    if 'ModelSpider' in summary:
        plot_full_actual_vs_predicted1(model,ts_data,exog,model_type,summary)
```

run_pipeline()

In [120…

```
# ============================================================
# STEP 5: MAIN PIPELINE
# ============================================================
import ast
def run_pipeline(modelDict, forecast_days=30):
    """
```

```
Complete pipeline:
1. Extract best models from modelDict
2. Prepare time series data
3. Fit on full data
4. Generate forecasts
"""

# Step 1: Extract best models
best_models = extract_best_models(modelDict,True)

# Storage for results
fitted_models = {}
forecasts = {}
time_series_data = {}
model_summaries = {}

#Build future exog data
steps=30
weekend_cols = [c for c in exog_df.columns if 'weekend' in c.lower()]
campaign_cols = [c for c in exog_df.columns if 'campaign' in c.lower()]
last_date = exog_df.index[-1]
future_dates = pd.date_range(
    start=last_date + pd.Timedelta(days=1),
    periods=steps,
    freq='D'
)
exog_future = pd.DataFrame(index=future_dates)

for col in weekend_cols:
    exog_future[col] = (exog_future.index.weekday >= 5).astype(int)
#Add campaign columns (all zeros)
for col in campaign_cols:
    exog_future[col] = 0
exog_future = exog_future[exog_df.columns]
assert exog_future.shape == (steps, exog_df.shape[1])
assert list(exog_future.columns) == list(exog_df.columns)
assert exog_future.isna().sum().sum() == 0




print("\n" + "="*70)
print("FITTING MODELS ON FULL DATA & GENERATING FORECASTS")
print("="*70)



for segment_name, model_info in best_models.items():
    print(f"\n[{segment_name}]")
    print("-" * 50)

    try:
        # Step 2: Prepare time series for this segment
        ts_data = timeSeriesFull_df[segment_name]

        last_date = ts_data.index[-1]

        # print(f"  Data: {n_obs} days ({ts_data['date'].min().date()} to {last_date.date()})")

        # Step 3: Fit model on full data
        model_type = model_info['model_type']

        if model_type in ['SARIMAX','SARIMA', 'ARIMA']:
            order = ast.literal_eval(model_info["order"])
            seasonal = ast.literal_eval(model_info["seasonal"])
            exogColumns = list(model_info.get('exog',[]))
            fitted = fit_sarimax_full(ts_data, order, seasonal, exog_df[exogColumns])

            print(f"  Fitted: {model_type}{order}x{seasonal}x{exogColumns}")
            print(f"  AIC: {fitted.aic:.1f}")

            # # Step 4: Forecast
            ##TODO: Forecast logic TBD
            # forecast_df = forecast_sarimax(fitted, forecast_days, last_date,exog_future[exogColumns])

            plot_full_actual_vs_predicted(
            model = fitted,
            ts_data = ts_data,
            exog=exog_df[exogColumns],
            model_type=model_type,
            summary=model_info["summary"])

        elif model_type == 'Prophet':
```

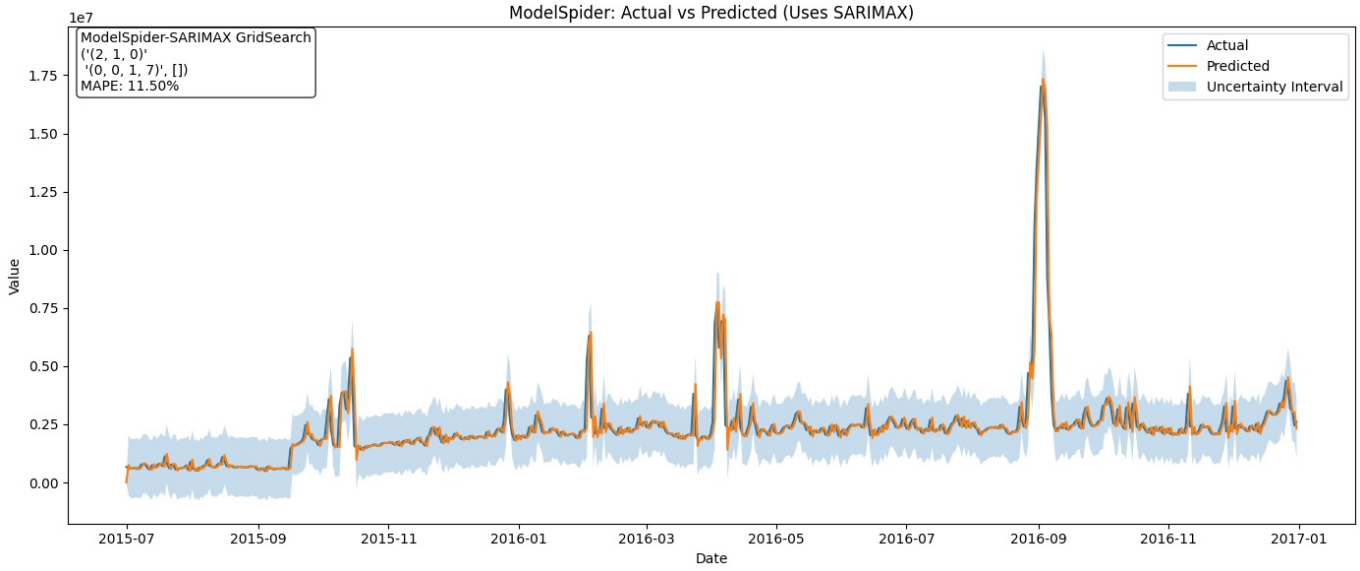
```
                params = model_info['params']
                exogColumns = list(params.get('exog',[]))
                fitted = fit_prophet_full(ts_data, exog=exog_df[exogColumns], params=params)
                print(f"  Fitted: Prophet")

                # Step 4: Forecast
                ##TODO: Forecast logic TBD
                # forecast_df = forecast_prophet(fitted, forecast_days,exog_future)

                plot_full_actual_vs_predicted(
                model = fitted,
                ts_data = ts_data,
                exog=exog_df[exogColumns],
                model_type=model_type,
                summary=model_info["summary"])

            # Store results
            fitted_models[segment_name] = {
                'model': fitted,
                'model_type': model_type,
                'last_date': last_date
            }

            # forecasts[segment_name] = forecast_df

            # model_summaries[segment_name] = {
            #     'model_type': model_type,
            #     'model_name': model_info['model_name'],
            #     'mape': model_info['mape'],
            #     'n_observations': n_obs,
            #     'last_date': last_date,
            #     'forecast_mean': forecast_df['forecast'].mean()
            # }

            # print(f"  ✓ Forecast generated: {forecast_days} days")
            # print(f"  Forecast mean: {forecast_df['forecast'].mean():,.0f}")

        except Exception as e:
            print(f"  ✗ Error: {str(e)}")
            import traceback
            traceback.print_exc()

    return {
        'best_models': best_models,
        'fitted_models': fitted_models,
        'forecasts': forecasts,
        'time_series_data': time_series_data,
        'model_summaries': model_summaries
    }
```

RUN THE PIPELINE

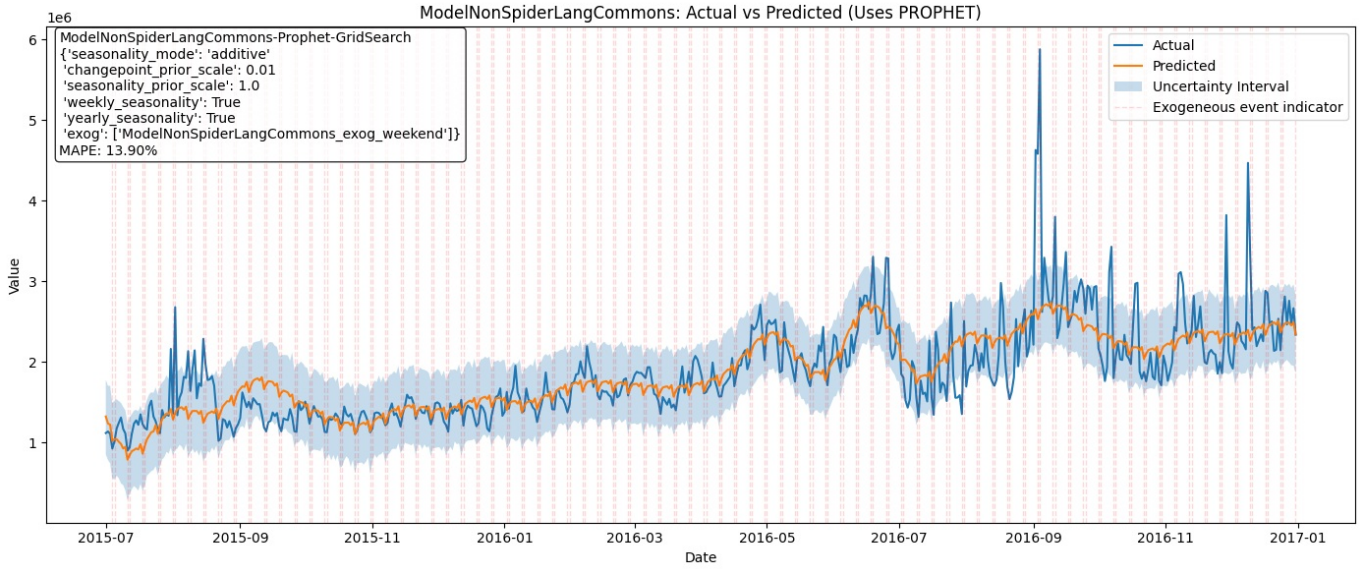
In [120…

```
# ============================================================
# RUN THE PIPELINE
# ============================================================

results = run_pipeline(modelDict, forecast_days=30)
fitted_models = results['fitted_models']
```

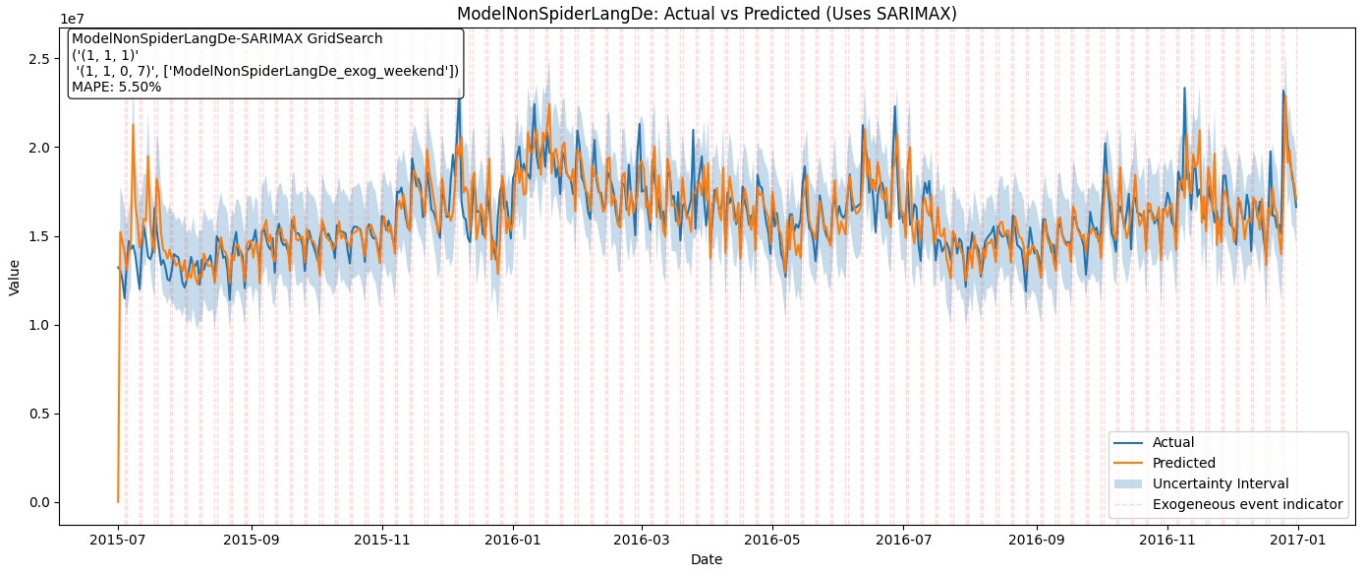
```
======================================================================
EXTRACTING BEST MODELS FROM modelDict
======================================================================

✓ ModelSpider-SARIMAX GridSearch,('(2, 1, 0)', '(0, 0, 1, 7)', []),MAPE: 11.50%
✓ ModelNonSpiderLangCommons-Prophet-GridSearch,{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.01,
'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpider
LangCommons_exog_weekend']},MAPE: 13.90%
✓ ModelNonSpiderLangDe-SARIMAX GridSearch,('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend']),M
APE: 5.50%
✓ ModelNonSpiderLangEn-Prophet-GridSearch,{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.1, 'seas
onality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangE
n_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']},MAPE: 4.80%
✓ ModelNonSpiderLangEs-Prophet-GridSearch,{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.001, 'se
asonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLan
gEs_exog_weekend']},MAPE: 7.50%
✓ ModelNonSpiderLangFr-SARIMAX GridSearch,('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend']),M
APE: 7.10%
✓ ModelNonSpiderLangJa-SARIMAX GridSearch,('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend']),M
APE: 6.40%
✓ ModelNonSpiderLangRu-SARIMA GridSearch,('(1, 0, 0)', '(1, 1, 0, 7)', []),MAPE: 7.10%
✓ ModelNonSpiderLangWww-Prophet-GridSearch,{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.05, 'se
asonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': False, 'exog': ['ModelNonSpiderLa
ngWww_exog_weekend']},MAPE: 15.10%
✓ ModelNonSpiderLangZh-SARIMAX GridSearch,('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend']),M
APE: 4.70%

======================================================================
FITTING MODELS ON FULL DATA & GENERATING FORECASTS
======================================================================

[ModelSpider]
--------------------------------------------------
  Fitted: SARIMAX(2, 1, 0)x(0, 0, 1, 7)x[]
  AIC: 16026.8
```



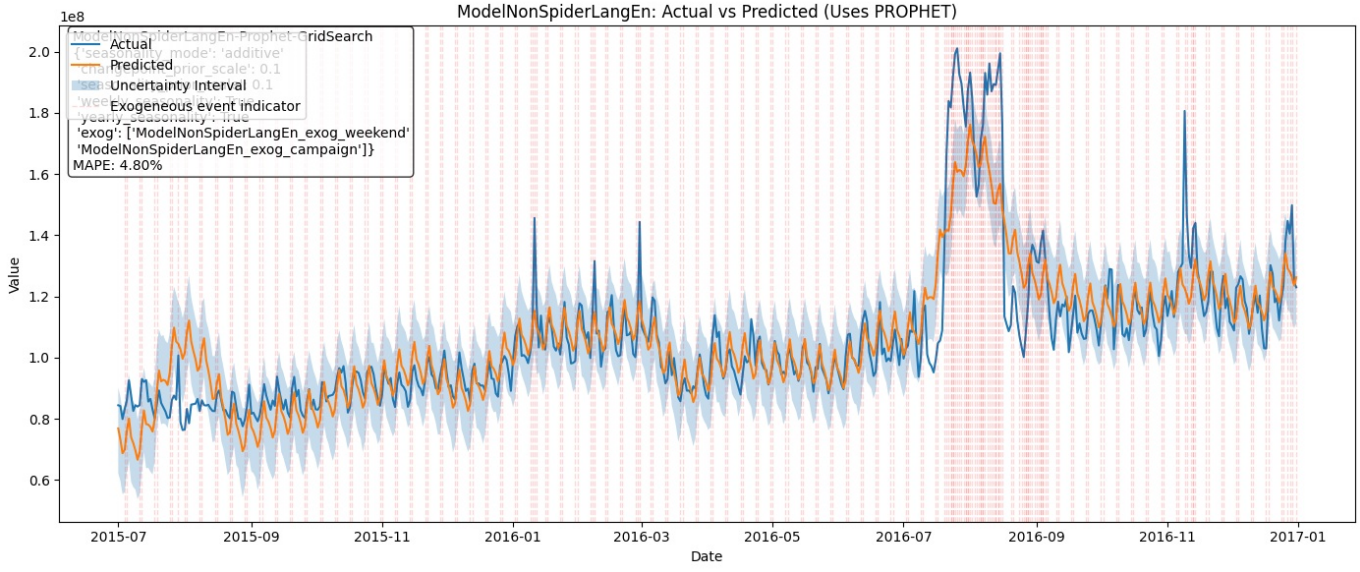

```
21:21:54 - cmdstanpy - INFO - Chain [1] start processing
21:21:54 - cmdstanpy - INFO - Chain [1] done processing
[ModelNonSpiderLangCommons]
--------------------------------------------------
  Fitted: Prophet
```




```
[ModelNonSpiderLangDe]
-----------------------------------------------------
  Fitted: SARIMAX(1, 1, 1)x(1, 1, 0, 7)x['ModelNonSpiderLangDe_exog_weekend']
  AIC: 16449.2
```



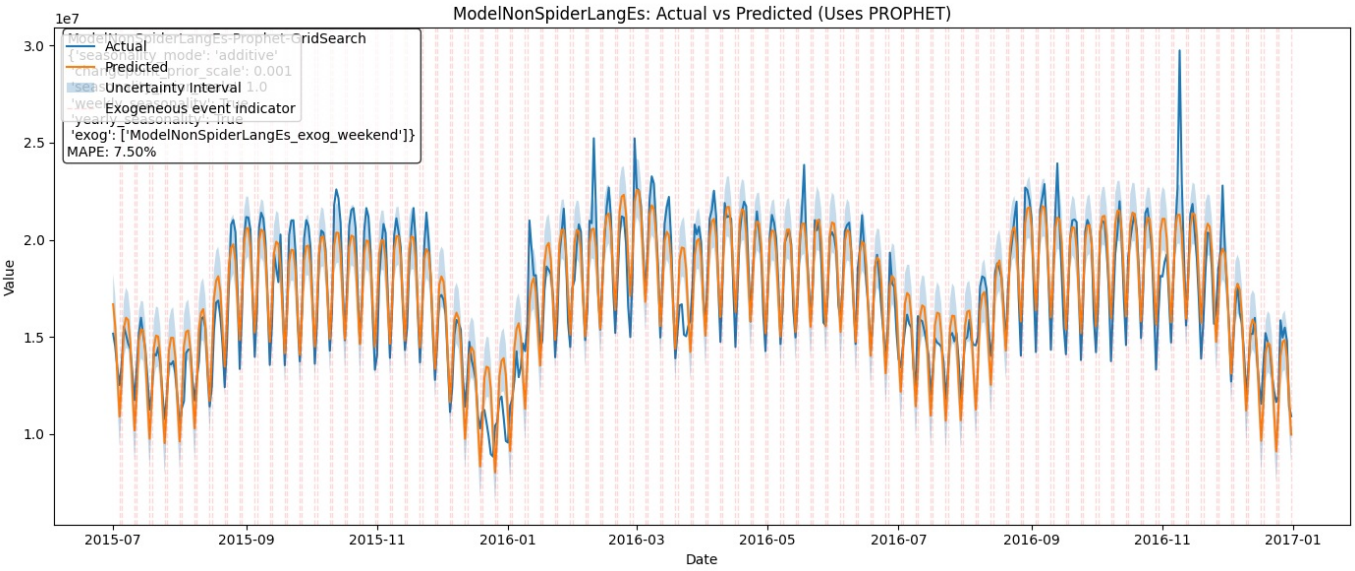

```
21:21:56 - cmdstanpy - INFO - Chain [1] start processing
21:21:56 - cmdstanpy - INFO - Chain [1] done processing
[ModelNonSpiderLangEn]
-----------------------------------------------------
  Fitted: Prophet
```




```
21:21:56 - cmdstanpy - INFO - Chain [1] start processing
21:21:56 - cmdstanpy - INFO - Chain [1] done processing
21:21:56 - cmdstanpy - ERROR - Chain [1] error: code '1' Operation not permitted
Optimization terminated abnormally. Falling back to Newton.
21:21:56 - cmdstanpy - INFO - Chain [1] start processing
[ModelNonSpiderLangEs]
-----------------------------------------------------
```

```
21:21:57 - cmdstanpy - INFO - Chain [1] done processing
```

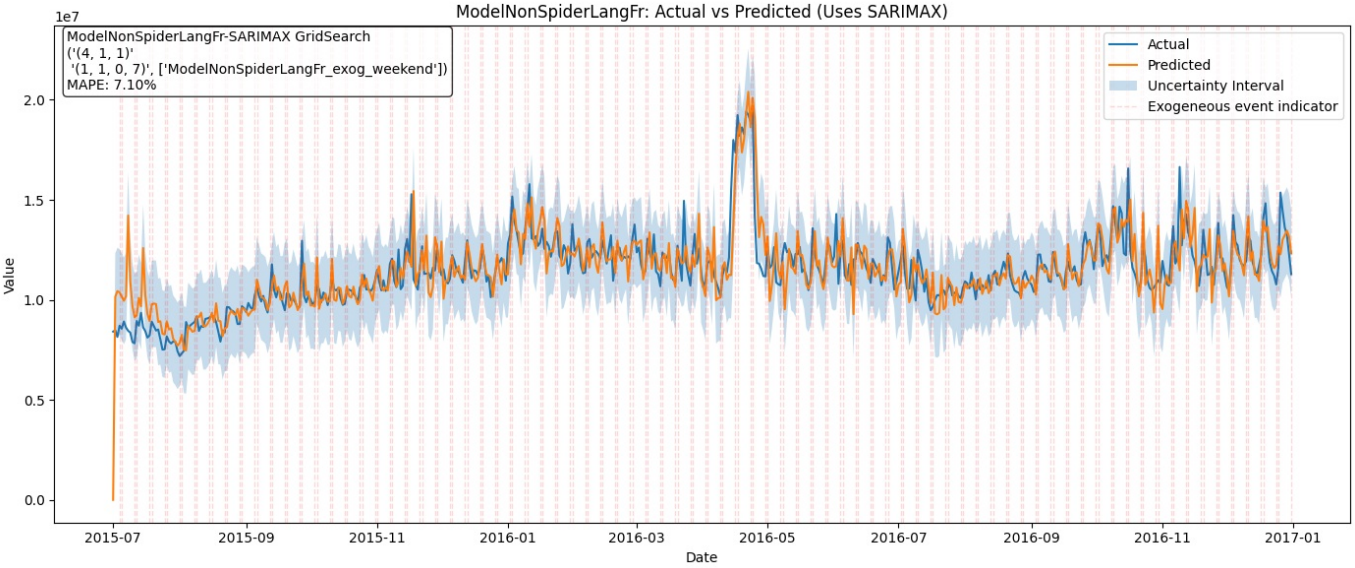
Fitted: Prophet



[ModelNonSpiderLangFr]

Fitted: SARIMAX(4, 1, 1)x(1, 1, 0, 7)x['ModelNonSpiderLangFr_exog_weekend']

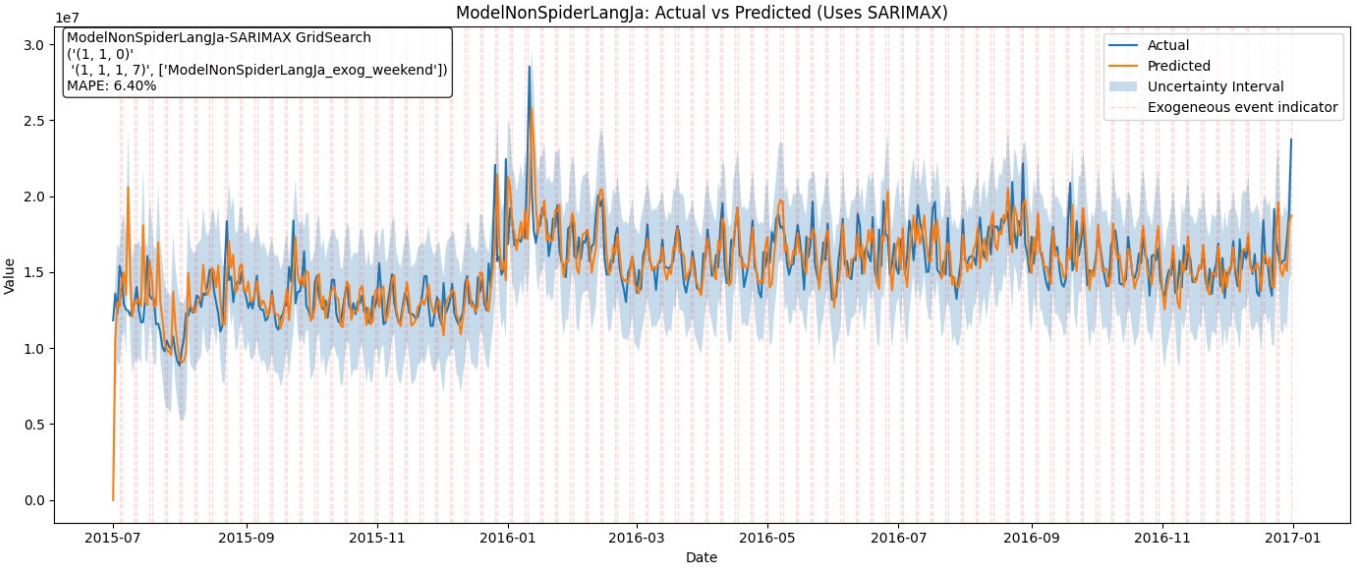
AIC: 16192.5



[ModelNonSpiderLangJa]

Fitted: SARIMAX(1, 1, 0)x(1, 1, 1, 7)x['ModelNonSpiderLangJa_exog_weekend']

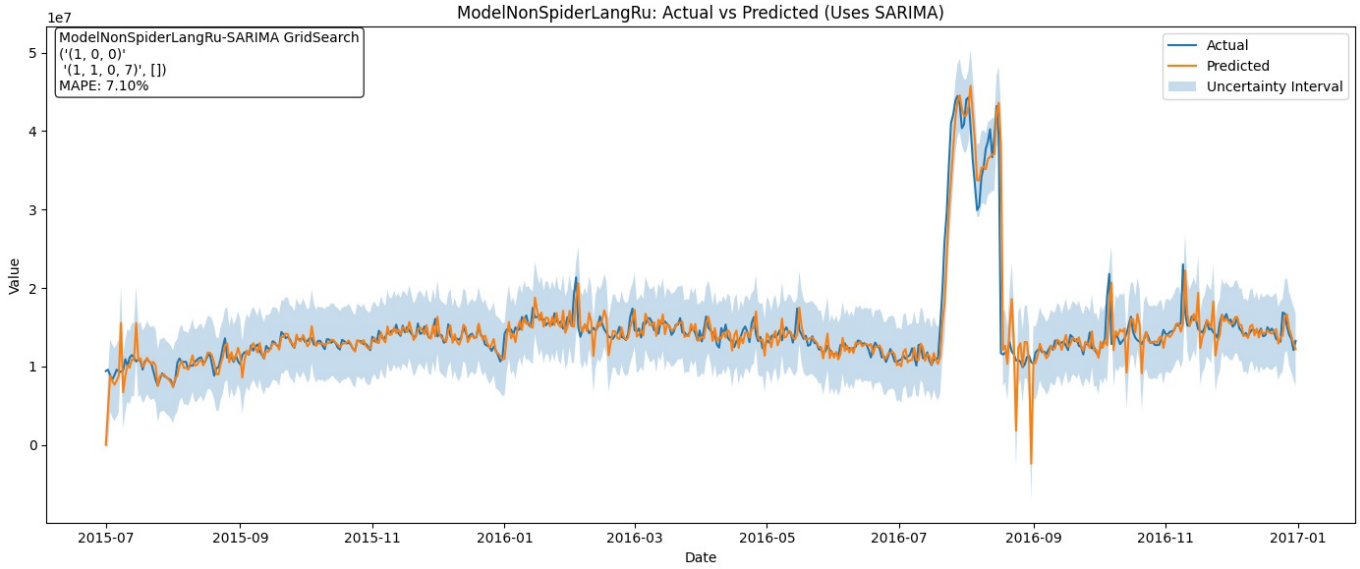
AIC: 16812.7



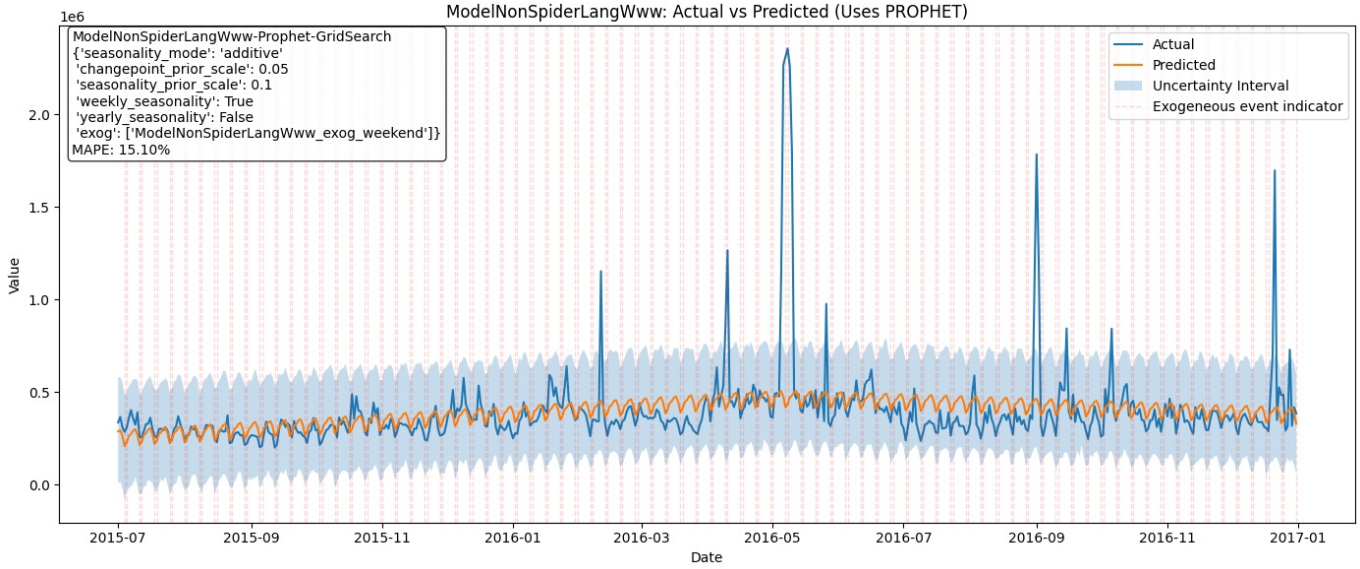
[ModelNonSpiderLangRu]

Fitted: SARIMA(1, 0, 0)x(1, 1, 0, 7)x[]

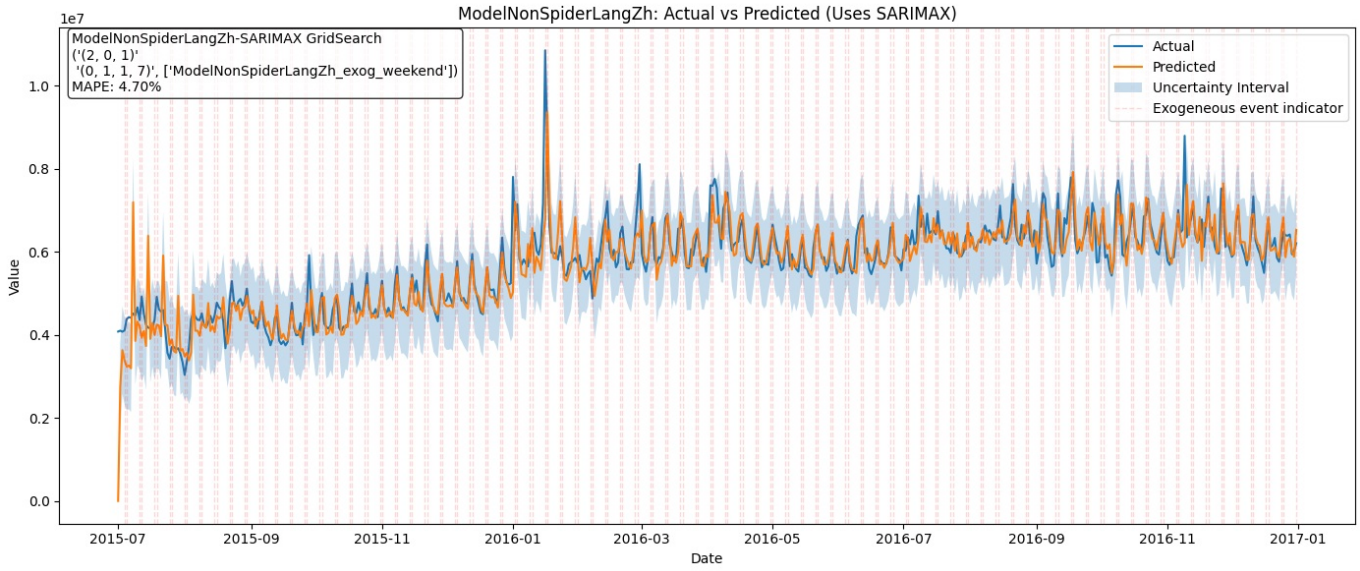
AIC: 17090.3




```
21:22:00 - cmdstanpy - INFO - Chain [1] start processing
21:22:00 - cmdstanpy - INFO - Chain [1] done processing
[ModelNonSpiderLangWww]
--------------------------------------------------
  Fitted: Prophet
```




```
[ModelNonSpiderLangZh]
--------------------------------------------------
  Fitted: SARIMAX(2, 0, 1)x(0, 1, 1, 7)x['ModelNonSpiderLangZh_exog_weekend']
  AIC: 15438.6
```



SAVE MODELS

In [120…

```
# ============================================================
# SAVE MODELS AND FORECASTS
# ============================================================
```

```python
import pickle
from datetime import datetime
import os

def save_results(fitted_models, forecasts=None, model_summaries=None, output_dir='./production_models'):
    """Save all results to disk"""

    os.makedirs(output_dir, exist_ok=True)
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

    # 1. Save fitted models
    models_file = f"{output_dir}/fitted_models_{timestamp}.pkl"

    # Extract just the model objects
    models_to_save = {
        seg: info['model']
        for seg, info in fitted_models.items()
    }

    with open(models_file, 'wb') as f:
        pickle.dump(models_to_save, f)
    print(f"✓ Saved models: {models_file}")

    # TODO Forecast and summaries.
    # # 2. Save all forecasts to single CSV
    # all_forecasts = []
    # for segment_name, forecast_df in forecasts.items():
    #     temp = forecast_df.copy()
    #     temp['segment'] = segment_name
    #     all_forecasts.append(temp)

    # combined_forecasts = pd.concat(all_forecasts, ignore_index=True)
    # forecasts_file = f"{output_dir}/all_forecasts_{timestamp}.csv"
    # combined_forecasts.to_csv(forecasts_file, index=False)
    # print(f"✓ Saved forecasts: {forecasts_file}")

    # # 3. Save model metadata
    # metadata_file = f"{output_dir}/model_metadata_{timestamp}.pkl"
    # metadata = {
    #     seg: {k: str(v) if isinstance(v, pd.Timestamp) else v
    #           for k, v in info.items() if k != 'model'}
    #     for seg, info in fitted_models.items()
    # }

    # with open(metadata_file, 'wb') as f:
    #     pickle.dump(metadata, f)
    # print(f"✓ Saved metadata: {metadata_file}")

    return timestamp

# Save everything
# save_timestamp = save_results(fitted_models, forecasts, model_summaries)
save_timestamp = save_results(fitted_models)
```

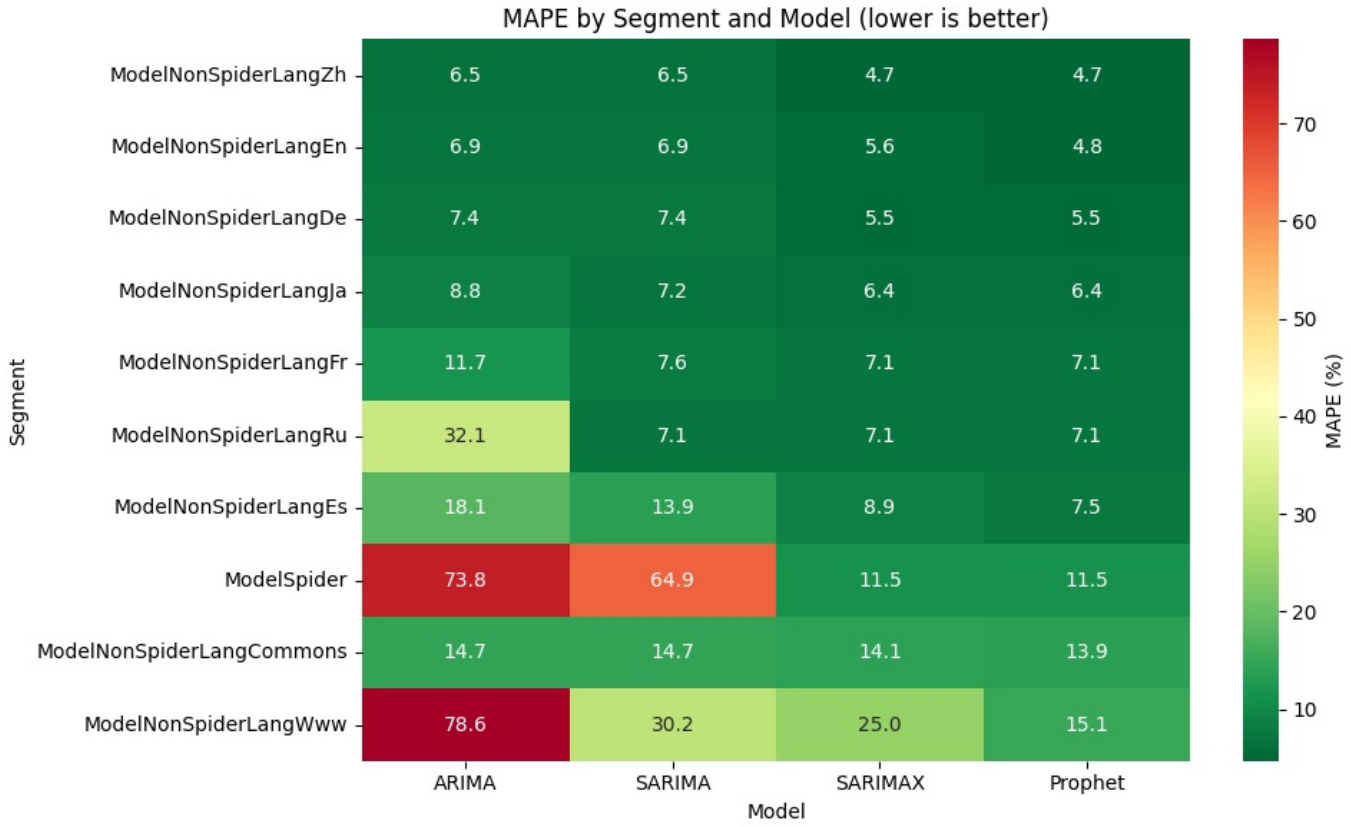
```
✓ Saved models: ./production_models/fitted_models_20260126_212202.pkl
```

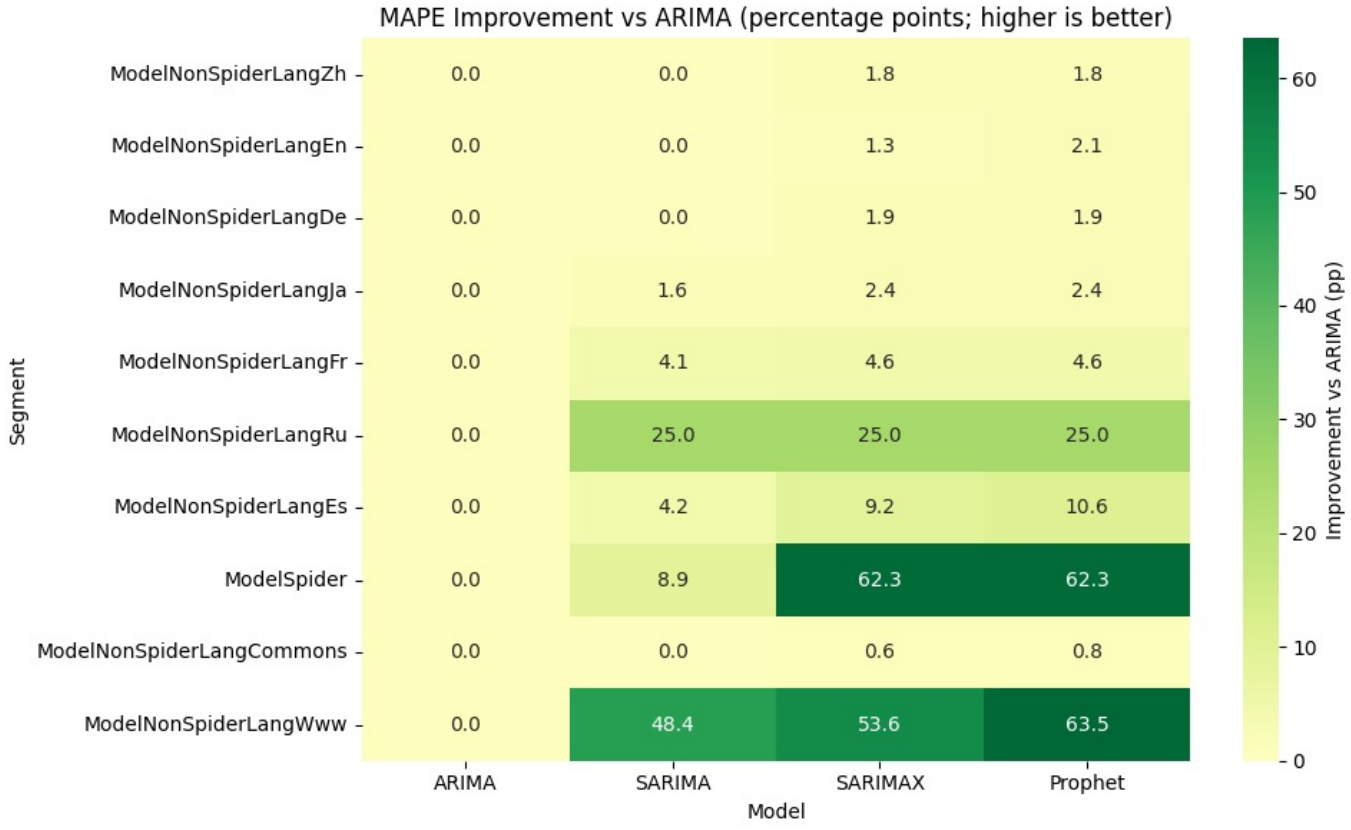
Consolidated Results of ALL models after ARIMA, SARIMA, SARIMAX and Prophet

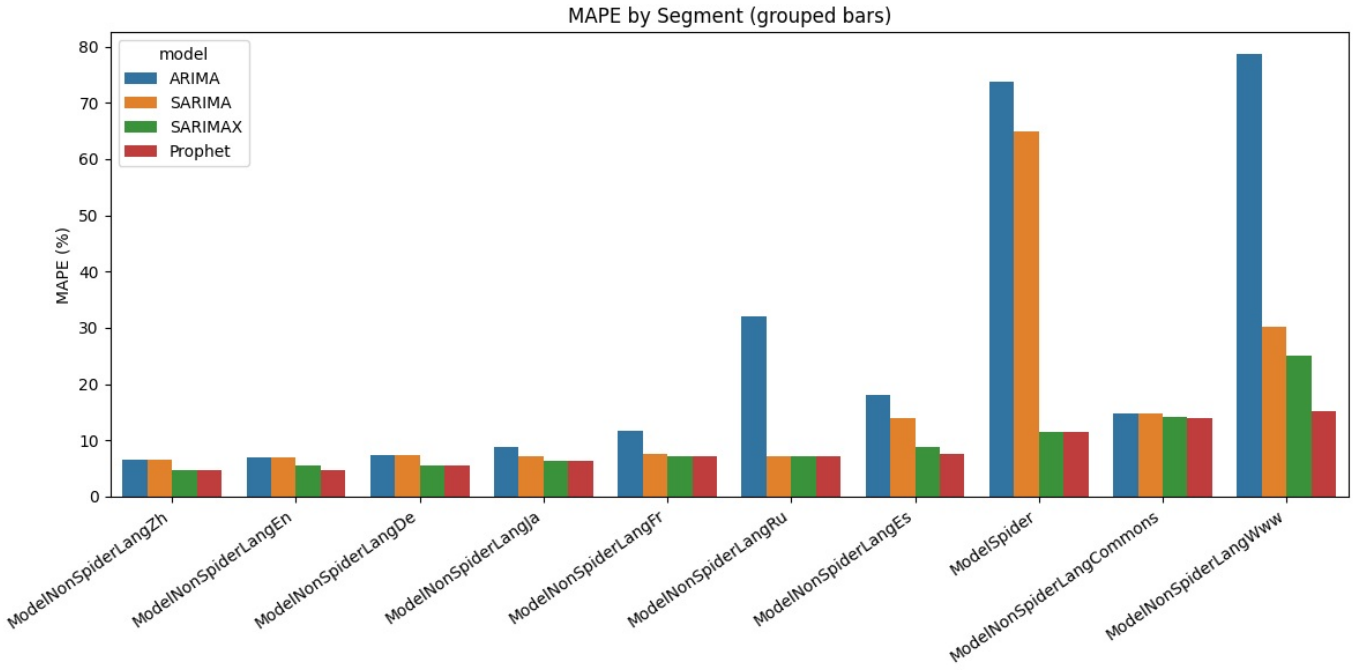
In [121…

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# -----------------------
# 1) Data (from your screenshots)
# -----------------------
data = [
    {"segment":"ModelNonSpiderLangZh", "ARIMA":6.5, "SARIMA":6.5, "SARIMAX":4.7, "Prophet":4.7},
    {"segment":"ModelNonSpiderLangEn", "ARIMA":6.9, "SARIMA":6.9, "SARIMAX":5.6, "Prophet":4.8},
    {"segment":"ModelNonSpiderLangDe", "ARIMA":7.4, "SARIMA":7.4, "SARIMAX":5.5, "Prophet":5.5},
    {"segment":"ModelNonSpiderLangJa", "ARIMA":8.8, "SARIMA":7.2, "SARIMAX":6.4, "Prophet":6.4},
    {"segment":"ModelNonSpiderLangFr", "ARIMA":11.7,"SARIMA":7.6, "SARIMAX":7.1, "Prophet":7.1},
    {"segment":"ModelNonSpiderLangRu", "ARIMA":32.1,"SARIMA":7.1, "SARIMAX":7.1, "Prophet":7.1},
    {"segment":"ModelNonSpiderLangEs", "ARIMA":18.1,"SARIMA":13.9,"SARIMAX":8.9, "Prophet":7.5},
    {"segment":"ModelSpider",          "ARIMA":73.8,"SARIMA":64.9,"SARIMAX":11.5,"Prophet":11.5},
    {"segment":"ModelNonSpiderLangCommons","ARIMA":14.7,"SARIMA":14.7,"SARIMAX":14.1,"Prophet":13.9},
    {"segment":"ModelNonSpiderLangWww", "ARIMA":78.6,"SARIMA":30.2,"SARIMAX":25.0,"Prophet":15.1},
]
df = pd.DataFrame(data).set_index("segment")
models = ["ARIMA","SARIMA","SARIMAX","Prophet"]
```

```
# ----------------------
# 2) Improvements vs ARIMA (percentage points)
# positive = better reduction vs ARIMA
# ----------------------
impr_pp = df[models].rsub(df["ARIMA"], axis=0)   # ARIMA - each model
for m in models:
    impr_pp[m] = df["ARIMA"] - df[m]
impr_pp = impr_pp[models]  # only model columns

# ----------------------
# 3) Plot: Heatmap of MAPE
# ----------------------
plt.figure(figsize=(10, 6))
sns.heatmap(df[models], annot=True, fmt=".1f", cmap="RdYlGn_r", cbar_kws={"label":"MAPE (%)"})
plt.title("MAPE by Segment and Model (lower is better)")
plt.ylabel("Segment")
plt.xlabel("Model")
plt.tight_layout()
plt.show()

# ----------------------
# 4) Plot: Heatmap of Improvement vs ARIMA (pp)
# ----------------------
plt.figure(figsize=(10, 6))
sns.heatmap(impr_pp, annot=True, fmt=".1f", cmap="RdYlGn", center=0,
            cbar_kws={"label":"Improvement vs ARIMA (pp)"})
plt.title("MAPE Improvement vs ARIMA (percentage points; higher is better)")
plt.ylabel("Segment")
plt.xlabel("Model")
plt.tight_layout()
plt.show()

# ----------------------
# 5) Plot: Grouped bar chart of MAPE
# ----------------------
long = df[models].reset_index().melt(id_vars="segment", var_name="model", value_name="mape")

plt.figure(figsize=(12, 6))
sns.barplot(data=long, x="segment", y="mape", hue="model")
plt.title("MAPE by Segment (grouped bars)")
plt.xticks(rotation=35, ha="right")
plt.ylabel("MAPE (%)")
plt.xlabel("")
plt.tight_layout()
plt.show()

# ----------------------
# 6) Optional: choose best (min MAPE) per segment and print summary
# ----------------------
best_model = df[models].idxmin(axis=1)
best_mape  = df[models].min(axis=1)
summary = pd.DataFrame({"best_model": best_model, "best_mape": best_mape,
                        "arima_mape": df["ARIMA"],
                        "improvement_pp": df["ARIMA"] - best_mape})
print(summary.sort_values("best_mape"))
```


MAPE by Segment and Model (lower is better)
Segment
ModelNonSpiderLangZh
ModelNonSpiderLangEn
ModelNonSpiderLangDe
ModelNonSpiderLangJa
ModelNonSpiderLangFr
ModelNonSpiderLangRu
ModelNonSpiderLangEs
ModelSpider
ModelNonSpiderLangCommons
ModelNonSpiderLangWww
6.5 6.5 4.7 4.7
6.9 6.9 5.6 4.8
7.4 7.4 5.5 5.5
8.8 7.2 6.4 6.4
11.7 7.6 7.1 7.1
32.1 7.1 7.1 7.1
18.1 13.9 8.9 7.5
73.8 64.9 11.5 11.5
14.7 14.7 14.1 13.9
78.6 30.2 25.0 15.1
ARIMA
SARIMA
SARIMAX
Prophet
Model
MAPE (%)
70
60
50
40
30
20
10


MAPE Improvement vs ARIMA (percentage points; higher is better)
Segment
ModelNonSpiderLangZh
ModelNonSpiderLangEn
ModelNonSpiderLangDe
ModelNonSpiderLangJa
ModelNonSpiderLangFr
ModelNonSpiderLangRu
ModelNonSpiderLangEs
ModelSpider
ModelNonSpiderLangCommons
ModelNonSpiderLangWww
0.0 0.0 1.8 1.8
0.0 0.0 1.3 2.1
0.0 0.0 1.9 1.9
0.0 1.6 2.4 2.4
0.0 4.1 4.6 4.6
0.0 25.0 25.0 25.0
0.0 4.2 9.2 10.6
0.0 8.9 62.3 62.3
0.0 0.0 0.6 0.8
0.0 48.4 53.6 63.5
ARIMA
SARIMA
SARIMAX
Prophet
Model
Improvement vs ARIMA (pp)
60
50
40
30
20
10
0



segment	best_model	best_mape	arima_mape	improvement_pp
ModelNonSpiderLangZh	SARIMAX	4.7	6.5	1.8
ModelNonSpiderLangEn	Prophet	4.8	6.9	2.1
ModelNonSpiderLangDe	SARIMAX	5.5	7.4	1.9
ModelNonSpiderLangJa	SARIMAX	6.4	8.8	2.4
ModelNonSpiderLangFr	SARIMAX	7.1	11.7	4.6
ModelNonSpiderLangRu	SARIMA	7.1	32.1	25.0
ModelNonSpiderLangEs	Prophet	7.5	18.1	10.6
ModelSpider	SARIMAX	11.5	73.8	62.3
ModelNonSpiderLangCommons	Prophet	13.9	14.7	0.8
ModelNonSpiderLangWww	Prophet	15.1	78.6	63.5

6. Actionable Insights & Recommendations

6.1 Summary of Observations from EDA

- We have 48874 unique pages across 144411 rows, as one page can be in different languages
- We have three access_types - all-access(51%), desktop(24%) and mobile(25%)
- In terms of origin of request, 24% originate as spider queries, while rest are all-agents
- The page name has useful info. We use a small embedding model to help understand the category. We can cluster pages and add another segmentation variable, along with what was done in this study
- Data across pages is NOT dependent on access_type as we can see that for same access type, randomly selected rows dont have similar pattern
- Click Count across pages **IS** dependent on access_origin as we can clearly see a difference.
  - In spider, the graph is having almost no seasonality or variation except the occasional spikes
  - In all_agents, data is much more varying across time
- Like language, domain too has a significant difference in distribution. It can be used as a criteria for segmenting and creating separate model based on domain. However later we identified that language and domain are dependent, so only language was sufficient for segmentation.
- language and domain are DEPENDENT, p-value:0.0. And, association is Strong(CramersV=1.00)
- access_type and domain are DEPENDENT' is True, p-value:6.432905337976179e-28. However, association is Negligible(CramersV=0.02)
- access_origin and domain are independent
- access_type and language are DEPENDENT' is True, p-value:9.96357592036832e-202. However, association is Negligible(CramersV=0.06)
- access_origin and language are DEPENDENT. p-value:1.1012230133778792e-41. However, association is Negligible(CramersV=0.04)
- access_origin and access_type are DEPENDENT. p-value:0.0. And association is Strong(CramersV=0.55)

6.2 Insights from EDA

- Visual inspection of plots revealed that
  - EN pages are quite different compared to other languages. There are dramatic ups and downs
  - DE pages are having much less spikes

    - Compared to langugage pages, www has almost NO spikes ! Even commons is also flattish
    - FR is flattish
- Overall conclusion was that we should build different models for different languages.
- We have figured out from all the EDA analysis that `access_origin` and `language`s are two good criteria for segmenting the data.
- As expected in a typical web based business ,there is a weekly cycle.
- Campaigns can have very significant (multifold) positive impact on page views.

6.4 Insights from Model Results

> **Multi-Series Forecasting Approach:** Each segment contains thousands of individual Wikipedia pages (e.g., ModelSpider has ~35,000 pages), making page-level modeling computationally prohibitive. We adopted an **aggregated time series approach**, where daily pageviews across all pages within a segment are summed to create a single representative time series per segment. This aggregation captures the collective behavior and trends of the segment while smoothing out noise from individual page fluctuations. The best-performing model parameters (identified via grid search on a sample page) were then applied to this aggregated series for final forecasting. This approach is appropriate when the objective is **segment-level demand forecasting** rather than individual page predictions.

> **Alternative Approaches Considered:** Beyond our chosen aggregated approach, other options included:
>
> - (1) multi-sample tuning—validating parameters across multiple random pages for robustness;
> - (2) global ML models—training a single LightGBM/XGBoost model on features from all series;
> - (3) clustering—grouping similar pages and modeling each cluster separately; and
> - (4) deep learning methods like DeepAR or N-BEATS designed for large-scale multi-series forecasting. The aggregated approach was selected for its simplicity and alignment with segment-level forecasting objectives.

Best MAPE Ranking

Rank	Segment Name	Best MAPE	Source
1	ModelNonSpiderLangZh	4.7%	SARIMAX GridSearch('(2, 0, 1)', '(0, 1, 1, 7)', ['ModelNonSpiderLangZh_exog_weekend'])
2	ModelNonSpiderLangEn	4.8%	Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.1, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEn_exog_weekend', 'ModelNonSpiderLangEn_exog_campaign']}
3	ModelNonSpiderLangDe	5.5%	SARIMAX GridSearch('(1, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangDe_exog_weekend'])
4	ModelNonSpiderLangJa	6.4%	SARIMAX GridSearch('(1, 1, 0)', '(1, 1, 1, 7)', ['ModelNonSpiderLangJa_exog_weekend'])
5	ModelNonSpiderLangFr	7.1%	SARIMAX GridSearch('(4, 1, 1)', '(1, 1, 0, 7)', ['ModelNonSpiderLangFr_exog_weekend'])
6	ModelNonSpiderLangRu	7.1%	SARIMA GridSearch('(1, 0, 0)', '(1, 1, 0, 7)', [])
7	ModelNonSpiderLangEs	7.5%	Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.001, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangEs_exog_weekend']}
8	ModelSpider	11.5%	SARIMAX GridSearch('(2, 1, 0)', '(0, 0, 1, 7)', [])
9	ModelNonSpiderLangCommons	13.9%	Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.01, 'seasonality_prior_scale': 1.0, 'weekly_seasonality': True, 'yearly_seasonality': True, 'exog': ['ModelNonSpiderLangCommons_exog_weekend']}
10	ModelNonSpiderLangWww	15.1%	Prophet-GridSearch{'seasonality_mode': 'additive', 'changepoint_prior_scale': 0.05, 'seasonality_prior_scale': 0.1, 'weekly_seasonality': True, 'yearly_seasonality': False, 'exog': ['ModelNonSpiderLangWww_exog_weekend']}

Best MAPE per Segment by Model Type (ARIMA / SARIMA / SARIMAX / Prophet)

Cells are colored by MAPE band; the winning model cell and the Winner column cell are highlighted in green.

≤10% 10–20% 20–40% >40% Winner highlight

Segment	ARIMA (best)	SARIMA (best)	SARIMAX (best)	Prophet (best)	Winner
ModelNonSpiderLangZh	6.5%	45.7%	4.7%	5.4%	4.7% SARIMAX
ModelNonSpiderLangEn	6.9%	88.5%	5.6%	4.8%	4.8% Prophet
ModelNonSpiderLangDe	7.4%	7.4%	5.5%	7.2%	5.5% SARIMAX
ModelNonSpiderLangJa	8.8%	7.2%	6.4%	6.8%	6.4% SARIMAX
ModelNonSpiderLangFr	11.7%	7.6%	7.1%	8.1%	7.1% SARIMAX
ModelNonSpiderLangRu	32.1%	7.1%	7.6%	9.3%	7.1% SARIMA
ModelNonSpiderLangEs	18.1%	13.9%	8.9%	7.5%	7.5% Prophet
ModelSpider	73.8%	64.9%	11.5%	53.1%	11.5% SARIMAX
ModelNonSpiderLangCommons	14.7%	29.4%	14.1%	13.9%	13.9% Prophet
ModelNonSpiderLangWww	78.6%	30.2%	25.0%	15.1%	15.1% Prophet

segment	best_model	best_mape	arima_mape	improvement_pp
ModelNonSpiderLangZh	SARIMAX	4.7	6.5	1.8
ModelNonSpiderLangEn	Prophet	4.8	6.9	2.1
ModelNonSpiderLangDe	SARIMAX	5.5	7.4	1.9
ModelNonSpiderLangJa	SARIMAX	6.4	8.8	2.4
ModelNonSpiderLangFr	SARIMAX	7.1	11.7	4.6
ModelNonSpiderLangRu	SARIMA	7.1	32.1	25.0
ModelNonSpiderLangEs	Prophet	7.5	18.1	10.6
ModelSpider	SARIMAX	11.5	73.8	62.3
ModelNonSpiderLangCommons	Prophet	13.9	14.7	0.8
ModelNonSpiderLangWww	Prophet	15.1	78.6	63.5

6.5 **Recommendations**

Customers can use this model reliably to forecast what would be the expected page views based on language and region

6.6 Questionnaire:

1. Defining the problem statements and where can this and modifications of this be used?

- The the goal of the case study is to predict the page view per segment (segment could be language or language + some more criteria, which will be determined during the course of the study)
- Final aim of user is to answer this question
  - **If I place my ad on French pages, how many page views can I expect**,
  - **If I place my ad on English pages, how many page views can I expect**

---

2. Write 3 inferences you made from the data visualizations

- Click Count across pages **IS** dependent on access_origin as we can clearly see a difference.
  - In spider, the graph is having almost no seasonality or variation except the occasional spikes
  - In all_agents, data is much more varying across time
- Visual inspection of plots revealed that
  - EN pages are quite different compared to other languages.
  - DE pages are having much less spikes
  - www has almost NO spikes !
  - commons is also flattish
  - Conclusion was that we should segment data based on language
- 100% pages with mediawiki domain belong to www (language identifier)
- 100% pages with wikimedia domain belong to commons (language identifier)

---

3. What does the decomposition of series do?

- Decompose the time series into trend, seasonality and residuals component series. Each of the component series can then be plotted and analyzed.

---

4. What level of differencing gave you a stationary series?

- One

---

5. Difference between arima, sarima & sarimax.

- SARIMAX offers ARIMA + Seasonal components(P,D,Q,s) and Exogeneous varlables.
- SARIMA does not offer Exogeneous varlables.
- ARIMA does not offer seasonal components or exogeneous regressors.

---

6. Compare the number of views in different languages

- commons 10477
- de 18438
- en 24010
- es 14041
- fr 17761

- ja 20340
- ru 14990
- www 7251
- zh 17103

7. What other methods other than grid search would be suitable to get the model for all languages?

- pmdarima.arima.auto_arima